MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITE TUNIS EL MANAR
INSTITUT SUPERIEUR D'INFORMATIQUE

RAPPORT DE STAGE DE FIN D'ETUDES

Présenté en vue de l'obtention du

Diplôme National de Licence en Sciences de l'Informatique

Spécialité : Génie Logiciel et Systèmes d'Information

Tunisian Legal AI

Retrieval-Augmented Generation & Domain-Adaptive Fine-Tuning

Par

Louai BOUBAKER

Encadrant professionnel : Madame Dhikra Ben Mahmoud
Encadrant académique : Madame Bakhta Haouari

Réalisé au sein de

Année universitaire **2025/2026**

Company Supervisor

I authorize the student to submit their internship report for defense.

E-TAFAKNA
19,Rue Mahmoud Chebaane
2037 Ennassr 2
MF:1771137N/A/M/000

Academic Supervisor (ISI)

I authorize the student to submit their internship report for defense.

Signature :

In memory of the children of Palestine, Lebanon, Congo, and Sudan taken by war before they had the chance to live.

To every innocent child who lost their life in the war in Palestine, Lebanon, the Democratic Republic of Congo, and Sudan. may their memory never fade, and may the world never forget their faces, and may allah bless their souls.

To those who are still here, still standing, still holding on through everything they have endured, you are not alone. We see you. We think of you. We are here for you.

Stay strong.

"You cannot witness injustice and remain silent."

To my parents,

for everything you sacrificed, everything you gave,

and for never once doubting me.

To my family,

for the warmth and support that carried me

through every difficult moment.

To my friends who stayed,

through the good days and the hard ones,

through the stress and the laughter

thank you for being there.

This work is dedicated to all of you.

Acknowledgments

I would like to express my sincere gratitude to the people who guided and supported me throughout this project.

To **Madame Dhikra Ben Mahmoud**, my professional supervisor at E-Tafakna, for her trust, her availability, and the guidance she provided throughout the internship. Her practical insight and encouragement were invaluable in shaping this work.

To **Madame Bakhta Haouari**, my academic supervisor at ISI, for her careful follow-up, her constructive feedback, and for always pointing me in the right direction whenever I needed it.

To **Madame Norchen Mezni**, CEO of E-Tafakna, for welcoming me into her company and giving me the opportunity to work on a meaningful and technically challenging project. Her vision for the company made this internship an experience I am truly grateful for.

To each of you, thank you.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

ATI Agence Tunisienne d'Internet

BERT Bidirectional Encoder Representations from Transformers

CPU Central Processing Unit

CRISP-DM Cross-Industry Standard Process for Data Mining

DSR Design Science Research

FAISS Facebook AI Similarity Search

GGUF GPT-Generated Unified Format

GPT Generative Pre-trained Transformer

GPTQ GPT Post-Training Quantization

GPU Graphics Processing Unit

HTTP Hypertext Transfer Protocol

JORT Journal Officiel de la République Tunisienne

LLM Large Language Model

LoRA Low-Rank Adaptation

MENA Middle East and North Africa

NLP Natural Language Processing

OCR Optical Character Recognition

PDF Portable Document Format

PEFT Parameter-Efficient Fine-Tuning

PLE Per-Layer Embeddings

Q&A Question and Answer

QLoRA Quantized Low-Rank Adaptation

RAG Retrieval-Augmented Generation

RAM Random Access Memory

RRF Reciprocal Rank Fusion

SLM Small Language Model

SME Small and Medium-sized Enterprise

TDSP Team Data Science Process

VRAM Video Random Access Memory

General Introduction

Legal services have historically been expensive, complex, and difficult to access, particularly for individuals and small businesses navigating an increasingly regulated environment. The rapid development of Artificial Intelligence (AI) is beginning to change this : language models can now engage in open-ended dialogue about legal matters, interpret regulations, and assist users in understanding their rights and obligations. For a LegalTech company like E-Tafakna, which already offers a comprehensive platform for contract management and legal document processing across the Middle East and North Africa (MENA) region, this evolution opens the door to a qualitative improvement in the intelligence it can offer to its users.

Two obstacles, however, stand between the current state of the art and a genuinely useful Tunisian legal assistant. First, general-purpose language models were not trained on Tunisian legislation : they lack the specific vocabulary, article structure, and regulatory context of local law, and their responses on Tunisian legal matters are correspondingly unreliable. Second, the legal sensitivity of the data makes external cloud-based inference unacceptable. E-Tafakna's planned deployment at Agence Tunisienne d'Internet (ATI) requires a system that runs entirely on local infrastructure, with no data leaving the controlled environment during operation.

This graduation project addresses both obstacles simultaneously. We design, build, and evaluate a domain-specific legal AI assistant that combines a language model fine-tuned on Tunisian legal texts with a Retrieval-Augmented Generation (RAG) pipeline that retrieves relevant legal articles at inference time, all deployed on-premises with no external dependencies at query time.

This report is organized in four chapters. Chapter I situates the project within its institutional and methodological context : it presents E-Tafakna, analyses the limitations of its current AI agents, defines the problem and the proposed solution, and justifies the adoption of Design Science Research (DSR) as the guiding methodology. Chapter II reviews the scientific and technical literature relevant to the project, covering RAG architectures, parameter-efficient fine-tuning, and on-premises deployment of large language models. Chapter III describes the complete design and development of the artifact : the seven-phase offline preparation pipeline, the real-time query architecture, the fine-tuning of two candidate models on a curated Tunisian legal dataset, and their quantized local deployment. Chapter IV closes the DSR cycle by demonstrating the system in operation and evaluating it against the three objectives defined in Chapter III, covering training outcomes, retrieval quality, expert assessment, and on-premises compliance.

Chapitre I

PROJECT GENERAL FRAMEWORK

1 Introduction

This chapter provides an overview of the context surrounding our graduation project, beginning with a presentation of the host organization, E-TAFAKNA, where we developed our solution. We will analyze the specific issues that motivated this initiative, then explain the original idea we proposed. Finally, we will summarize the detailed procedures we followed to ensure the smooth progress and successful completion of the project.

2 Host Organization Presentation

Founded in 2022, E-Tafakna stands as a trailblazing Tunisian LegalTech startup transforming legal document management across the MENA region [1]. Through its intuitive web and mobile platform, the company enables users to seamlessly create, collaborate on, sign, and monitor legal documents from any location, effectively digitizing the entire contract lifecycle, as shown in Figure 1.1.

Fig. 1.1 : E-Tafakna Company Logo [2]

E-Tafakna distinguishes itself through a deeply user-centric philosophy. The platform offers over 100 pre-designed contract templates covering everything from non-disclosure agreements to employment contracts, coupled with real-time document status tracking and trilingual support in Arabic, French, and English. This strategic combination of simplicity and robust functionality democratizes access to professional legal services, making them affordable and accessible for freelancers, entrepreneurs, and Small and Medium-sized Enterprise (SME)s. Recognized with the prestigious StartUp'Act label and winner of the HiiL Innovating Justice Challenge 2024, E-Tafakna continues to solidify its position as a regional leader in legal innovation [1].

3 General Project Overview

3.1 Project Framework

Legal services have long been associated with complexity, high costs, and limited accessibility, particularly for individuals, freelancers, and small businesses navigating an increasingly regulated environment. The digitalization of these services has become a strategic priority, as modern users expect instant, accurate, and accessible legal support. In this context, AI is emerging as a transformative force, enabling platforms to automate complex legal tasks, centralize expertise, and deliver seamless experiences across multiple channels.

E-Tafakna addresses this need through a comprehensive legal platform that covers the entire contract lifecycle. Its services span contract template creation, professional consulting, document management, real-time contract collaboration, and electronic signature. To further enhance this offering, E-Tafakna has integrated two intelligent AI agents : Elyssa, a smart legal co-pilot focused on document drafting and compliance, and Kahina, an intelligent legal avatar designed to explain laws, contracts, and regulations through natural conversation. Together, these agents position E-Tafakna as a pioneer in AI-powered LegalTech across the MENA region.

3.2 Study of Existing Solutions

E-Tafakna currently offers two AI-powered agents to assist users with their legal needs.

Elyssa is a smart legal co-pilot designed to assist users throughout the document drafting and review process. It provides :

- **Legal Chatbot :** Instant legal assistance through a conversational interface.
- **Clause Assistant :** Smart generation of contract clauses tailored to the user's context.
- **Contract Analyzer :** Automatic identification of risks and missing clauses within uploaded documents.
- **Compliance Scorer :** Rates contracts against legal standards and suggests targeted improvements.

Kahina is an intelligent legal avatar built to make legal knowledge accessible to SMEs, freelancers, and startups. It provides :

- **Legal Intelligence :** Helps users understand legal concepts, procedures, and applicable regulations.

- **Virtual Assistant :** Delivers contextual guidance tailored to the user's specific questions.
- **Multilingual Support :** Communicates fluently in Arabic, French, and English.
- **Conversational Avatar :** Simulates human-like legal consultations for a more natural user experience.

Together, Elyssa and Kahina form the intelligent core of E-Tafakna's platform, each serving a distinct but complementary role : Elyssa focuses on document-level assistance, while Kahina handles broader legal understanding and user guidance [2].

3.3 Critical Analysis of the Existing System

While Elyssa and Kahina represent meaningful advances in accessible legal assistance, a critical analysis reveals that the current system still falls short of its full potential. Kahina's conversational avatar interface has already addressed the interaction barriers previously faced by users, offering multilingual dialogue in Arabic, French, and English. However, two fundamental limitations remain unresolved :

- **Lack of Tunisian Legal Specificity :** Both Elyssa and Kahina rely on general-purpose language models that were not trained on Tunisian legal texts. As a result, responses often lack the precision required for local legal matters, missing the nuances of Tunisian legislation, procedures, and regulatory frameworks.
- **Dependency on External Cloud Services :** The current AI infrastructure relies on external cloud-based Application Programming Interface (API)s, which raises concerns around data sovereignty, response latency, and compliance with Tunisian data regulations. E-Tafakna's planned deployment at ATI Tunisie requires a fully on-premises solution with no external dependencies, ensuring that all legal data remains within a secure and controlled local infrastructure, which is a requirement the current system cannot fulfill.

These two limitations, namely insufficient Tunisian legal knowledge and reliance on external cloud services, define the core technical challenge that our project aims to address.

3.4 Problem Statement

The analysis of E-Tafakna's existing AI agents reveals a clear and twofold challenge. First, both Elyssa and Kahina operate on general-purpose language models that lack the domain-specific knowledge required to handle Tunisian legal content with the accuracy and reliability that legal matters demand. Second, the current architecture depends on external cloud services, making it incompatible with the on-premises deployment requirements of ATI Tunisie, where data sovereignty and infrastructure autonomy are non-negotiable constraints.

This raises a central research question : *How can we build a legal AI assistant that is both deeply grounded in Tunisian law and fully deployable within a secure, self-contained local infrastructure ?*

Addressing this question requires tackling three concrete technical problems :

- **Domain Adaptation :** General-purpose language models must be specialized through fine-tuning on a curated corpus of Tunisian legal texts, enabling them to understand the specific terminology, structure, and logic of local legislation.
- **Knowledge Retrieval :** Fine-tuning alone is insufficient for handling the breadth and evolving nature of legal knowledge. A RAG pipeline must be designed to allow the model to dynamically retrieve relevant legal documents at inference time, improving both accuracy and up-to-date coverage.
- **Local Deployment :** The resulting system must be optimized to run efficiently on on-premises infrastructure at ATI Tunisie, without relying on any external API or cloud service, ensuring full data sovereignty and regulatory compliance.

3.5 Proposed Solution

To address the identified challenges, we propose the development of a domain-specific legal AI assistant built around two complementary components : a fine-tuned Small Language Model (SLM) or Large Language Model (LLM) specialized in Tunisian law, and a RAG pipeline designed to dynamically incorporate legal knowledge at inference time. The resulting system will be deployed entirely on-premises at ATI Tunisie, with no dependency on external cloud services.

Component 1: Fine-Tuned Language Model

Rather than relying on a generic pre-trained model, we will fine-tune a base SLM or LLM on a curated corpus of Tunisian legal documents, including legislation, regulatory texts, and legal procedures. This supervised fine-tuning process will enable the model to internalize the specific vocabulary, structure, and reasoning patterns of Tunisian law, producing responses that are more accurate, contextually appropriate, and legally sound than those of a general-purpose model.

Component 2: Retrieval-Augmented Generation Pipeline

Fine-tuning alone cannot capture the full breadth of Tunisian legal knowledge, nor can it account for legislative updates over time. To overcome this, we will design a RAG pipeline that connects the language model to a vector database populated with indexed legal documents. At inference time, the system will retrieve the most relevant legal excerpts based on the user's query and inject them into the model's context, significantly improving the precision and coverage of generated responses.

Component 3: On-Premises Deployment at ATI Tunisie

The complete system, including the language model, RAG pipeline, and vector database, will be deployed locally on ATI Tunisie's infrastructure using Ollama as the local LLM inference framework. This architecture guarantees full data sovereignty, eliminates latency caused by external API calls, and ensures compliance with Tunisian data regulations. All legal data processed by the system remains within ATI's secure on-premises environment.

Expected Outcomes

This solution is expected to deliver the following improvements over the existing system :

- **Higher Legal Accuracy :** Domain-specific fine-tuning ensures the model understands and correctly applies Tunisian legal concepts, reducing erroneous or imprecise responses.
- **Up-to-Date Legal Knowledge :** The RAG pipeline allows the system to retrieve current legal documents dynamically, keeping responses relevant even as legislation evolves.
- **Full Data Sovereignty :** On-premises deployment at ATI Tunisie ensures that no legal data leaves the organization's infrastructure, meeting both internal security requirements and national data regulations.
- **Seamless Integration :** The system is designed to integrate with E-Tafakna's existing platform, enhancing the capabilities of both Elyssa and Kahina without requiring changes to the user-facing interface.

Together, these components form a cohesive and technically grounded solution that directly addresses each of the three problems identified in the problem statement, transforming E-Tafakna's legal AI offering into a specialized, autonomous, and locally sovereign system.

4 Adopted Methodology

Choosing the right methodology is one of the most consequential decisions in any technical project. It determines not only how the work is organized, but how decisions are made, how progress is measured, and how results are ultimately validated. In the context of an AI-driven system designed for a real-world legal environment, this choice carries particular weight.

4.1 Methodological Framework Selection

A wide range of methodological frameworks exists for guiding the development of software and intelligent systems. These can be broadly grouped into two families : traditional project management methodologies, and data science or research-oriented methodologies.

Traditional project management methodologies, such as Scrum and Kanban, are general-purpose frameworks focused on how a team organizes and delivers work. Scrum structures development into fixed-length iterations called sprints, with regular reviews and adaptation cycles, while Kanban emphasizes continuous flow and visual task management. Both are well-suited for software delivery projects where requirements are relatively stable and the primary challenge is coordination and delivery pace. However, they do not address the specific challenges inherent to AI development, they provide no guidance on data collection, model training, evaluation of predictive performance, or the construction and validation of a research artifact. Applying them alone to a project involving fine-tuning a language model and building a RAG pipeline would leave the most critical technical and scientific dimensions of the work entirely unguided.

Data science and research-oriented methodologies, by contrast, are designed precisely for projects where the core challenge is building, evaluating, and deploying an intelligent system or a research artifact. In the field of data science and intelligent systems design, several such frameworks have been proposed. These range from traditional process-oriented approaches, such as Cross-Industry Standard Process for Data Mining (CRISP-DM) [3] and Team Data Science Process (TDSP) [4], to research-centered paradigms such as DSR [5], [6]. Each addresses a different type of project and carries distinct strengths and limitations that must be carefully weighed against the specific requirements at hand.

Our project presents a specific set of requirements that constrain this choice :

- **Domain Specialization :** Fine-tuning a language model on Tunisian legal texts requires multiple cycles of data preparation, training, and evaluation before acceptable performance is achieved.
- **Pipeline Complexity :** Integrating a fine-tuned model with a RAG pipeline and a vector database introduces interdependencies between components that must be developed, tested, and refined in coordination.
- **Deployment Constraints :** The on-premises deployment at ATI Tunisie imposes strict infrastructure requirements that must be addressed from the earliest stages of the project.
- **Artifact-Centered Development :** The primary deliverable of this project is a concrete technological artifact, a specialized legal chatbot, whose utility must be rigorously designed, built, and evaluated within an academic framework.

The following sections present an overview of each candidate methodology and the rationale behind our final selection.

4.2 Overview of CRISP-DM

CRISP-DM [3] is a traditional process model that emerged in the late 1990s to standardize the practice of data mining, the automated extraction of patterns and knowledge from large datasets. It was conceived as an industry-neutral framework applicable across sectors and tools, structured around six sequential phases [3] :

- **Business Understanding :** Defining project objectives and requirements from a business perspective.
- **Data Understanding :** Collecting initial data and identifying quality issues and relevant subsets.
- **Data Preparation :** Cleaning, transforming, and structuring data for modeling.
- **Modeling :** Applying modeling techniques and calibrating parameters for optimal results.

- **Evaluation :** Assessing the model against business objectives.
- **Deployment :** Integrating the model into operational systems.

Figure 1.2 illustrates these six phases and the iterative nature of the methodology.

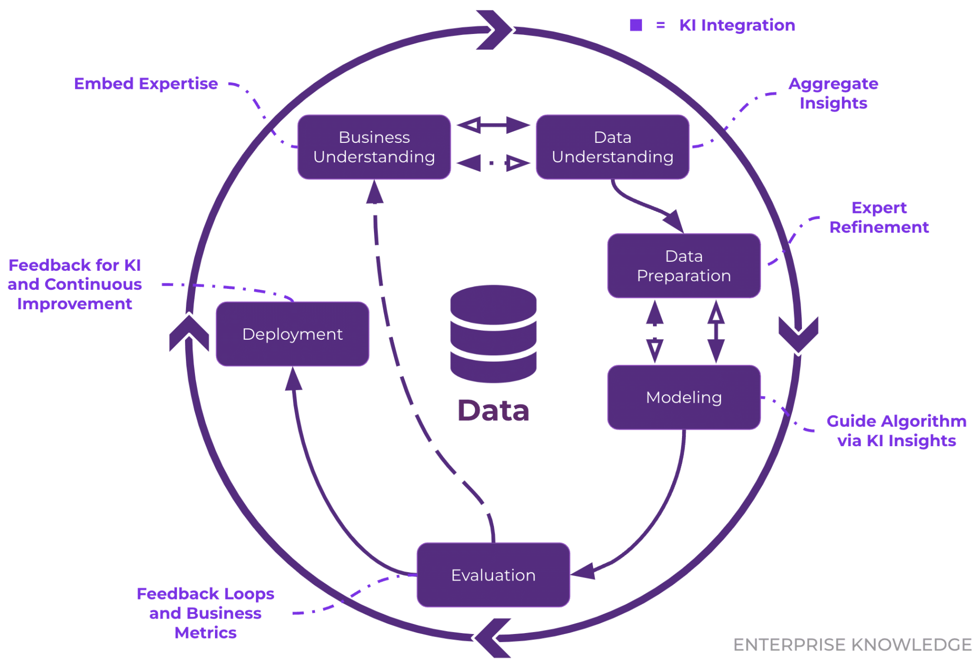

FIG. 1.2 : CRISP-DM Methodology Phases [3]

While CRISP-DM has been widely adopted in traditional data mining and analytics projects, its linear progression and narrow focus on pattern extraction from existing datasets make it ill-suited for projects that involve designing and deploying an intelligent system from the ground up. It provides no framework for artifact construction, formal evaluation of system utility, or academic communication of results [3].

4.3 Overview of TDSP

TDSP [4] is a more recent methodology developed specifically for AI and machine learning projects in enterprise environments. First released in 2016, it combines agile principles with a structured data science lifecycle to address the challenges of building intelligent applications intended for production deployment.

The methodology is built on four core components [4] :

- **Data Science Lifecycle :** An iterative process guiding projects from business understanding through to deployment and continuous improvement.
- **Standardized Project Structure :** Consistent templates and folder organization that promote team collaboration and reproducibility.

- **Infrastructure Recommendations :** Guidelines for cloud resources, development environments, and deployment platforms.
- **Tool and Utility Guidance :** A curated selection of frameworks and libraries supporting the entire AI development pipeline.

Figure 1.3 depicts the iterative lifecycle of the TDSP methodology.

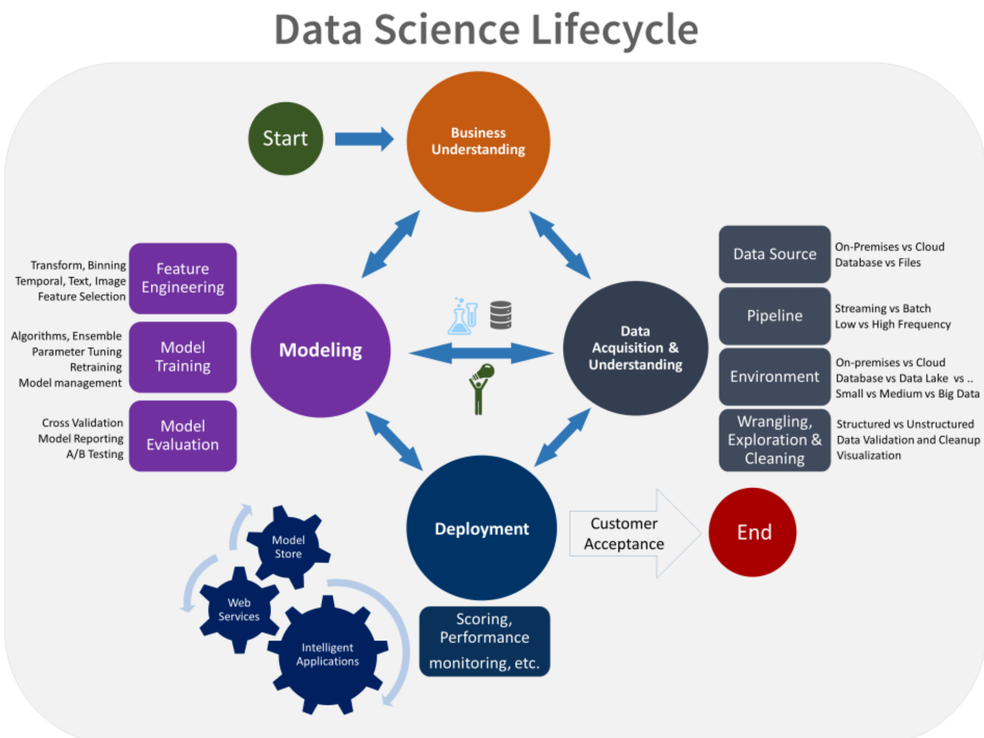

Fig. 1.3 : TDSP Iterative Lifecycle [4]

Although TDSP represents a significant improvement over CRISP-DM for modern AI projects, it remains fundamentally an industry-oriented engineering framework. It does not provide a formal mechanism for artifact-centered research, rigorous academic evaluation, or scientific communication of results, all of which are essential requirements for an academic graduation project [4].

4.4 Overview of DSR

DSR [5], [6] is a well-established research paradigm in information systems and computer science, specifically designed for projects whose primary output is a technological artifact, a system, model, method, or tool, built to solve a clearly identified real-world problem. Unlike traditional process methodologies, DSR is rooted in a scientific approach : it requires not only that an artifact be constructed, but that its utility and effectiveness be formally evaluated and communicated as a research contribution [5].

The DSR process model [6] consists of six iterative activities :

- **Problem Identification and Motivation :** Defining the research problem and justifying the value of developing a solution.
- **Definition of Objectives :** Specifying what a successful artifact must achieve, derived directly from the problem definition.
- **Design and Development :** Constructing the artifact, encompassing data collection, model training, system architecture, and integration.
- **Demonstration :** Deploying and using the artifact to solve the identified problem in a realistic setting.
- **Evaluation :** Assessing how well the artifact addresses the problem using appropriate metrics, comparisons, and validation techniques.
- **Communication :** Disseminating the problem, the artifact, and the findings to relevant audiences through academic reporting and presentation.

Figure 1.4 presents the complete DSR process model as defined by Peffers et al. [6].

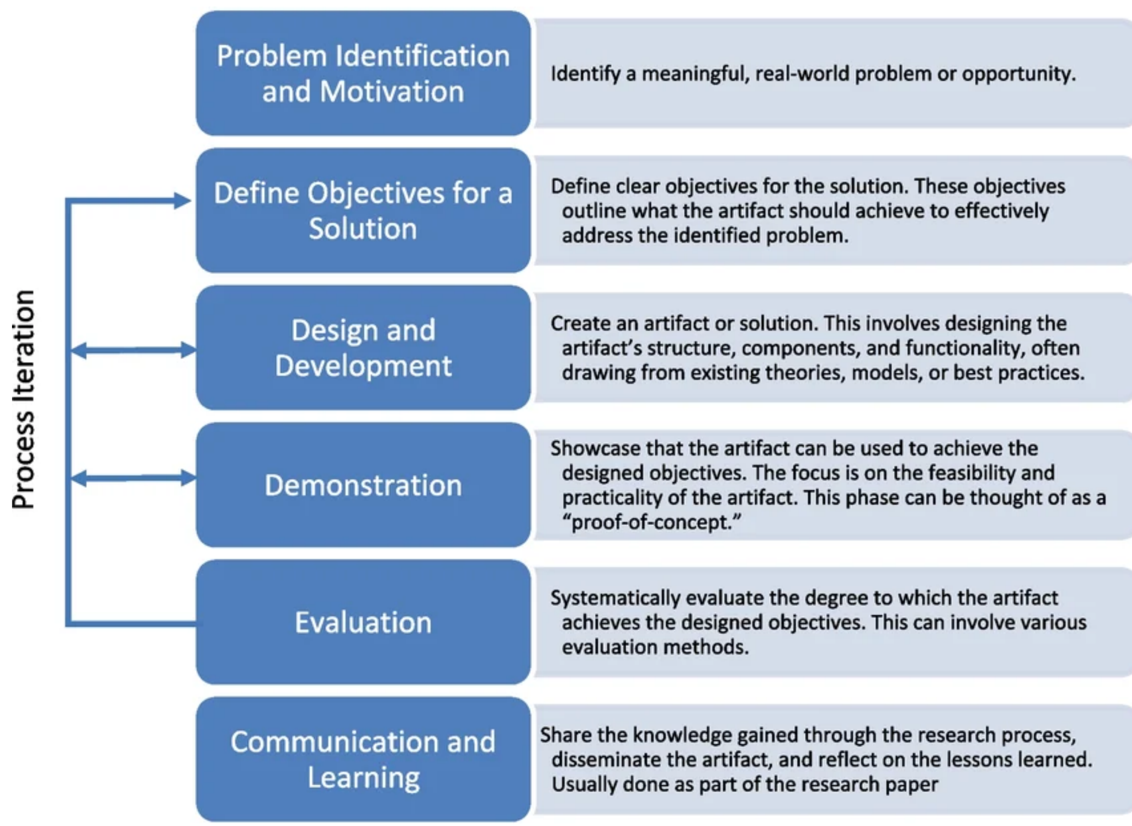

Fig. 1.4 : DSR Process Model [6]

4.5 Methodologies Comparison

Table 1.1 presents a structured comparison of the three methodologies across criteria relevant to the specific requirements of our project.

TAB. 1.1 : Comparative Analysis of CRISP-DM, TDSP, and DSR

Criteria	**CRISP-DM**	**TDSP**	**DSR**
Primary Focus	Data mining and pattern extraction	Production AI and ML systems	Design and evaluation of technological artifacts
Development Approach	Linear, sequential	Agile, iterative	Iterative, artifact-centered
Academic Rigor	Low	Low	High
Evaluation Framework	Basic model metrics	Deployment monitoring	Formal utility and efficacy assessment
Artifact Orientation	None	Partial	Central
Communication Phase	Not included	Not included	Explicitly included
Deployment Guidance	Basic system integration	Detailed production recommendations	Demonstration in realistic environment
Suitability for Small-Scale Projects	Weak	Moderate	Strong

4.6 Why We Chose DSR

The comparative analysis reveals that DSR [5], [6] is the methodology best aligned with the nature and objectives of our project, for the following reasons :

- **Artifact-Centered :** Our project's primary deliverable is a specialized legal chatbot, a concrete technological artifact. DSR is built precisely around the design, construction, and evaluation of such artifacts [5].
- **Academic Rigor :** Unlike CRISP-DM and TDSP, DSR provides a formal evaluation framework that allows the utility and performance of the system to be rigorously assessed, an essential requirement for an academic graduation project.
- **Structured Evaluation :** The dedicated evaluation phase enables a principled comparison between the fine-tuned model and a generic baseline, providing scientifically grounded evidence of improvement [6].

- **Communication Phase :** Unlike CRISP-DM and TDSP, DSR treats communication as a first-class activity rather than an afterthought, requiring researchers to document and disseminate the artifact, the design decisions, and the evaluation results to a relevant audience [6]. This maps naturally onto the written report and oral defense that conclude this graduation project.
- **Iterative by Design :** The build-and-evaluate cycle at the core of DSR [5] naturally accommodates the multiple iterations of fine-tuning, testing, and refinement that domain adaptation of a language model requires.

4.7 Our Adapted DSR Lifecycle

We tailored the DSR lifecycle [6] to fit the specific structure of our project, embedding the full AI technical pipeline within the Design and Development phase :

Phase 1: Problem Identification and Motivation. We identified the core limitations of E-Tafakna's existing AI agents, insufficient Tunisian legal knowledge and dependency on external cloud services, and established the need for a domain-specific, locally deployable solution.

Phase 2: Definition of Objectives. We defined three concrete objectives for the artifact : domain adaptation through fine-tuning on Tunisian legal corpora, dynamic knowledge retrieval through a RAG pipeline, and full on-premises deployment at ATI Tunisie.

Phase 3: Design and Development. This phase encompasses the complete AI technical pipeline :

- Collection and preprocessing of Tunisian legal documents.
- Fine-tuning of the base SLM or LLM on legal data.
- Design and implementation of the RAG pipeline and vector database.
- Integration of the complete system into E-Tafakna's platform architecture.

Phase 4: Demonstration. The artifact is deployed on ATI Tunisie's on-premises infrastructure and demonstrated through realistic legal consultation scenarios covering the use cases of both Elyssa and Kahina.

Phase 5: Evaluation. The system is evaluated using quantitative metrics (including response accuracy, legal precision, and retrieval quality) and compared against a generic baseline model to measure the concrete contribution of domain-specific fine-tuning.

Phase 6: Communication. The findings, artifact design, and evaluation results are communicated through this academic report and its oral defense before the graduation jury.

5 Choice of Services, Technologies, and Tools

Beyond selecting a methodological framework, the successful execution of our project required identifying the right set of services and technologies to support its different phases. This section presents the main platform we relied on during the design and development of our solution, and explains the rationale behind its selection.

5.1 Azure AI Foundry : Access to State-of-the-Art Language Models

Azure AI Foundry is Microsoft's unified platform for building, evaluating, and deploying generative AI applications. It provides centralized access to a catalog of foundation models from multiple providers (including OpenAI, Meta, and Mistral) through a single secure endpoint, along with tooling for prompt engineering, model evaluation, and enterprise-grade governance [7], as shown in Figure 1.5.

Fig. 1.5 : Azure AI Foundry Platform [7]

Within this ecosystem, we specifically relied on the **GPT-4.1** model made available through Azure OpenAI Service. GPT-4.1 is one of the most advanced large language models offered on the platform, featuring an expanded context window, strong multimodal capabilities, and improved instruction following compared to previous generations, characteristics that are particularly valuable when processing dense and heterogeneous legal documents [8].

Role in Our Project

GPT-4.1 played a central role in the data preparation phase of our project. The Tunisian legal corpus required for fine-tuning was originally available as a large collection of PDF documents (including legislation, official gazettes, and regulatory texts) whose content was often embedded in complex layouts, multi-column formats, and scanned pages. Traditional text extraction tools proved insufficient to reliably capture the structure and semantics of these documents.

To overcome this limitation, we used GPT-4.1 as an intelligent extraction engine capable of :

- **Accurate Text Extraction :** Converting raw PDF content into clean, structured text while preserving the hierarchical organization of legal documents (articles, sections, and clauses).
- **Semantic Structuring :** Identifying and labeling key components of legal texts, such as article numbers, titles, and references to other legal provisions.

- **Noise Removal :** Filtering out irrelevant artifacts introduced by the PDF format, including headers, footers, page numbers, and watermarks.

The output of this extraction process served as the foundation of our curated Tunisian legal dataset, which was subsequently used for fine-tuning the target language model and populating the vector database of the RAG pipeline.

Rationale for Selection

Azure AI Foundry was chosen for its enterprise-grade security guarantees, its native integration with other Azure services, and its compliance with data protection standards, all of which align with the confidentiality requirements of legal data handling. Although the final production system is deployed entirely on-premises at ATI Tunisie, leveraging GPT-4.1 during the data preparation phase allowed us to build a high-quality legal corpus efficiently, without compromising the on-premises nature of the final deliverable.

5.2 Google Vertex AI : OCR Extraction from Image-Based PDFs

Google Vertex AI is Google Cloud's unified machine learning platform, bringing together a broad suite of AI services (including foundation models, AutoML, custom training, and managed inference) under a single API and console [9]. Among its services, **Document AI** provides enterprise-grade document processing capabilities, including optical character recognition (Optical Character Recognition (OCR)), form parsing, and intelligent document splitting [10], as shown in Figure 1.6.

Vertex AI

FIG. 1.6 : Google Vertex AI [9]

Role in Our Project

A significant portion of the Tunisian legal corpus was available only as scanned image-based Portable Document Format (PDF) files, meaning the content was stored as rasterized images rather than selectable text, making standard text extraction tools entirely ineffective. To handle this category of documents, we relied on Google Vertex AI's Document AI OCR capabilities, which can process image-based PDF pages and return high-accuracy text output by performing character recognition directly on the visual content.

Specifically, Vertex AI Document AI was used to :

- **Optical Character Recognition (OCR) Processing :** Extract machine-readable text from scanned legal documents where the content existed solely as images, covering legislation texts, official gazettes, and regulatory decrees unavailable in digital form.

- **Multi-page Document Handling :** Process lengthy PDF files spanning hundreds of pages in batch, returning structured text output that could be integrated into the same data pipeline as digitally-born documents.

- **Arabic and French Script Recognition :** Accurately recognize both Arabic and French characters present in Tunisian legal texts, given that many official documents are bilingual.

The extracted text was then passed through the same cleaning and structuring pipeline used for the digitally-born documents, ensuring a uniform representation across the entire legal corpus.

Rationale for Selection

Google Vertex AI Document AI was selected specifically for its strong multilingual OCR performance on Arabic-script and Latin-script documents, a critical requirement given the bilingual nature of the Tunisian legal corpus. Its cloud-based batch processing model also allowed us to handle large volumes of scanned documents without requiring local infrastructure during the data preparation phase, while the final production system remained fully on-premises at ATI Tunisie.

5.3 OVH AI Notebooks : GPU Resources for Fine-Tuning

OVH AI Notebooks is a managed cloud service provided by OVHcloud that delivers on-demand Jupyter notebook environments backed by dedicated Graphics Processing Unit (GPU) instances. It is designed to make computationally intensive AI workloads accessible without the overhead of provisioning and maintaining dedicated hardware, giving practitioners direct access to GPU resources through a familiar notebook interface [11], as shown in Figure 1.7.

Fig. 1.7 : OVHcloud AI Notebooks [11]

Role in Our Project

Fine-tuning a large language model on a domain-specific corpus is among the most computationally demanding tasks in the AI pipeline. Running such a workload on a standard development machine would require days or even weeks of processing time, making rapid experimentation practically impossible. To address this, we used OVH AI Notebooks to run the fine-tuning experiments on GPU-accelerated instances, which dramatically reduced training time and allowed us to iterate efficiently over different configurations :

- **Fine-Tuning Execution :** Running the supervised fine-tuning pipeline on the curated Tunisian legal corpus, leveraging GPU acceleration to complete training jobs in a fraction of the time that would be required on standard hardware.
- **Experiment Tracking :** Using the notebook environment to monitor training metrics, compare runs, and adjust hyperparameters interactively between iterations.
- **Model Evaluation :** Running inference on the fine-tuned checkpoints directly within the notebook environment to perform initial quality assessments before moving to formal evaluation.

Rationale for Selection

OVH AI Notebooks was chosen for the combination of accessible GPU capacity, competitive pricing, and European data residency, a relevant consideration given the legal sensitivity of the training data. Its notebook-native interface integrated smoothly with our existing Python and Hugging Face Transformers workflow, avoiding the need to learn a new toolchain. As with the other cloud services used during data preparation, the reliance on OVH was limited strictly to the training phase. The final production system remains entirely self-contained and deployed on-premises at ATI Tunisie.

5.4 Visual Studio Code

Visual Studio Code is a free, lightweight code editor maintained by Microsoft. While it remains light on system resources, it can be extended to support virtually any language or workflow through its extension marketplace, and brings together editing, debugging, version control, and a built-in terminal within a single interface [12].

Fig. 1.8 : Visual Studio Code [12]

Role in Our Project

VS Code acted as the single workspace where most of the technical work took place. Its versatility allowed us to handle several aspects of development without leaving the editor :

- **Python Development :** Writing and debugging the data preparation routines, the fine-tuning scripts, and the RAG integration code, with the help of the official Python extension and interactive Jupyter notebooks.

- **Version Control :** Managing the project's source code directly through the built-in Git interface, which made tracking changes and navigating the project history straightforward.
- **Integrated Terminal :** Launching training jobs, installing dependencies, and running command-line utilities from within the editor itself, avoiding the need to jump between different windows.

Rationale for Selection

We chose VS Code mainly for practical reasons : it runs smoothly on modest hardware, supports the full Python ecosystem we needed, and brings editing, debugging, and version control together in one interface. Keeping everything in a single workspace noticeably reduced the friction of switching between tasks, which matters in a project that combines data handling, model training, and integration work.

5.5 Python

Python is a high-level, interpreted programming language known for its clear syntax and the maturity of its scientific and AI libraries. It is today the most widely adopted language in the AI and machine learning community, largely because of the richness of its ecosystem around data processing, model training, and evaluation [13].

Fig. 1.9 : Python Programming Language [13]

Role in Our Project

Python served as the main language for every technical component of our solution. Its rich ecosystem allowed us to cover the entire AI pipeline without having to mix multiple languages :

- **Data Processing :** Cleaning and restructuring the Tunisian legal corpus extracted from PDF documents, using libraries such as Pandas and NumPy.
- **Model Fine-Tuning :** Building the fine-tuning pipeline around widely used AI frameworks, including Hugging Face Transformers and PyTorch.
- **RAG Pipeline Development :** Implementing the different stages of the retrieval-augmented generation workflow, from embedding generation to vector store queries and orchestration of the overall flow.
- **Scripting and Automation :** Writing smaller utility scripts to automate recurring tasks such as dataset preparation, evaluation runs, and aggregation of results.

Rationale for Selection

Python was chosen for its central role in the AI ecosystem and the availability of mature libraries covering every stage of our pipeline, from data extraction to model training, evaluation, and deployment. Its readability also made the codebase easier to document and maintain, which was particularly valuable in the academic context of this project.

5.6 n8n

n8n is an open-source workflow automation platform that allows users to design processing pipelines visually, by connecting nodes that each perform a specific action, from calling an API to reading a file or running a script. It can be fully self-hosted, meaning workflow data never has to leave the local environment [14].

Fig. 1.10 : n8n Workflow Automation Platform [14]

Role in Our Project

n8n was used to orchestrate the different stages of our processing pipeline. Rather than chaining scripts manually or writing a dedicated runner, we built workflows that trigger each step in the right order and give us a clear visual overview of the whole process :

- **Data Pipeline Orchestration :** Coordinating the sequence of operations involved in preparing the legal corpus, from raw document ingestion to structured output generation.
- **API Integration :** Connecting the different components of the system through Hypertext Transfer Protocol (HTTP) calls, which simplified communication between services without adding custom glue code.
- **Process Automation :** Scheduling and triggering recurring tasks automatically, reducing manual work and keeping pipeline executions consistent.

Rationale for Selection

n8n was selected mainly because it is open-source and self-hostable, which aligns with the on-premises deployment constraints imposed by ATI Tunisie. Beyond that, its visual approach lowered the overhead of managing a growing number of processing steps, and the ability to keep everything on local infrastructure ensured that no data ever left the controlled environment during automated processing.

5.7 FastAPI

FastAPI is a modern Python web framework used to expose code as HTTP services. It is designed around Python type hints, which it leverages to validate requests automatically and generate interactive documentation, while offering strong performance thanks to its native support for asynchronous operations [15].

FastAPI

Fig. 1.11 : FastAPI Framework [15]

Role in Our Project

FastAPI served as the bridge between our Python scripts and the n8n automation platform. Since n8n interacts with external services through HTTP calls, we wrapped our extraction and RAG-related scripts behind small FastAPI endpoints so that n8n workflows could invoke them like any other integration :

- **Script Exposition :** Turning standalone Python scripts, including the legal data extraction routines and the RAG pipeline components, into HTTP endpoints that could be consumed directly from n8n.
- **Seamless Orchestration :** Allowing n8n to trigger Python operations as part of larger automated workflows, keeping a clean separation between orchestration on one side and execution on the other.
- **Automatic Validation and Documentation :** Relying on FastAPI's type-based validation and auto-generated interactive interface to speed up the development and testing of each endpoint.

Rationale for Selection

FastAPI was chosen because it makes exposing Python functions as web endpoints almost trivial, while still offering solid performance under the kind of long-running inference calls our pipeline involves. Its native support for asynchronous operations fit naturally with our AI workloads, and its tight integration with the Python ecosystem made it the most practical choice to connect our processing scripts to the n8n automation layer.

6 Conclusion

This chapter introduced E-Tafakna and its existing AI agents, identified the two core limitations of the current system, namely the lack of Tunisian legal grounding and the dependency on external cloud services, and proposed a solution combining a fine-tuned language model, a RAG pipeline, and fully on-premises deployment at ATI Tunisie. We also selected

DSR as our guiding methodology, given its artifact-centered structure and formal evaluation framework.

The following chapter grounds this project in the existing literature, examining the key concepts and technologies that underpin our proposed solution.

Chapitre II

PROBLEM IDENTIFICATION AND MOTIVATION

1 Introduction

This chapter grounds the project in the existing literature and formalizes its requirements. We cover the core concepts behind LLMs, fine-tuning for domain adaptation, the RAG paradigm, and the challenges of on-premises deployment. A comparative study of existing legal AI solutions confirms the gap our work addresses, and we close by defining the functional and non-functional requirements that will guide the system design.

2 Large Language Models

2.1 Defining Large Language Models

A LLM can be broadly understood as a deep neural network trained on massive volumes of textual data with the objective of modeling the statistical structure of natural language. Unlike earlier Natural Language Processing (NLP) systems that relied on hand-crafted rules or shallow pattern matching, LLMs learn rich internal representations of language through exposure to billions of tokens drawn from diverse sources such as books, web pages, and scientific articles [16]. The resulting models are capable of generating coherent text, answering questions, summarizing documents, and performing a wide range of language tasks without being explicitly programmed for any of them.

What distinguishes a LLM from earlier language models is primarily its scale, both in terms of parameter count (often ranging from several billion to hundreds of billions) and the volume of training data consumed. This scale enables emergent capabilities : behaviors that do not appear in smaller models but arise once the model reaches a sufficient size, such as in-context learning and multi-step reasoning [16].

2.2 The Transformer Architecture

The architecture underpinning virtually all modern LLMs is the Transformer, introduced by Vaswani et al. in 2017 [17]. Before its arrival, sequence modeling tasks in NLP were dominated by recurrent neural networks, which processed tokens one at a time in sequential order. This sequential nature created a bottleneck : long-range dependencies between distant words were difficult to capture, and training could not be effectively parallelized across hardware.

The Transformer resolved both limitations through a mechanism called **self-attention**. Rather than reading a sequence token by token, the self-attention mechanism allows every token in a sequence to directly attend to every other token, computing a weighted representation that captures contextual relationships regardless of distance. This parallel computation of attention across all positions makes the Transformer significantly faster to train on modern GPU hardware and far more effective at modeling long-range dependencies [17].

A standard Transformer consists of two main components : an **encoder**, which reads and encodes the input sequence into a set of continuous representations, and a **decoder**, which generates an output sequence one token at a time by attending to both the encoded input and its own previously generated tokens. Different model families use different subsets of this architecture :

- **Encoder-only models** (e.g., Bidirectional Encoder Representations from Transformers (BERT)) focus on understanding and classifying input text by building bidirectional contextual representations [18].
- **Decoder-only models** (e.g., Generative Pre-trained Transformer (GPT)) generate text autoregressively, predicting the next token based on all preceding tokens, and form the basis of most modern conversational LLMs [16].
- **Encoder-decoder models** (e.g., T5) combine both components and are commonly used for tasks that involve transforming one sequence into another, such as translation or summarization.

Figure 2.12 illustrates the overall architecture of the Transformer model, including its encoder and decoder stacks.

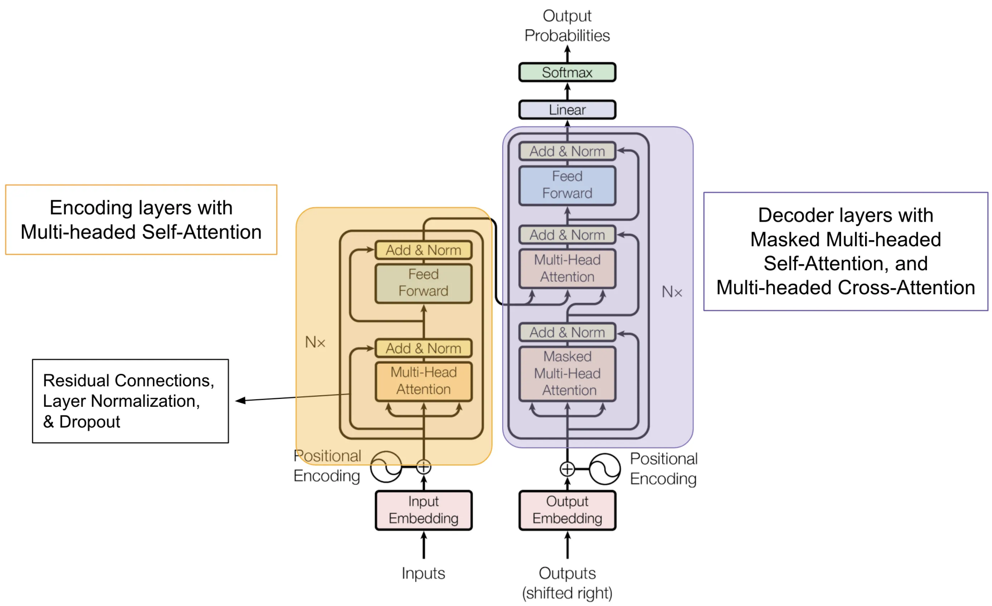

Fig. 2.12 : The Transformer Architecture [19]

2.3 Pre-Training and Transfer Learning

The power of LLMs stems largely from a two-stage training paradigm. In the first stage, called **pre-training**, the model is exposed to an enormous corpus of unlabeled text and learns to predict the next word in a sequence (for decoder-only models) or to reconstruct masked words (for encoder-only models). This unsupervised process forces the model to internalize grammar, factual knowledge, reasoning patterns, and even stylistic conventions present in the training data [18].

The second stage, called **fine-tuning** or **transfer learning**, adapts the pre-trained model to a specific downstream task or domain by continuing training on a smaller, task-specific dataset. Because the model has already acquired broad linguistic knowledge during pre-training, fine-tuning requires comparatively little data and computation to achieve strong performance on the target task. This paradigm has become the standard approach in modern NLP, enabling practitioners to leverage the general capabilities of large pre-trained models while specializing them for particular applications [16].

2.4 Key Model Families

The landscape of open-weight and proprietary LLMs has expanded rapidly in recent years. Several model families are particularly relevant to our project.

GPT (OpenAI). The GPT series pioneered the decoder-only autoregressive approach to language modeling. GPT-3, with 175 billion parameters, demonstrated that scaling model size and training data could unlock powerful few-shot learning capabilities. GPT-4 and GPT-4.1 extended these capabilities further with multimodal understanding and improved instruction following [16], as shown in Figure 2.13.

Fig. 2.13 : GPT - OpenAI [16]

LLaMA (Meta). The LLaMA family, introduced by Meta, was designed to achieve strong performance at smaller scales by training on more data per parameter than previous approaches. LLaMA models range from 7 billion to 70 billion parameters and have become a popular foundation for community-driven fine-tuning and adaptation [20], as shown in Figure 2.14.

Fig. 2.14 : LLaMA - Meta [20]

Qwen (Alibaba Cloud). The Qwen family, developed by Alibaba Cloud, covers a wide range of model sizes from 0.5 billion to over 70 billion parameters. Qwen models stand out for their strong multilingual capabilities, with particular emphasis on Arabic and other non-Latin scripts, making them well suited for applications that span multiple languages and writing systems [21], as shown in Figure 2.15.

Fig. 2.15 : Qwen - Alibaba Cloud [21]

Gemma (Google). The Gemma family represents Google's contribution to the open-weight model ecosystem. Built on the same research foundations as the Gemini series, Gemma models are available in compact sizes (2B and 7B parameters) designed for efficient deployment, making them particularly attractive for on-premises scenarios with limited hardware resources [22], as shown in Figure 2.16.

Fig. 2.16 : Gemma - Google [22]

3 Fine-Tuning Techniques for Domain Adaptation

While pre-trained LLMs possess broad linguistic competence, they often lack the specialized vocabulary, reasoning patterns, and domain-specific knowledge required for high-stakes applications such as legal analysis. Domain adaptation through fine-tuning addresses this gap by continuing the model's training on a curated corpus of domain-specific data, allowing it to internalize the particular conventions, terminology, and logic of the target field.

3.1 Full Fine-Tuning

The most straightforward approach to domain adaptation is **full fine-tuning**, in which all parameters of the pre-trained model are updated using the domain-specific dataset. During this process, the model's weights are adjusted through standard backpropagation and gradient descent, allowing every layer of the network to adapt to the new data distribution.

Full fine-tuning typically produces the highest quality adaptation, as the entire model capacity is available to learn domain-specific patterns. However, it comes with significant practical drawbacks : it requires storing and updating a complete copy of all model parameters, which translates to very high GPU memory consumption and substantial computational cost. For a model with billions of parameters, full fine-tuning may require multiple high-end GPUs and several days of training, making it impractical for many organizations and research groups [23].

3.2 Parameter-Efficient Fine-Tuning

To overcome the resource limitations of full fine-tuning, the research community has developed a family of methods collectively known as Parameter-Efficient Fine-Tuning (PEFT). These techniques aim to achieve comparable adaptation quality while modifying only a small fraction of the model's parameters, dramatically reducing both memory requirements and training time.

3.2.1 LoRA : Low-Rank Adaptation

Low-Rank Adaptation (LoRA), proposed by Hu et al. [23], is one of the most widely adopted PEFT methods. The core insight behind LoRA is that the weight updates required for domain adaptation tend to lie in a low-dimensional subspace. Instead of modifying the full weight matrices of the model, LoRA freezes the original pre-trained weights and injects small trainable low-rank matrices alongside them. During fine-tuning, only these low-rank matrices are updated, while the vast majority of the model's parameters remain unchanged.

In practice, this means that a LoRA adapter might add only a few million trainable parameters to a model that contains billions, reducing memory consumption by an order of magnitude while achieving adaptation quality that is often indistinguishable from full fine-tuning [23]. Figure 2.17 illustrates how LoRA injects trainable low-rank matrices alongside the frozen pre-trained weights.

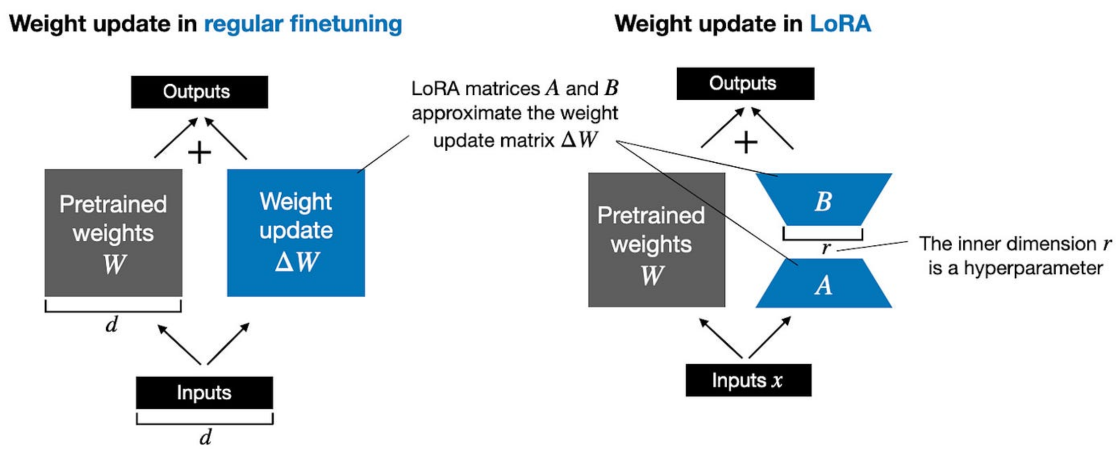

FIG. 2.17 : LoRA : Low-Rank Adaptation of frozen weight matrices [24]

3.2.2 QLoRA : Quantized Low-Rank Adaptation

Quantized Low-Rank Adaptation (QLoRA), introduced by Dettmers et al. [25], extends LoRA by combining it with aggressive model quantization. The pre-trained model is first quantized to 4-bit precision using a novel data type called NormalFloat4, which reduces its memory footprint by approximately four times compared to standard 16-bit representations. The LoRA adapters are then trained in 16-bit precision on top of this quantized base.

This combination makes it possible to fine-tune models with tens of billions of parameters on a single consumer-grade GPU, a capability that was previously restricted to organizations with access to expensive multi-GPU clusters. QLoRA has been shown to match the performance of full 16-bit fine-tuning on a variety of benchmarks while using a fraction of the hardware resources [25]. Figure 2.18 compares the parameter update strategies of LoRA and full fine-tuning.

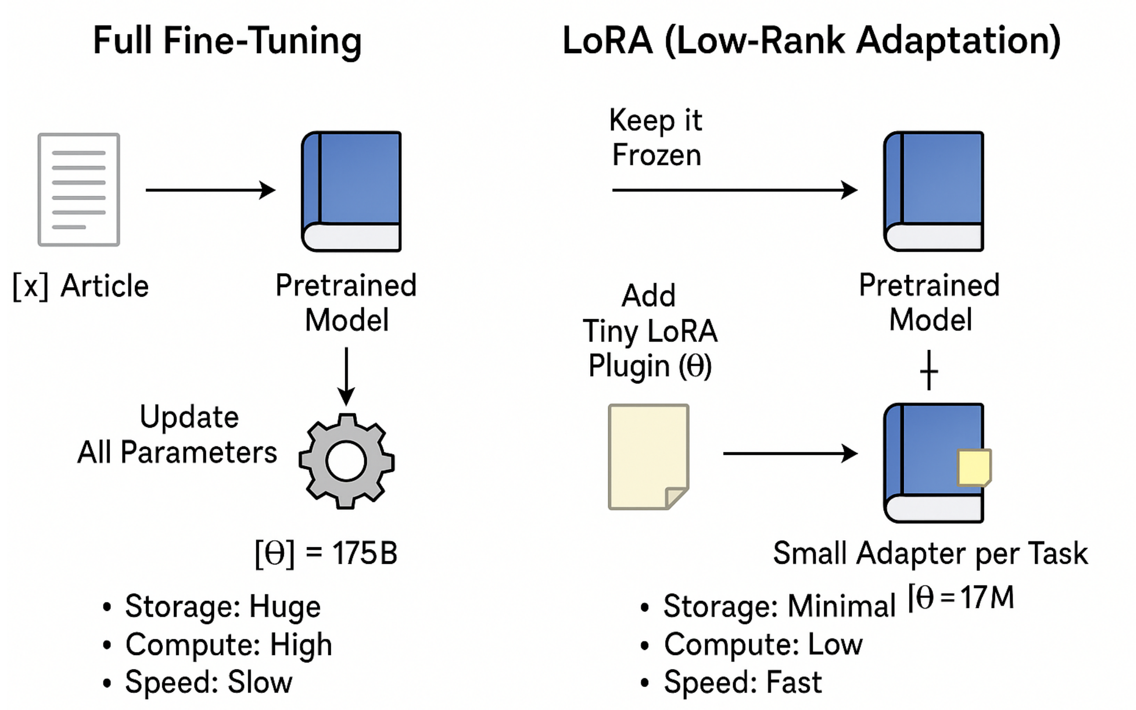

Fig. 2.18 : LoRA vs Full Fine-Tuning : comparison of parameter update strategies [26]

3.3 Domain Adaptation in Specialized Fields

Fine-tuning for domain adaptation has been successfully applied across a range of specialized fields, demonstrating that even moderately sized models can achieve expert-level performance when trained on high-quality domain data :

- **Medical Domain :** Models fine-tuned on clinical notes, medical literature, and diagnostic guidelines have shown significant improvements in medical question answering and clinical reasoning compared to their general-purpose counterparts.
- **Financial Domain :** Language models adapted to financial texts (including earnings reports, regulatory filings, and market analyses) have demonstrated improved accuracy in sentiment analysis, risk assessment, and financial question answering.
- **Legal Domain :** Fine-tuning on legal corpora enables models to better understand the hierarchical structure of legislation, the precise meaning of legal terminology, and the citation conventions that pervade legal reasoning, capabilities that general-purpose models consistently lack.

These examples validate the approach at the core of our project : fine-tuning a base model on a curated Tunisian legal corpus to produce a specialized assistant grounded in the specific vocabulary, structure, and logic of Tunisian law.

4 Retrieval-Augmented Generation

4.1 The Hallucination Problem

Despite their remarkable fluency, LLMs are prone to a well-documented failure mode known as **hallucination** : the generation of text that sounds plausible but is factually incorrect, unsupported, or entirely fabricated. This occurs because language models are fundamentally trained to produce statistically likely continuations of text, not to verify the factual accuracy of their outputs [27].

In high-stakes domains such as law, hallucination is particularly dangerous. A legal assistant that confidently cites a non-existent article of law or misrepresents the content of a regulation could mislead users into making consequential decisions based on false information. Fine-tuning alone cannot fully eliminate hallucination, because the model's knowledge remains bounded by its training data and cannot account for legislative updates, newly enacted regulations, or documents not included in the training corpus.

4.2 The RAG Paradigm

RAG, first formalized by Lewis et al. [27], addresses the hallucination problem by augmenting the language model with an external knowledge retrieval mechanism. Rather than relying solely on the knowledge encoded in its parameters, a RAG-enabled system retrieves relevant documents from an external knowledge base at inference time and injects them into the model's input context before generating a response.

The RAG pipeline typically operates in three stages :

1. **Indexing :** Documents from the knowledge base are split into manageable chunks, each of which is converted into a dense vector representation (embedding) and stored in a vector database.
2. **Retrieval :** When a user submits a query, the query is similarly converted into an embedding, and the vector database is searched for the most semantically similar document chunks using approximate nearest neighbor algorithms.
3. **Generation :** The retrieved chunks are concatenated with the original query and passed to the language model, which generates a response grounded in the retrieved evidence.

This architecture enables the system to provide responses that are anchored in actual documents, significantly reducing hallucination and allowing the knowledge base to be updated independently of the model itself [27]. Figure 2.19 illustrates the three stages of the RAG pipeline.

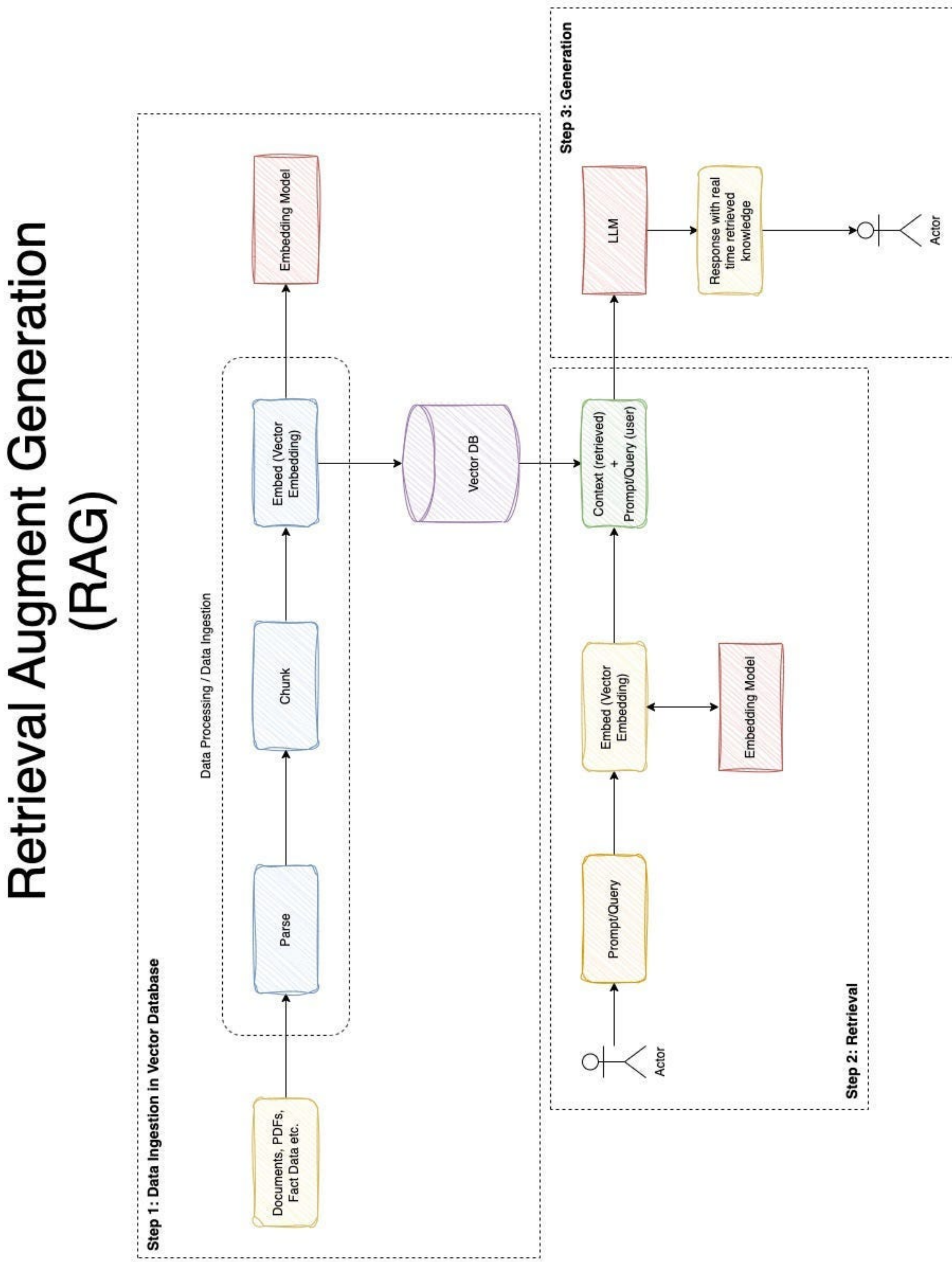

Fig. 2.19 : The RAG Pipeline : indexing, retrieval, and generation stages [28]

4.3 Vector Databases and Embedding Models

A critical component of any RAG pipeline is the **vector database**, a specialized storage system built to index and search high-dimensional vector representations at scale. Rather than matching records on exact field values as relational databases do, vector databases rank results by geometric proximity, using metrics such as cosine similarity or dot product to surface the vectors closest to a given query.

Facebook AI Similarity Search (FAISS) (Meta). Developed by Meta Research, FAISS is an open-source library purpose-built for dense vector search over very large collections. It supports both exact and approximate nearest-neighbor algorithms, giving practitioners a trade-off between search precision and speed depending on the scale of their dataset [29].

Chroma. Chroma is a lightweight vector store aimed at developers building AI applications. Its straightforward API makes it easy to store, query, and manage embeddings, and it can operate either as an embedded in-process database or as a standalone server, keeping the integration overhead low [30], as shown in Figure 2.20.

Fig. 2.20 : Chroma Vector Database [30]

Qdrant. Qdrant is a production-grade vector database that goes beyond basic similarity search by supporting rich payload filtering alongside vector queries. Its distributed architecture allows it to scale horizontally, making it suitable for workloads that combine dense retrieval with structured metadata filtering [31], as shown in Figure 2.21.

Fig. 2.21 : Qdrant Vector Database [31]

Retrieval quality depends not only on the vector database but equally on the **embedding model** responsible for converting text into vector representations. The embedding model must map semantically related passages to nearby points in vector space so that a user query retrieves the most relevant content even when the phrasing differs from the source document. Several embedding models are widely used in RAG pipelines :

- **Sentence Transformers (SBERT) :** A family of models built on top of BERT and trained specifically to produce sentence-level embeddings. They are fast, lightweight, and well suited for monolingual or bilingual applications, but their multilingual coverage can be limited depending on the variant used [32].

- **OpenAI text-embedding models :** Proprietary embedding models accessible via API that deliver strong performance across a wide range of languages and tasks. Their main drawback is that they require sending data to an external cloud service, which makes them incompatible with on-premises deployment requirements.
- **BGE-M3 (BAAI) :** Developed by the Beijing Academy of Artificial Intelligence, BGE-M3 handles over 100 languages within a single model and supports dense, sparse, and multi-vector retrieval simultaneously [33]. This combination is particularly valuable for legal corpora that mix Arabic and French, and where both semantic similarity and precise keyword matching are needed.

For our project, we selected **BGE-M3** as the embedding model. Its native support for Arabic and French within a single unified model, combined with its ability to run fully on-premises without any external dependency, makes it the most suitable choice for our Tunisian legal corpus [33].

4.4 Chunking Strategies

Before documents can be embedded and indexed, they must be divided into smaller units called **chunks**. The choice of chunking strategy directly impacts retrieval quality :

- **Fixed-size chunking** divides documents into segments of a predetermined token or character count. It is simple to implement but may split semantically coherent passages across chunk boundaries.
- **Semantic chunking** uses structural cues, such as paragraph boundaries, section headings, or sentence boundaries, to create chunks that preserve the logical organization of the text.
- **Recursive chunking** attempts to split text at natural boundaries (paragraphs first, then sentences, then words) while respecting a maximum chunk size, striking a balance between semantic coherence and size constraints.

For legal documents, where the meaning of a clause or article depends on its complete text and its position within a hierarchical structure, semantic or recursive chunking strategies are generally preferred over fixed-size approaches. Figure 2.22 summarizes the main chunking strategies applicable to RAG pipelines.

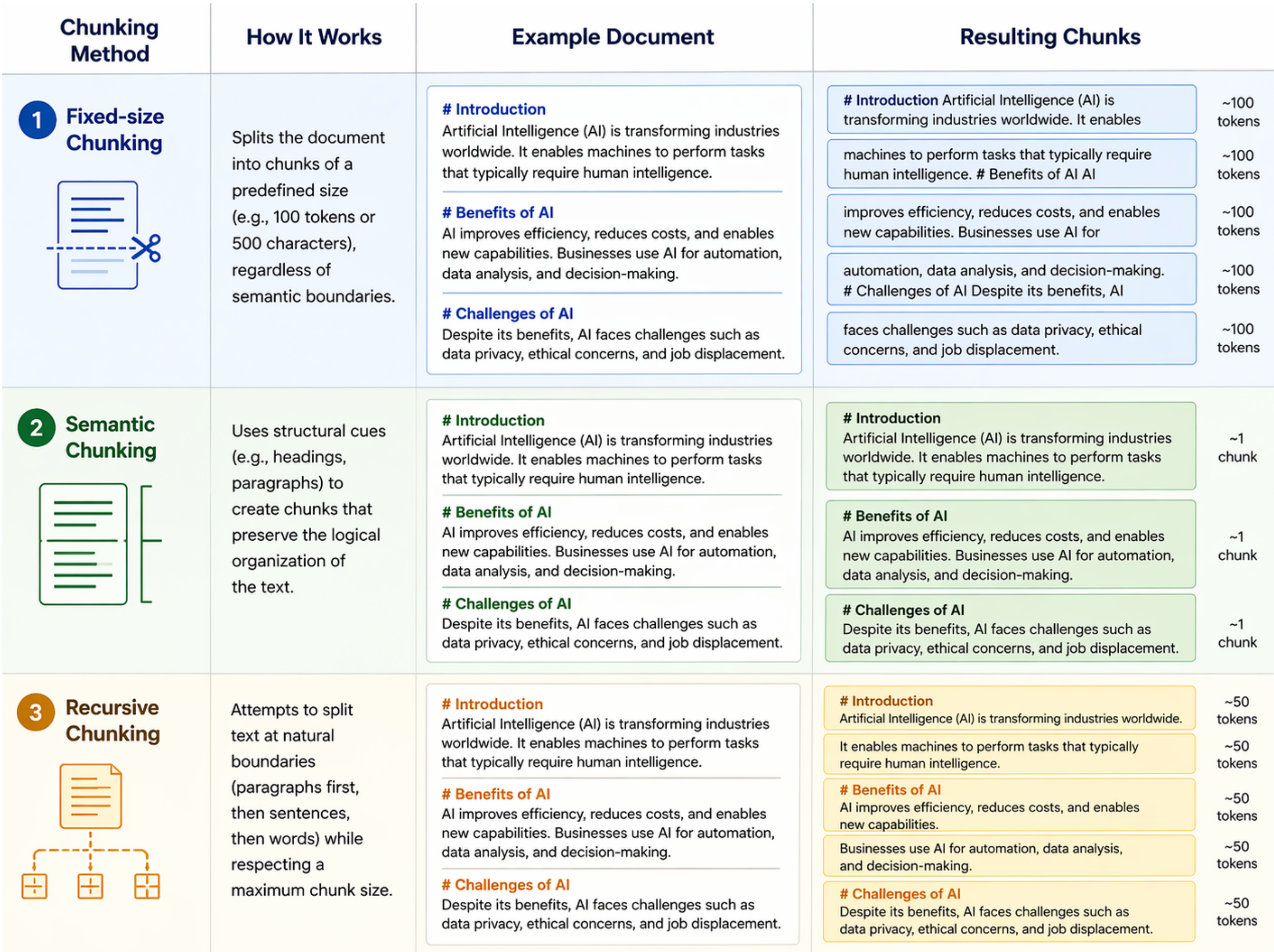

FIG. 2.22 : Overview of Common Chunking Strategies for RAG Pipelines

5 AI for Legal Applications

5.1 Challenges Specific to Legal NLP

Applying NLP techniques to legal texts presents a distinct set of challenges that differ significantly from general-purpose language tasks :

- **Terminological Precision :** Legal language relies on a specialized vocabulary where terms carry precise definitions that differ from their everyday usage. A model that conflates the legal and colloquial meanings of a term may produce misleading outputs.
- **Hierarchical Document Structure :** Legal texts are organized in rigid hierarchical structures (codes, titles, chapters, sections, articles, and clauses) that carry semantic significance. Understanding a legal provision often requires awareness of its position within this hierarchy.
- **Cross-Referencing and Citation :** Legal reasoning is deeply referential : articles frequently cite other articles, regulations reference enabling legislation, and judicial decisions build upon prior rulings. A model must be capable of following these reference chains to provide accurate answers.
- **Jurisdictional Specificity :** Legal systems vary significantly across jurisdictions. A model trained primarily on common law texts will perform poorly on questions about Tunisian civil law, as the legal principles, procedural rules, and legislative structures are fundamentally different.
- **Multilingual Requirements :** In countries like Tunisia, legal documents may exist in multiple languages, primarily Arabic and French, and a model must handle both languages to cover the full scope of the legal corpus.

5.2 Existing Legal AI Systems Worldwide

The application of AI to legal services has grown rapidly in recent years, with several notable systems emerging across different jurisdictions :

- **Harvey AI :** A legal AI platform built on top of OpenAI's models, designed for large law firms. Harvey provides contract analysis, legal research assistance, and document drafting capabilities, primarily targeting English-speaking common law jurisdictions [34].
- **Doctrine.fr :** A French legal intelligence platform that uses AI to search and analyze French case law, legislation, and legal commentary. It provides contextualized search results and cross-references across French legal sources [35].
- **CaseText (now part of Thomson Reuters) :** An AI-powered legal research platform that uses natural language search to help lawyers find relevant case law and statutory provisions, primarily within the United States legal system [36].

- **Luminance :** A legal AI tool focused on contract review and due diligence, using machine learning to identify anomalies, risks, and key provisions across large volumes of documents [37].

5.3 Arabic and French Legal NLP

Research on legal NLP in Arabic and French remains significantly less mature than its English counterpart. While a growing body of work has explored Arabic NLP in general domains, including sentiment analysis, named entity recognition, and machine translation, legal-specific Arabic NLP remains an underexplored area. The challenges are compounded by :

- The morphological complexity of Arabic, which makes tokenization and entity extraction more difficult than in European languages.
- The scarcity of publicly available Arabic legal corpora suitable for training or evaluating language models.
- The diversity of Arabic dialects and legal terminology across different Arab countries, which limits the transferability of models trained on one country's legal texts to another.

French legal NLP benefits from a larger body of digitized legal texts and more active research communities, but specialized models for Tunisian law, which operates under a distinct civil code and regulatory framework, remain virtually nonexistent. This gap is precisely what our project seeks to address.

6 On-Premises LLM Deployment

Deploying LLMs on local infrastructure, rather than relying on cloud-hosted APIs, introduces a distinct set of technical challenges that must be addressed to make the system viable within the resource constraints of an on-premises environment.

6.1 Model Quantization Techniques

Running a language model with billions of parameters in its original precision (typically 16-bit or 32-bit floating point) requires enormous amounts of Video Random Access Memory (VRAM). **Quantization** is the process of reducing the numerical precision of model weightsm for instance, from 16-bit to 8-bit or 4-bit, to decrease memory consumption and accelerate inference, often with minimal impact on output quality.

Two widely adopted quantization formats are :

- **GPT-Generated Unified Format (GGUF) :** A file format designed for efficient local inference, particularly on Central Processing Unit (CPU)-based systems. GGUF supports a range of quantization levels (from 2-bit to 8-bit) and is the native format used by llama.cpp and Ollama for loading and running models locally [38].

- **GPT Post-Training Quantization (GPTQ) :** A post-training quantization method that compresses model weights to 4-bit or 3-bit precision while minimizing the loss in output quality. GPTQ is optimized for GPU-based inference and is widely supported by inference frameworks such as vLLM and text-generation-inference [39].

6.2 Local Inference Frameworks

Several frameworks have been developed to facilitate the deployment and execution of LLMs on local hardware.

Ollama. An open-source tool that simplifies the process of downloading, running, and managing LLMs on local machines. Ollama provides a command-line interface and a local API server, supports GGUF-quantized models, and handles model lifecycle management (downloading, loading, and unloading) automatically. It is the inference framework selected for our project's on-premises deployment at ATI Tunisie [38], as shown in Figure 2.23.

Fig. 2.23 : Ollama Local Inference Framework [38]

vLLM. A high-throughput inference engine designed for production-grade LLM serving. vLLM introduces PagedAttention, a memory management technique that significantly improves GPU memory utilization during inference, enabling higher concurrent request throughput than traditional serving approaches [40], as shown in Figure 2.24.

Fig. 2.24 : vLLM Inference Engine [40]

llama.cpp. A lightweight C/C++ inference library focused on running LLMs efficiently on CPU hardware. It supports a wide range of quantization formats and is the engine underlying Ollama's model execution [38], as shown in Figure 2.25.

Fig. 2.25 : llama.cpp Inference Library [38]

6.3 Hardware Constraints and Optimization

On-premises deployment imposes strict hardware constraints that influence every architectural decision. Unlike cloud environments where resources can be scaled elastically, a local deployment must operate within fixed Random Access Memory (RAM), VRAM, and compute budgets. Key considerations include :

- **Model Size vs. Quality Trade-off :** Smaller quantized models run faster and require less memory, but may sacrifice output quality. The choice of model size and quantization level must balance response quality against the available hardware resources.
- **Inference Latency :** Users expect near-instantaneous responses from a conversational assistant. Model quantization and efficient batching strategies help reduce the time between receiving a query and producing the first output token.
- **Concurrent Users :** The system must be capable of serving multiple users simultaneously without degrading response times. This requires careful memory management and potentially request queuing mechanisms.

7 Comparative Study of Existing Legal AI Solutions

To position our project within the broader landscape of legal AI, Table 2.2 presents a structured comparison of existing solutions against the specific requirements identified in our problem statement.

TAB. 2.2 : Comparative Analysis of Existing Legal AI Solutions

Criteria	**Harvey AI**	**Doctrine.fr**	**CaseText**	**Luminance**	**Our Solution**
Target Jurisdiction	Common Law (US/UK)	French Law	US Law	Multi-jurisdiction	Tunisian Law
Tunisian Legal Knowledge	None	None	None	None	Fine-tuned on Tunisian corpus
Language Support	English	French	English	English	Arabic & French
Deployment Model	Cloud-only	Cloud-only	Cloud-only	Cloud-only	On-premises
Data Sovereignty	No	No	No	No	Full
Knowledge Update Mechanism	Periodic retraining	Database updates	Database updates	Periodic retraining	RAG pipeline
Domain Adaptation Method	Prompt engineering	Custom search engine	Prompt engineering	Proprietary AI	Fine-tuned LLM + RAG
Open Source	No	No	No	No	Yes

The comparative analysis reveals that no existing solution addresses the combination of requirements central to our project : deep specialization in Tunisian law, support for both Arabic and French, full on-premises deployment with data sovereignty guarantees, and a hybrid fine-tuning plus RAG approach for domain adaptation. Each existing platform is either jurisdiction-locked to a foreign legal system, dependent on cloud infrastructure, or both. This gap validates the need for the specialized system proposed in Chapter I and provides a clear positioning for our contribution.

8 Functional and Non-Functional Requirements

Before proceeding to the design and implementation of our solution, it is essential to formalize the requirements that the system must satisfy. These requirements are derived from the problem analysis conducted in Chapter I, the technical constraints identified in the preceding sections, and the operational needs expressed by E-Tafakna and ATI Tunisie.

8.1 Functional Requirements

Functional requirements define what the system must be able to do, the concrete capabilities it must provide to its users and to the broader platform it integrates with.

Tab. 2.3 : Functional Requirements

ID	Requirement	Description
RF01	Legal Question Answering	The system must be able to receive a legal question in Arabic or French and produce an accurate, contextually relevant answer grounded in Tunisian legislation.
RF02	Document-Grounded Responses	Each response must be supported by specific legal texts retrieved from the knowledge base. The system must cite the source documents (law number, article, decree) used to generate its answer.
RF03	Corpus Ingestion and Indexing	The system must provide a mechanism to ingest new legal documents (in PDF or text format), process them, and add them to the vector database for future retrieval.
RF04	Bilingual Support	The system must handle queries and produce responses in both Arabic and French, reflecting the bilingual nature of the Tunisian legal corpus.
RF05	Conversational Interface	The system must expose a conversational interface (via API) that allows natural language interaction, maintaining context across multiple turns within a session.
RF06	Platform Integration	The system must provide API endpoints compatible with E-Tafakna's existing platform, enabling integration with both Elyssa and Kahina without requiring changes to the user-facing interface.
RF07	Knowledge Base Update	The system must support the addition of new legal documents to the knowledge base without requiring model retraining, ensuring that legislative updates can be incorporated dynamically through the RAG pipeline.

8.2 Non-Functional Requirements

Non-functional requirements define the quality attributes that the system must exhibit, constraints on how it performs its functions rather than what functions it provides.

TAB. 2.4 : Non-Functional Requirements

ID	Requirement	Description
RNF01	On-Premises Deployment	The entire system (language model, RAG pipeline, and vector database) must run on ATI Tunisie's local infrastructure with no dependency on external cloud services or APIs.
RNF02	Data Sovereignty	All legal data processed by the system must remain within ATI Tunisie's secure environment. No data may be transmitted to external servers at any stage of processing or inference.
RNF03	Response Latency	The system must generate a complete response within a reasonable time frame (target : under 30 seconds for a typical legal query), ensuring a fluid conversational experience for end users.
RNF04	Legal Accuracy	Responses must demonstrate a measurably higher accuracy on Tunisian legal questions compared to a generic, non-fine-tuned baseline model, as validated through the evaluation protocol defined in the DSR methodology.
RNF05	Scalability	The system architecture must support serving multiple concurrent users without significant degradation of response quality or latency.
RNF06	Maintainability	The system must be designed with clear separation of concerns (model serving, retrieval pipeline, and API layer) to facilitate future maintenance, updates, and component-level improvements.
RNF07	Security	Access to the system's API endpoints must be secured through authentication mechanisms, and all communication between components must occur within the local network perimeter.

9 Conclusion

This chapter provided a comprehensive review of the technological foundations and the current state of research underpinning our project. We examined the architecture and capabilities of modern LLMs, explored fine-tuning techniques, from full fine-tuning to parameter-efficient methods such as LoRA and QLoRA, and detailed the RAG paradigm as a solution to the hallucination problem inherent in language models.

We then surveyed the specific challenges of applying NLP to legal texts, reviewed existing legal AI solutions across different jurisdictions, and confirmed that none addresses the unique combination of requirements central to our project : deep specialization in Tunisian law, bilingual Arabic-French support, and fully on-premises deployment with data sovereignty guarantees. The technical challenges of local LLM deployment, including model quantization and inference framework selection, were also examined.

Finally, we formalized the functional and non-functional requirements that will guide the design and implementation of our system, providing a clear specification against which the final artifact can be evaluated.

The following chapter will present the detailed design and architecture of our solution, translating these requirements into concrete technical choices and system components.

Chapitre III

DESIGN AND DEVELOPMENT

1 Introduction

This chapter covers the design and construction phase of the DSR lifecycle, translating the three objectives defined in Section 2 into a working system. It is organized in two parts. The first builds the retrieval backbone of the legal assistant : acquiring official Tunisian legal texts, structuring individual articles, validating their quality, and indexing them so that the most relevant ones can be surfaced in response to any user question ; this is implemented as a RAG pipeline. The second adapts a base language model to understand and respond within the vocabulary, citation conventions, and reasoning patterns of Tunisian law, grounding its answers in the retrieved articles rather than general-purpose knowledge ; this is achieved through supervised fine-tuning.

2 Definition of Objectives

The problem analysis conducted in Chapter I identified two fundamental gaps in E-Tafakna's existing system : the absence of Tunisian legal grounding in the AI agents, and an architecture that depends on external cloud services incompatible with ATI Tunisie's on-premises requirements. Addressing these gaps translates into three concrete, measurable objectives that govern the design of the artifact :

- **O1: Domain Adaptation.** Fine-tune a base language model on a curated corpus of Tunisian legal texts so that it understands the specific vocabulary, structure, and reasoning patterns of Tunisian law more accurately than a general-purpose model.
- **O2: Dynamic Knowledge Retrieval.** Design and implement a RAG pipeline connecting the language model to an indexed knowledge base of legal documents, grounding responses in retrieved legal evidence at inference time and allowing the knowledge base to be updated independently of model retraining.
- **O3: On-Premises Deployment.** Build and deploy the complete system on ATI Tunisie's local infrastructure using open-source tools, ensuring full data sovereignty with

no dependency on external APIs or cloud services at inference time.

Table 3.5 maps each objective to the problem it addresses and the validation criterion that will be used to assess it in Chapter IV.

TAB. 3.5 : Objectives, Problems Addressed, and Evaluation Criteria

ID	Objective	Problem Addressed	Evaluation Criterion
O1	Domain Adaptation via Fine-Tuning	Lack of Tunisian legal knowledge in general-purpose models	Legal Q&A accuracy compared to a generic baseline
O2	Dynamic Retrieval via RAG	Knowledge bounded by training data ; cannot update without retraining	Retrieval precision on held-out legal queries
O3	On-Premises Deployment	Dependency on external cloud APIs	Complete inference with zero external calls

3 Part I : RAG System Implementation

3.1 System Architecture

3.1.1 Overall Architecture

The RAG system is built around two distinct flows that share the same data layer. The first is an data pipeline responsible for acquiring, processing, and indexing the Tunisian legal corpus. This pipeline runs once at system initialization and is triggered again whenever new legal texts are published. The second is a query-time flow that handles user requests at runtime : a question is embedded, the vector database is searched for the most semantically relevant legal articles, and the retrieved content is injected into the language model's prompt before generating a response. Both flows run entirely on-premises. Figure 3.26 provides a visual overview of both flows and their shared data layer.

Both flows share the same embedding model and vector database, ensuring that the representations built at indexing time are identical to those used at retrieval time. No data leaves the local environment at any stage of either flow.

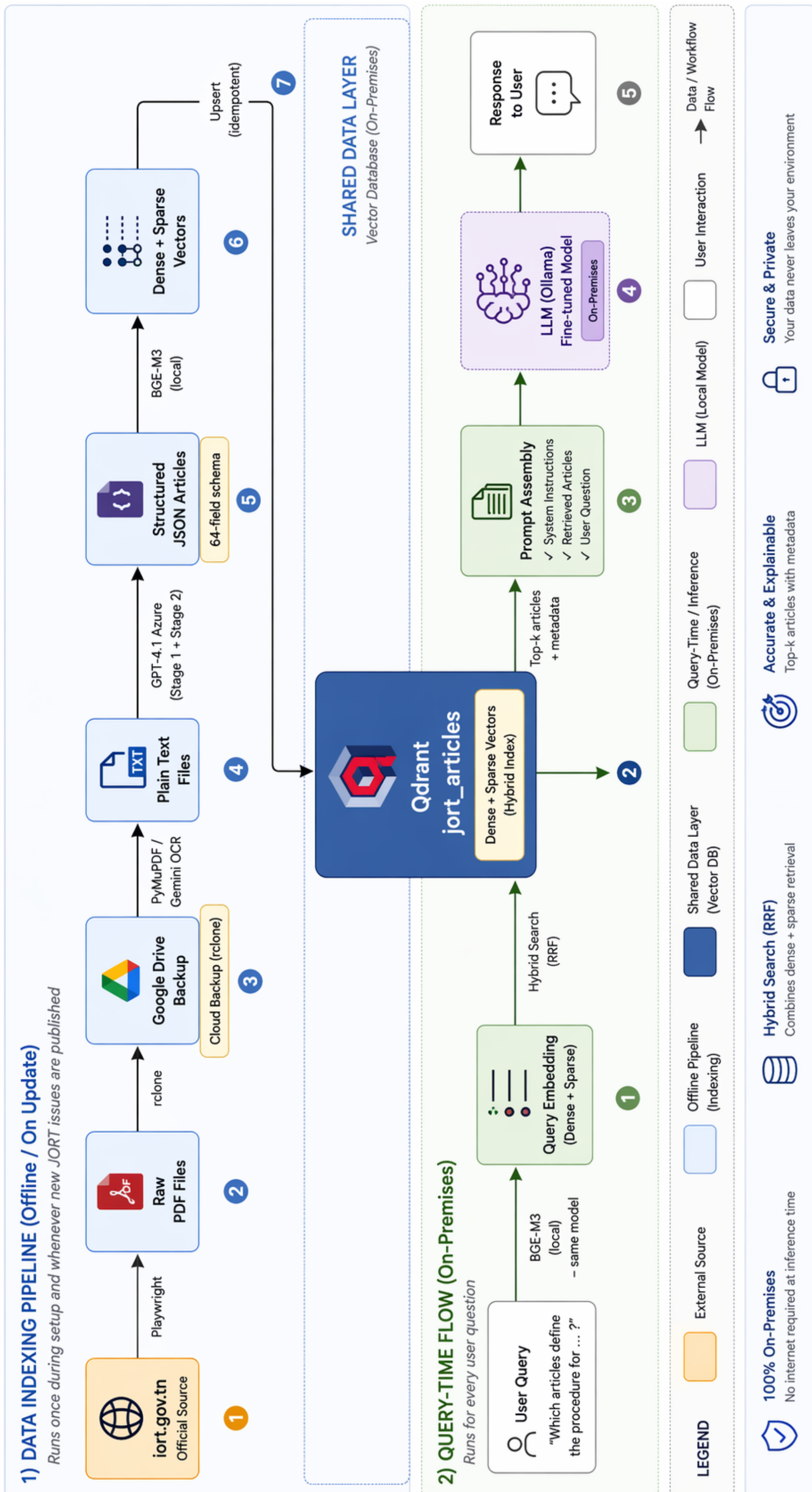

Fig. 3.26 : Overall Architecture of the Legal AI System

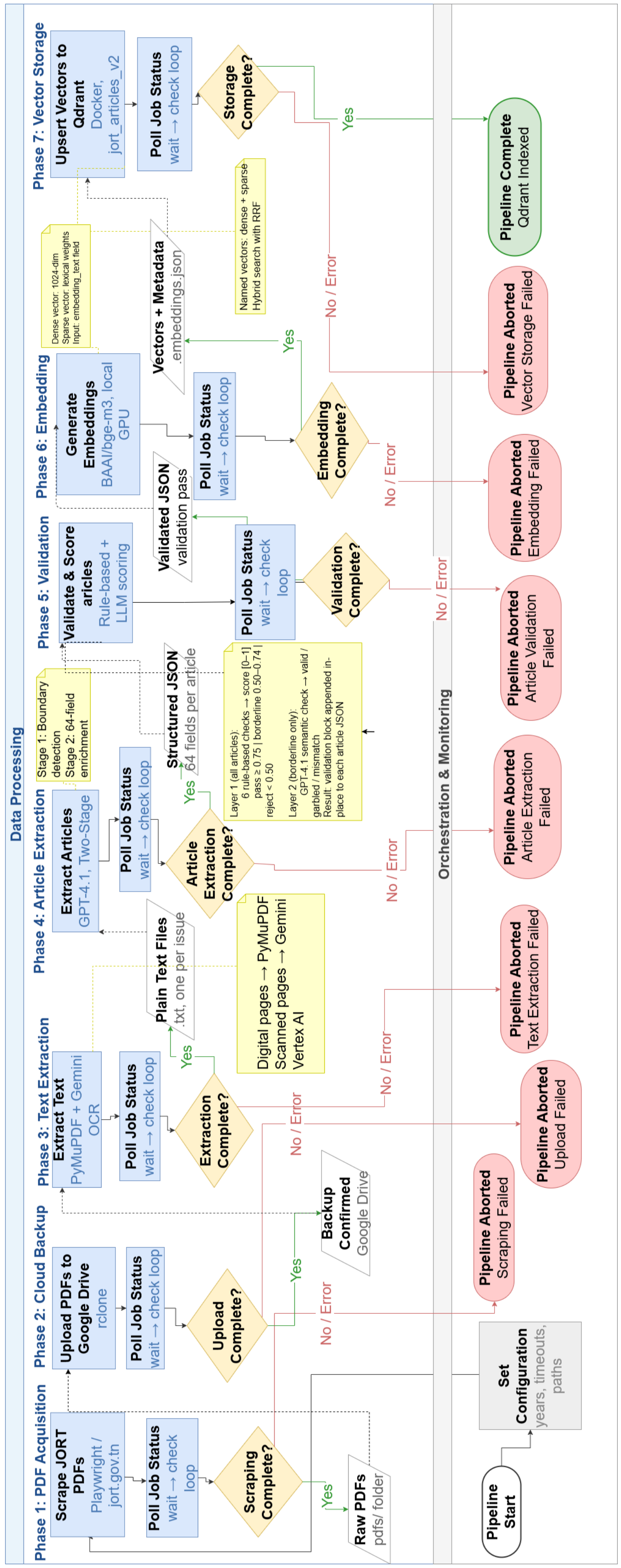

Fig. 3.27 : Flowchart of the Data Pipeline

3.1.2 Pipeline Data Flow

The pipeline consists of seven sequential phases, each producing structured output consumed by the next. Figure 3.27 illustrates the complete data flow from raw PDF acquisition to a queryable vector database.

Table 3.6 summarizes each phase, its inputs and outputs, and the key tool responsible for its execution.

Tab. 3.6 : Pipeline Phases, Inputs, Outputs, and Key Tools

#	**Phase**	**Input**	**Output**	**Key Tool**
1	PDF Acquisition	iort.gov.tn	Raw PDFs	Playwright
2	Cloud Backup	Raw PDFs	Google Drive copy	rclone + n8n
3	Text Extraction	Raw PDFs	Plain text files	PyMuPDF + Gemini
4	Article Extraction	Plain text files	Structured JSON	GPT-4.1 (Azure)
5	Validation & Scoring	Structured JSON	Validated JSON	Python + GPT-4.1
6	Embedding	Validated JSON	Dense + sparse vectors	BAAI/bge-m3
7	Vector Storage	Vectors + metadata	Qdrant collection	Qdrant

3.1.3 Technology Stack

Table 3.7 presents the complete technology stack organized by functional layer.

TAB. 3.7 : Technology Stack by Functional Layer

Layer	Technology	Role
Web Scraping	Playwright (Python) [41]	Browser automation for iort.gov.tn
Text Extraction	PyMuPDF [42] + Gemini [9]	Digital and scanned PDF text extraction
Article Structuring	GPT-4.1 (Azure OpenAI) [8]	Two-stage semantic article extraction
Embedding	BAAI/bge-m3 [33]	Bilingual dense and sparse vectorization
Reranking	BAAI/bge-reranker-v2-m3 [43]	Cross-encoder reranking of retrieval candidates
Vector Storage	Qdrant (Docker) [31]	Hybrid semantic and lexical search
API Layer	FastAPI [15]	HTTP interface for each pipeline phase
Orchestration	n8n (self-hosted) [14]	Automated workflow chaining and monitoring
Development	Python [13] + VS Code [12]	Scripting and pipeline development

3.2 Pipeline Orchestration

3.2.1 FastAPI Service Layer

Each of the seven pipeline phases is exposed as an asynchronous HTTP endpoint through a FastAPI [15] application. This design allows any phase to be triggered independently, monitored, and retried without rerunning the full pipeline. All endpoints follow a consistent job-based pattern :

- A `POST` request to the phase-specific endpoint starts a background job and returns a job identifier.
- A `GET` request to `/legal_extraction/status/{job_id}` returns the current status (`queued`, `running`, `done`, or `failed`), along with timing information and any associated error message.

This uniform interface decouples the orchestration logic from the implementation details of each phase, making it straightforward to replace or extend any phase without modifying the workflow that drives it.

3.2.2 n8n Workflow Automation

The n8n [14] workflow connects the seven pipeline phases into a single automated sequence. n8n is a self-hosted, open-source workflow automation platform that communicates with external services through configurable HTTP request nodes. Its self-hosted deployment model ensures that no data or credentials are transmitted to any external service, in line with the project's data sovereignty requirements.

The workflow follows a consistent pattern for each phase, as illustrated in Figure 3.28 :

1. **Trigger :** An HTTP node calls the FastAPI endpoint for the current phase and retrieves the job identifier.
2. **Notify :** A notification node sends an immediate message confirming the job has started.
3. **Poll :** A Wait node pauses for 30 to 60 seconds, followed by an HTTP node that polls the status endpoint.
4. **Branch :** A Switch node routes execution based on the returned status. A `done` status advances to the next phase ; a `failed` or `error` status triggers an error notification and halts the pipeline ; a `running` or `queued` status loops back to the Wait node.
5. **Guard :** An IF node at the beginning of each subsequent phase verifies the success of the previous one before triggering the next job.

The complete workflow consists of 50 nodes covering all seven phases, with notifications at each transition point for real-time monitoring without requiring active observation of the pipeline execution.

FIG. 3.28 : n8n Workflow : Automated Pipeline Orchestration

Each of the seven phases visible in Figure 3.28 is detailed individually in the sections that follow, with a dedicated sub-workflow diagram showing the nodes specific to that phase : Phase 1 in Figure 3.29, Phase 2 in Figure 3.30, Phase 3 in Figure 3.31, Phase 4 in Figure 3.32, Phase 5 in Figure 3.33, Phase 6 in Figure 3.34, and Phase 7 in Figure 3.35.

3.3 Legal Corpus Acquisition

3.3.1 Source : Journal Officiel de la République Tunisienne

The primary source for the legal corpus is the Journal Officiel de la République Tunisienne (JORT), the official gazette of the Tunisian state, published by the government and accessible at iort.gov.tn. The JORT is the authoritative publication for all Tunisian legislation and covers three main sections :

- **Lois, Décrets, Décisions et Avis :** The primary legislative section, containing laws, presidential and governmental decrees, ministerial decisions, and official notices. This section forms the core of the legal corpus.
- **Annonces Légales :** Legal notices covering sharia court decisions, judicial announcements, and commercial registrations.
- **Tribunal Foncier :** Land registry court rulings on property rights and real estate matters.

Each section is accessible through both Arabic and French interfaces on the website, yielding six distinct scraping targets. Coverage spans from [START_YEAR] to [END_YEAR], producing a corpus of [N] JORT issues and [N] PDF documents across all sections.

3.3.2 Downloading the Legal Corpus : Automated PDF Acquisition

The JORT website is a legacy application built on the WinDev/WebDev framework and exposes no public API. Automated acquisition therefore requires direct browser interaction. We implemented a Python scraping system using Playwright [41], a browser automation library that drives a Chromium instance to navigate the site, select the target year and issue, and trigger the PDF download for each entry.

Each scraping script is fully resumable. Progress is tracked in a checkpoint JSON file that records the download status of every issue as `downloaded`, `skipped`, or `failed`. Re-running the script on a partially completed session safely skips already-downloaded files without duplicating work. Download triggers differ across journal sections, as the site uses different WinDev named anchors and JavaScript calls for each section, requiring section-specific navigation logic.

The scripts are not invoked directly during pipeline execution. Instead, they are exposed through a FastAPI endpoint (`POST /legal_extraction/scraping/run`), which launches the scraper as a background job and returns a job identifier for status polling. Figure 3.29 shows the corresponding n8n sub-workflow for this phase.

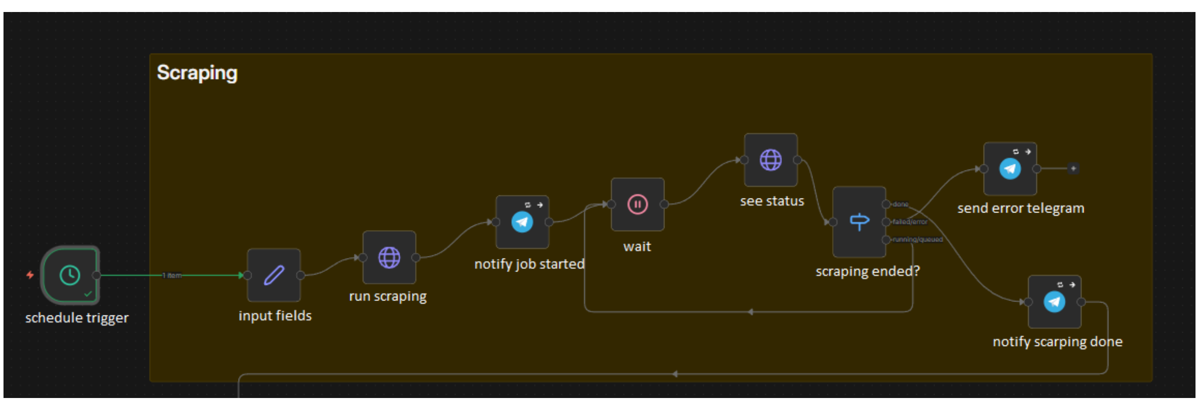

FIG. 3.29 : n8n Workflow : Phase 1 – PDF Acquisition

3.3.3 Cloud Backup

Downloaded PDFs are mirrored to Google Drive using rclone, a command-line utility for syncing local directories to cloud storage. This backup step is triggered automatically after each successful scraping run by the n8n orchestration workflow. The folder tree on Google Drive mirrors the local directory layout, preserving the section and year hierarchy for easy cross-referencing. The backup provides redundancy against local disk failure and gives the team a shared access point for the raw corpus during the project. Figure 3.30 shows the n8n sub-workflow for this phase.

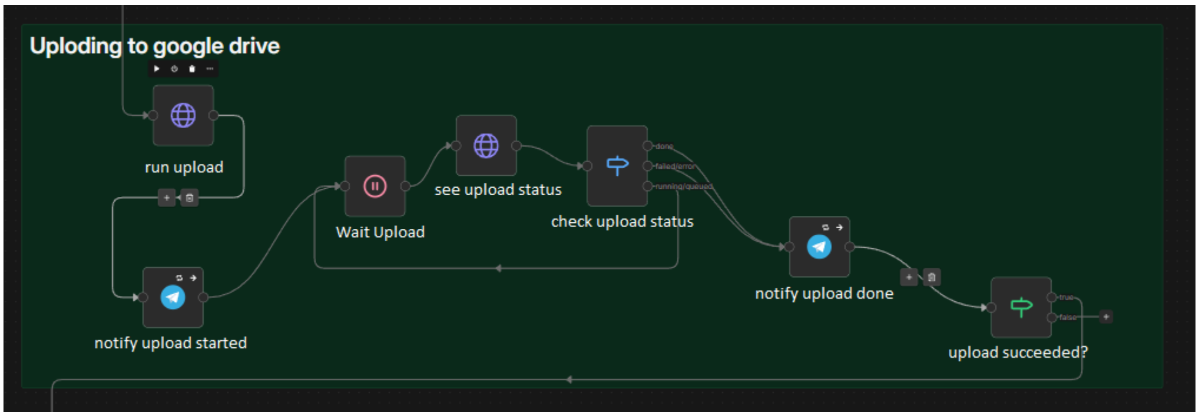

FIG. 3.30 : n8n Workflow : Phase 2 – Cloud Backup

3.4 Text Extraction and Article Structuring

3.4.1 Reading the Documents : Hybrid PDF-to-Text Conversion

The JORT corpus contains two fundamentally different categories of PDF : digitally generated files, where the text is encoded as a selectable text layer, and scanned files, where each page is stored as a rasterized image with no embedded text. These two categories require distinct extraction strategies, applied on a per-page basis.

For each page, a routing decision is made based on two conditions. A page is treated as digital and processed with PyMuPDF [42] if it contains more than 50 characters of selectable text and its image content occupies less than 15% of the page area. Pages that fail either condition are sent to Gemini via Google Vertex AI [9] for OCR.

The Gemini model receives a rendered image of the page and is prompted to return the extracted text in a structured format : Arabic tables are preserved as markdown pipe tables with right-to-left column ordering, and non-table content is returned as plain prose. This routing strategy ensures high-quality extraction across the full diversity of the corpus, combining the speed of direct text-layer extraction for the majority of pages with the accuracy of a vision-capable model for scanned content.

The output of this phase is one plain text file per JORT issue, with each page separated by a header marker that preserves page boundary information for downstream parsing.

Figure 3.31 shows the n8n sub-workflow for this phase.

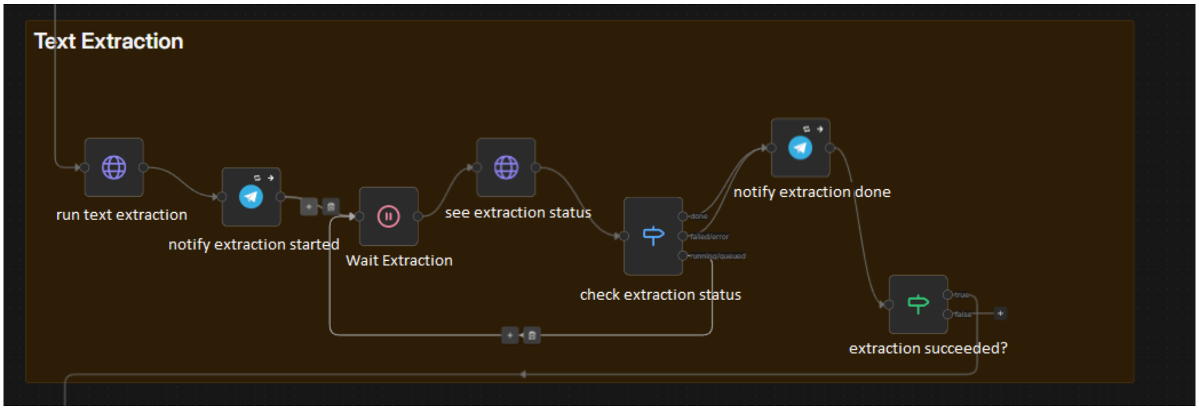

Fig. 3.31 : n8n Workflow : Phase 3 – Text Extraction

3.4.2 Isolating Each Legal Article : Article-Level Extraction Using GPT-4.1

Converting plain text JORT issues into individually structured legal articles is the most semantically demanding step of the pipeline. Rather than applying a generic text-splitting strategy, we designed a two-stage extraction pipeline powered by GPT-4.1 [8] that identifies and enriches legal articles as the natural semantic unit of the corpus.

Stage 1: Boundary Detection. The model receives the full text of a JORT issue and identifies the boundaries of each individual legal article within the document. It returns a raw list of article objects containing the article text, number, title, parent law name, and chapter. This stage operates at the document level, allowing the model to reason about the hierarchical structure of the issue before committing to individual article extractions.

Stage 2: Semantic Enrichment. Each article identified in Stage 1 is passed back to the model individually for deep enrichment. The model expands each raw article into a rich schema spanning approximately 45 fields across identification, content, legal analysis, and retrieval metadata. Table 3.8 presents the key field groups ; a complete field-by-field reference is provided in Annex A.

Tab. 3.8 : Legal Article Schema : Key Field Groups

Group	**Representative Fields**
Identification	`jurisdiction`, `law_type`, `law_number`, `year`, `status`
Content	`title_french`, `title_arabic`, `content_french`, `content_arabic`
Summaries	`summary_french`, `summary_arabic`
Legal Analysis	`keywords`, `legal_domains`, `legal_concepts`, `business_impact`
Structural Flags	`has_obligations`, `has_penalties`, `has_deadlines`, `is_abrogation`
Source	`source_name`, `source_date`, `parent_document_id`
Retrieval	`embedding_text` (purpose-built combination of title, content, and metadata)

Among these fields, `embedding_text` deserves particular attention. Rather than embedding raw article content directly, Phase 4 constructs a dedicated text field that combines the article title, body, key legal concepts, and the parent law name into a single coherent passage. This deliberate construction ensures that the resulting embedding captures both the specific content of the article and its broader legal context, which leads to meaningfully better retrieval quality compared to embedding the raw text alone.

The pipeline includes a fallback chain to handle extraction failures gracefully. If Stage 1 fails after retries, a regular expression (`Article \d+`) identifies article boundaries as a heuristic fallback. If Stage 2 enrichment fails for a specific article, a minimal schema is generated from the raw content with empty enrichment fields, ensuring the pipeline continues without data loss. Figure 3.32 shows the n8n sub-workflow for this phase.

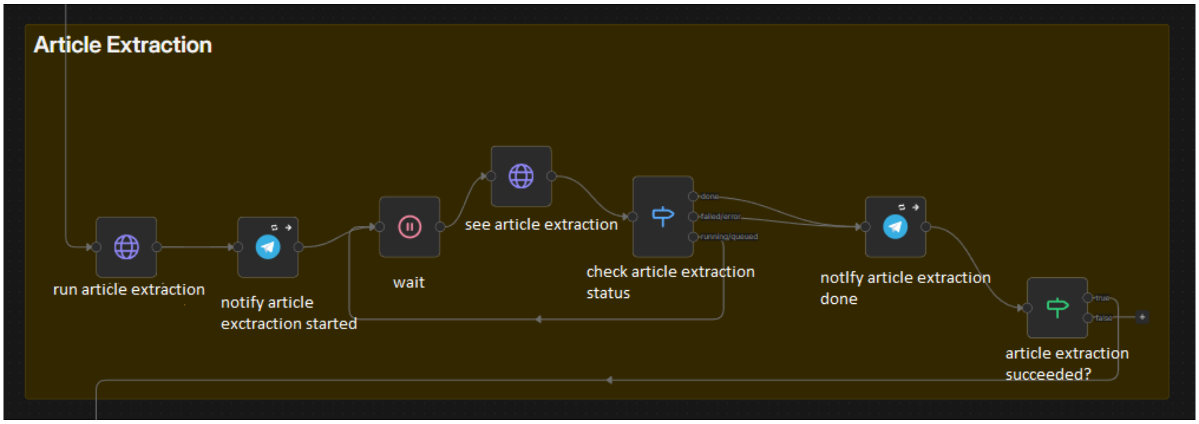

FIG. 3.32 : n8n Workflow : Phase 4 – Article Extraction

3.4.3 Why Article-Level Extraction Is Preferable to Generic Chunking

As discussed in Chapter II, the three most common chunking strategies for RAG pipelines are fixed-size, recursive, and semantic. For Tunisian legal texts, none of these generic approaches is fully satisfactory. Fixed-size chunking frequently splits articles across chunk boundaries, severing the logical connection between an article's conditions and its obligations. Recursive chunking, while more aware of sentence and paragraph boundaries, remains blind to the hierarchical structure of legal documents and may group unrelated provisions from adjacent articles.

The two-stage GPT-4.1 extraction implements a structurally superior form of semantic chunking, guided by the actual legal architecture of the document rather than arbitrary size thresholds. Each resulting unit is a complete, self-contained legal article enriched with metadata that enables precise filtering and ranking at retrieval time. In Tunisian legal texts, the article is simultaneously the natural unit of legislative meaning and the natural unit of legal citation, making article-level segmentation the most semantically faithful representation of the corpus.

3.4.4 Keeping Only Quality Articles : Two-Layer Validation and Scoring

Before any article proceeds to embedding, it passes through a two-layer quality gate designed to filter out records that are too incomplete or malformed to be useful in retrieval.

Layer 1: Rule-Based Scoring. Every article is evaluated against six deterministic checks, each carrying a weight. The checks cover the presence of required fields, minimum body length, OCR noise ratio, date format correctness, language consistency between the Arabic and French fields, and the existence of a non-empty `embedding_text` field. Weights sum to 1.0, and an article's score is the sum of the weights of all passing checks. An article scoring 0.75 or above is marked as passed and moves directly to embedding. An article scoring below 0.50 is rejected outright. Articles in the borderline range, between 0.50 and 0.74, proceed to Layer 2.

Layer 2: LLM Semantic Check. For borderline articles, a single GPT-4.1 call assesses whether the article body is semantically coherent with its title. The model responds with one of three labels : `valid` (coherent legal text that matches the title), `garbled` (body contains OCR errors or incoherent content), or `mismatch` (body exists but is unrelated to the title). A `valid` label raises the score by 0.10 and may lift the article above the passing threshold ; the other labels lower the score and add a diagnostic flag. Layer 2 is invoked only on borderline cases, keeping API costs low.

The output of this phase is the same set of JSON files enriched in place, with a `validation` block appended to each article recording its score, pass/fail status, and any diagnostic flags. The embedding phase reads this block and skips any article where `passed` is `false`, preventing low-quality records from entering the vector database. Figure 3.33 shows the corresponding n8n sub-workflow.

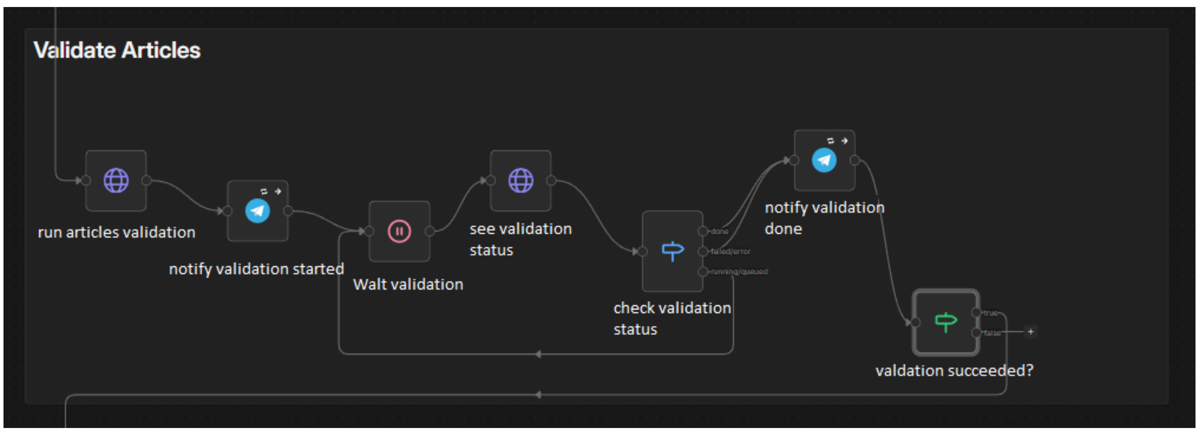

FIG. 3.33 : n8n Workflow : Phase 5 – Validation and Scoring

3.5 Embedding and Vector Storage

3.5.1 Turning Text into Searchable Numbers : Embedding Model (BAAI/bge-m3)

Each validated article is converted into a vector representation using BAAI/bge-m3 [33], a multilingual embedding model developed by the Beijing Academy of Artificial Intelligence. The model runs entirely on local hardware using the `FlagEmbedding` Python library, with no external API call required at any stage of the embedding process.

For each article, bge-m3 produces two complementary representations from the `embedding_text` field in a single encoding pass :

- **Dense vector :** A 1024-dimensional floating-point array encoding the semantic meaning of the text. Dense vectors support approximate nearest-neighbor search and surface results that are semantically related to a query even when the exact phrasing differs.

- **Sparse vector :** A set of token-level lexical weights, stored as a mapping from token identifiers to weight values. Sparse vectors support precise keyword matching, particularly effective for retrieving articles that contain specific legal terms, article numbers, or law references.

Both representations are generated alongside all article metadata and persisted to disk before being loaded into the vector database in the subsequent phase. The model operates with 16-bit floating-point precision (`use_fp16=True`) to remain within the VRAM budget of the available GPU while preserving embedding quality. Figure 3.34 shows the n8n sub-workflow for this phase.

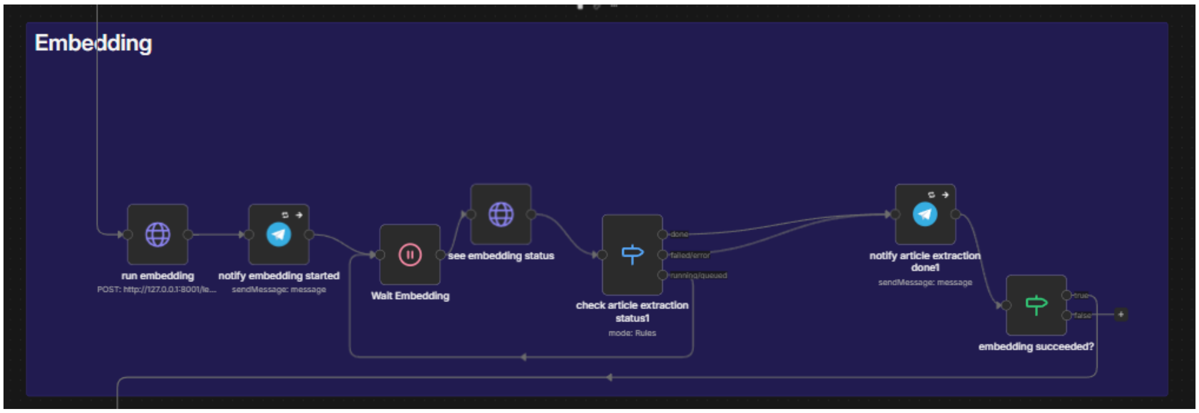

Fig. 3.34 : n8n Workflow : Phase 6 – Embedding

3.5.2 Where the Articles Are Stored : Hybrid Vector Database (Qdrant)

The embeddings and metadata produced in Phase 6 are stored in Qdrant [31], an open-source vector database running in a local Docker container. Qdrant was selected over alternatives such as FAISS and Chroma for three reasons : its native support for named vector fields enabling hybrid dense and sparse search within a single collection, its production-grade stability under concurrent queries, and its fully self-hosted deployment model that aligns with the on-premises constraints of the project.

The collection, named `jort_articles_v2`, is configured with two named vector fields :

- **dense** (1024 dimensions, cosine distance) : supports approximate nearest-neighbor search over the semantic embedding space.
- **sparse** (lexical weights, dot product) : supports exact and near-exact keyword matching using the sparse representations produced by bge-m3.

At query time, both vector types are searched simultaneously and their result sets are merged using Reciprocal Rank Fusion (RRF), a score combination technique that aggregates rankings from multiple retrieval signals without requiring manual weight calibration. This hybrid approach handles legal queries that may match an article semantically, such as a paraphrased question about contract validity, just as well as queries that rely on exact terminology, such as a direct citation like “Article 2 du Code des Obligations et des Contrats”.

Ten payload fields are indexed for filtered retrieval, including `year`, `law_type`, `institution`, `legal_domains`, `has_obligations`, `has_penalties`, `is_abrogation`, and `status`. These filters allow the retrieval layer to scope a search to a specific legal category or time period without scanning the full collection. Insertion into the database is idempotent : each article carries a stable identifier assigned at embedding time, and Qdrant's upsert operation inserts a new record or overwrites an existing one with the same identifier, making it safe to re-run the pipeline on the same corpus. Figure 3.35 shows the n8n sub-workflow for this phase.

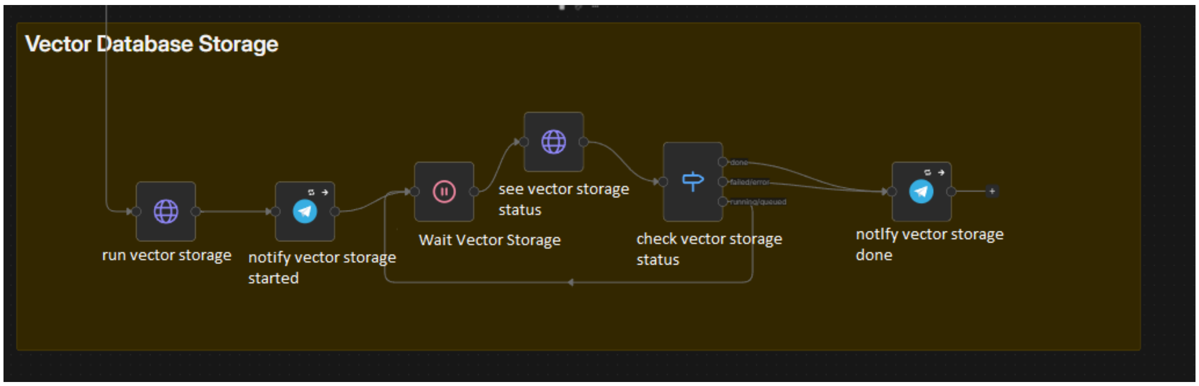

FIG. 3.35 : n8n Workflow : Phase 7 – Vector Storage

3.5.3 When a User Asks a Question : Retrieval at Inference Time

When a user submits a query to the legal assistant, the retrieval process closely mirrors the indexing process. The system first auto-detects the query language based on the proportion of Arabic Unicode characters in the input, selecting either the French or Arabic content fields for context injection accordingly. The query text is then embedded using the same bge-m3 model, producing a dense and a sparse vector, which are submitted to Qdrant as a hybrid search request. Qdrant returns an initial candidate pool of 20 articles ranked by RRF-fused score.

Before passing the results to the language model, the candidate pool is reranked by a cross-encoder model, BAAI/bge-reranker-v2-m3 [43], which scores each (question, article) pair jointly rather than comparing independent vector representations. Because the cross-encoder sees both texts together, its relevance signal is substantially more precise than vector similarity alone. The final set is trimmed to the top-k articles, where k is a tunable hyperparameter to be optimized during the evaluation phase described in Chapter IV. A confidence gate is applied at this point : if the highest reranker score falls below a minimum threshold, the language model call is skipped entirely and the system returns a bilingual "no relevant articles found" message rather than risk generating an unsupported answer.

The reranked articles are injected into the prompt of the fine-tuned language model, whose training and architecture are detailed in Part II of this chapter. Each article contributes its content in the language matching the query together with its source metadata, including the law name, article number, and publication date. This metadata enables the model to produce responses with proper legal citations, making each answer directly verifiable against the original JORT source. Figure 3.36 summarizes the complete query flow from user input to streamed answer.

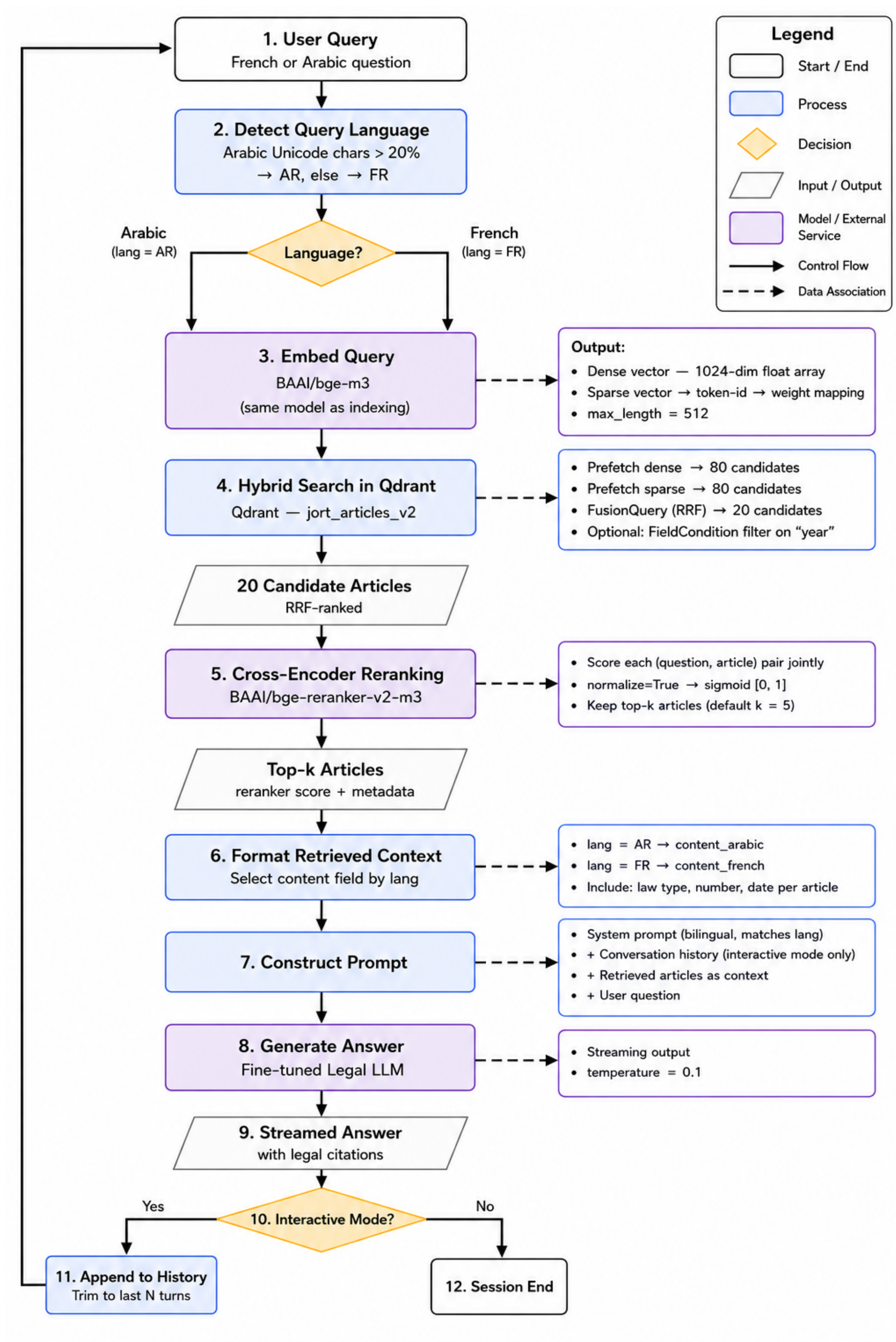

FIG. 3.36 : RAG Query Flow : From User Question to Grounded Answer

4 Part II : Fine-Tuning

4.1 Base Model Selection

4.1.1 Motivation for Using a Small Language Model

The on-premises deployment constraint defined in O3 immediately excludes any model accessible only through an external API, such as GPT-4.1 or Gemini. The entire inference stack must run on ATI Tunisie's local infrastructure, which means the chosen model must fit within the available GPU VRAM budget, respond at interactive latency, and carry a license permitting on-premises commercial deployment.

These constraints point toward open-weight SLMs in the 4B to 9B parameter range. Models of this size can be quantized and served locally with output quality that, when combined with the grounding provided by the RAG retrieval layer, is sufficient for answering structured legal questions in Arabic and French.

4.1.2 Selection Criteria

Four criteria guided the evaluation of candidate models. Table 3.9 summarizes each criterion and its requirement.

Tab. 3.9 : Model Selection Criteria

Criterion	**Requirement**
Multilingual capability	Strong Arabic and French support at both understanding and generation level
Instruction-following	Capable of following a structured prompt with a system role and retrieved context block
License	Permissive open-weight license allowing on-premises commercial use
Deployment compatibility	Exportable to GGUF format for serving via the local inference runtime

4.1.3 Candidate Models

Two open-weight models were selected for fine-tuning and direct comparison.

Qwen3.5 9B. Developed by Alibaba Cloud, Qwen3.5 9B [44] is a 9-billion-parameter model released under the Apache 2.0 license. Its architecture combines Gated Delta Networks, a form of linear attention that replaces the standard quadratic self-attention mechanism, with a sparse Mixture-of-Experts feed-forward layer. This hybrid design delivers substantially higher throughput and lower inference latency compared to dense models of equivalent parameter count. Qwen3.5 9B was pretrained on a corpus spanning 201 languages, with strong coverage of Arabic and French, and supports a native context window of 262,144 tokens. The instruction-tuned variant, Qwen3.5-9B-Instruct, follows structured prompts reliably and produces well-formed, citation-aware answers. The model is available in GGUF format for local serving.

Gemma 4 E4B. Developed by Google DeepMind, Gemma 4 E4B [45] is released under the Apache 2.0 license. The "E" in E4B stands for Effective : the model uses Per-Layer Embeddings (PLE) to share a large embedding lookup table across layers, so only the computationally active parameters are loaded into VRAM while the embedding table is accessed from cheaper memory. This architecture gives Gemma 4 E4B the memory footprint of a 4B model while retaining the representational capacity of a larger one. The model supports a context window of 128K tokens, is pretrained on 140 languages with strong out-of-the-box performance on over 35, including Arabic and French, and uses a hybrid attention mechanism that interleaves local sliding-window attention with full global attention. The instruction-tuned variant is `gemma-4-E4B-it`. GGUF quantizations are available via llama.cpp for local deployment.

4.1.4 Rationale for This Pair

Qwen3.5 9B and Gemma 4 E4B represent two distinct operating points on the size-versus-quality tradeoff. Qwen3.5 9B provides broader multilingual depth across 201 languages and benefits from a larger parameter budget and extended context, at the cost of higher VRAM consumption. Gemma 4 E4B is designed for memory-constrained hardware : its PLE architecture minimises the active VRAM footprint without sacrificing context length, making it well suited to deployment environments where GPU memory is limited. Fine-tuning both models on the same synthesized Tunisian legal dataset under identical hyperparameter configurations yields a direct empirical comparison between the two approaches.

4.2 Training Data Synthesis

4.2.1 Synthetic Data as a Training Strategy

Synthetic data is data that is artificially generated to imitate the statistical properties of real-world observations, produced through algorithms, generative models, or rule-based simulations rather than collected from actual events [46], [47]. The key distinction from anonymized or augmented data is that synthetic records are never derived from any specific real individual or event : they carry no inherent privacy risk because no original data subject underlies them.

Industry adoption has accelerated substantially over recent years. Gartner projected that by 2024, 60% of all data used for AI and analytics development would be synthetically generated, up from just 1% in 2021 [48]. The synthetic data generation market, estimated at approximately \$2 billion in 2025, is forecast to reach \$10 billion by 2033 at a compound annual growth rate of 25% [47]. This growth reflects a broad shift across data-intensive industries toward generating training data on demand rather than collecting and labelling it manually. Figure 3.37 illustrates this breadth, showing adoption across healthcare, banking, agriculture, e-commerce, manufacturing, automotive, and disaster prediction, among other sectors [47].

Fig. 3.37 : Industries and Domains Leveraging Synthetic Data (Source : [47])

For this project, synthetic data is the only viable training strategy, for three reasons. First, no annotated corpus of Tunisian legal question-answer pairs exists ; constructing one by recruiting legal professionals to formulate and verify thousands of examples would be prohibitively expensive and slow. Second, any dataset built from real user interactions with the E-Tafakna platform would contain sensitive legal consultations that cannot be used for model training without explicit user consent and formal legal review, conflicting directly with the project's data sovereignty requirements. Third, the synthesis process described in the following sections allows the distribution of question types, language balance, and legal code coverage to be specified precisely, producing a dataset that matches the target domain far more closely than any available general-purpose legal corpus.

4.2.2 Motivation and Approach

No publicly available annotated dataset exists for Tunisian legal question answering. To address this gap, a supervised fine-tuning dataset was constructed through a structured synthesis process : official Tunisian legal code PDFs were uploaded to three frontier language models, GPT 5.2, Qwen 3.6, and Claude Opus 4.6, together with a detailed generation prompt specifying format, citation rules, and answer style. The three models were used in rotation across the full dataset rather than being assigned to specific codes, which reduces stylistic uniformity and increases the diversity of formulation across examples. The resulting dataset totals 1,762 training examples.

A key design decision governs every example in the dataset : the user message is constructed in RAG format, embedding the question alongside a "*Documents pertinents*" block containing the verbatim text of the relevant legal articles. This mirrors exactly how the model will be queried at production inference time, where Qdrant retrieves articles and injects them into the prompt before the model generates a response. Training on this format ensures the model learns to ground its answers in the provided context rather than relying on parametric knowledge alone.

4.2.3 Dataset Structure and Volume

The dataset is organized in two tiers. The first, referred to as constructed data, contains 1,620 examples generated from legal code PDFs through the structured synthesis process described above. It is further divided into standard Question and Answer (Question and Answer (Q&A)) batches (1,543 examples across 20 codes) and complex multi-issue scenario batches (20 examples across 4 codes), with 77 Tunisian dialect Arabic examples distributed within the standard batches. The second tier, referred to as added data, contains 142 examples covering interaction patterns not produced by the synthesis prompt : 44 real multi-turn user conversations, 60 service-routing examples, and 38 out-of-scope refusal examples. Table 3.10 summarizes the composition and Figure 3.38 provides a visual breakdown of the full dataset by type.

Tab. 3.10 : Dataset Composition by Source and Type

Source	**Type**	**Examples**
Constructed data	Standard Q&A batches	1,543
Constructed data	Complex scenario batches	20
Constructed data	Tunisian dialect (Darija)	77
Constructed subtotal		**1,620**
Added data	Real multi-turn conversations	44
Added data	Service routing	60
Added data	Out-of-scope refusals	38
Added subtotal		**142**
Grand total		**1,762**

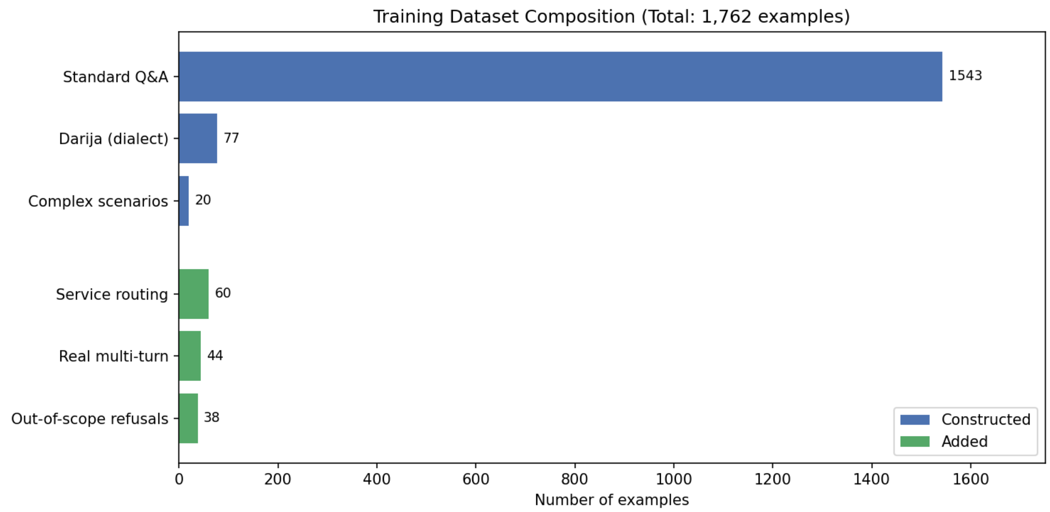

FIG. 3.38 : Fine-Tuning Dataset Composition by Type

4.2.4 Legal Codes Covered

Standard batches cover 20 distinct Tunisian legal codes, spanning civil, commercial, fiscal, maritime, and constitutional law. Complex scenario batches draw on four of those codes for multi-issue narratives. Table 3.11 lists the full set ; the complete batch-level breakdown with counts per code is provided in Annex B.

TAB. 3.11 : Tunisian Legal Codes Covered in the Training Dataset

Code	**Full Name**	**Code**	**Full Name**
CAC	Code des activités commerciales	CMM	Code maritime
CCL	Code civil (obligations/leasing)	CN	Code du notariat
CCP	Code de commerce (procédures)	COC	Code des obligations et contrats
CC	Code civil	Douanes	Code des douanes
CDET	Code des droits et engagements du trésor	Changes	Code des changes
CDIP	Code de droit international privé	Eaux	Code des eaux
CDPF	Code des droits et procédures fiscaux	Const. 2022	Constitution tunisienne 2022
CDR	Code de la route	CP	Code pénal
CFL	Code des finances locales	CTM	Code du travail maritime
CIRPP/IS	Code de l'IRPP et de l'IS	Presse	Code de la presse

4.2.5 Question Categories and Language Distribution

Each standard batch of 20 examples is distributed across six question categories. Table 3.12 lists each category, its description, and the total count across the full constructed dataset. Figure 3.39 visualises the distribution. The complete reference including all supplementary subcategories is provided in Annex B.

TAB. 3.12 : Question Categories in the Constructed Dataset

Category	Description	Count
Direct Q&A	Factual question answered by a single article	348
Multi-article synthesis	Question requiring two to three related articles	214
Principle and exception	States the general rule, then its exceptions or nuances	190
In-domain refusal	Plausible question the code does not answer ; model declines and recommends a lawyer	158
Procedural	Questions about sequential legal processes or phases	128
Clarification	Ambiguous question ; model requests clarification before answering	121
Complex scenario	Multi-issue legal narrative requiring four to eight articles	20

Language distribution within constructed batches is strictly balanced : each batch of 20 examples contains 10 French examples and 10 Arabic examples in formal Modern Standard Arabic, with two additional cross-lingual examples where the question and the source article are in different languages. The 77 Tunisian dialect examples represent a distinct informal Arabic register distributed across the standard batches. The 44 real multi-turn conversations from the added data are the only multi-turn examples in the full dataset. Figure 3.40 shows the language split across the full dataset.

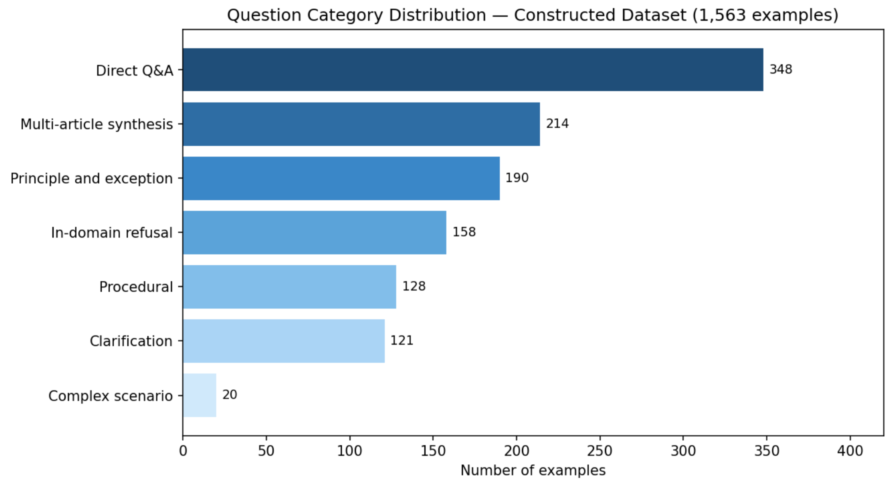

FIG. 3.39 : Distribution of Question Categories in the Constructed Dataset

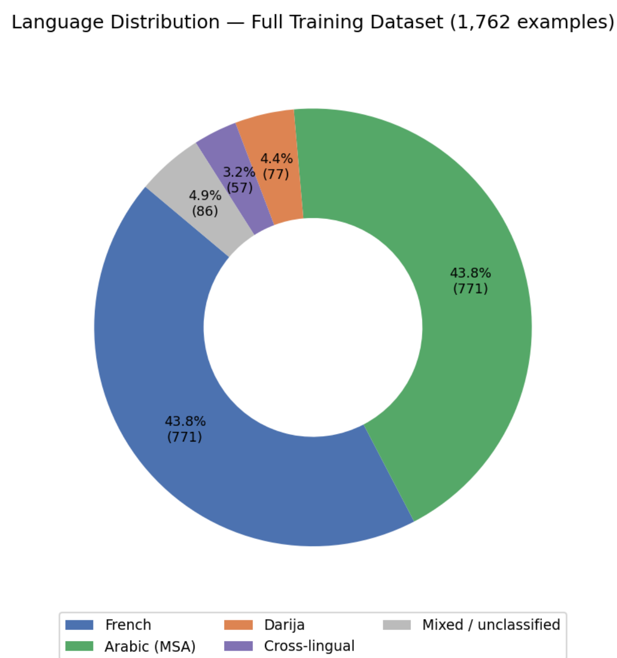

FIG. 3.40 : Language Distribution Across the Full Training Dataset

4.2.6 Anti-Hallucination Measures

Citation accuracy is the most critical quality constraint in a legal training dataset. Three measures were enforced throughout generation. First, every article cited in an answer must appear verbatim in the uploaded PDF ; no article numbers may be invented or approximated. Second, when OCR quality in a source PDF was ambiguous, a `confidence_flag` field was added to the example, marking it for manual review before inclusion in training. Third, the generation prompt explicitly instructed each model that producing fifteen solid examples with verified citations is preferable to producing twenty examples with hallucinated article numbers.

4.2.7 Human Validation by Legal Experts

Approximately 50% of the dataset was reviewed by legal professionals, with coverage distributed across all question categories and complexity levels. Reviewers assessed citation accuracy, the correctness and completeness of the legal reasoning in each answer, and the appropriateness of refusals and clarification requests. Across the full set of reviewed examples, fewer than 2% required correction, confirming that the multi-model synthesis process combined with the verbatim citation constraint produces legally sound training data at scale.

4.3 Fine-Tuning Methodology

4.3.1 Why Full Fine-Tuning Is Not Feasible

A standard full fine-tuning run updates every parameter in the model simultaneously. For a model with 7.98 billion parameters, storing the weights alone in 16-bit floating-point precision requires approximately 16 GB. During training, the Adam optimizer [49] must additionally maintain two momentum tensors per parameter, which together consume a further 32 GB in full precision. Adding gradients and intermediate activations, the total memory requirement for full fine-tuning exceeds 60 GB, well above the 32 GB ceiling of the Tesla V100S used for this project.

The solution is to freeze the base model weights entirely and attach a small number of trainable adapter parameters using LoRA [50]. Since the frozen weights do not require gradients, optimizer states and backward-pass memory are limited to the adapter parameters only, bringing the peak VRAM requirement within the available hardware budget. The two models use slightly different variants of this approach : Gemma 4 E4B is trained with standard LoRA on a full-precision base, while Qwen3.5 9B is trained with QLoRA [50], where the base model is additionally loaded in 4-bit NF4 quantization before adapters are attached, further reducing the base weight memory footprint.

4.3.2 Adapter Configuration

LoRA injects a pair of trainable low-rank matrices, $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$, alongside each frozen weight matrix W. Only the product BA is trained ; at inference time the adapter can be merged back into the base weights at no additional computational cost. QLoRA extends this by additionally quantizing the frozen base weights to 4-bit NF4 format before attaching the adapters, reducing the base model's VRAM footprint by approximately 75% relative to fp16 while keeping the adapter parameters in full precision for training.

Both models share the same adapter configuration : rank $r = 16$, scaling factor $\alpha = 32$, dropout disabled, and adapters attached to all seven linear projection layers in the attention and feed-forward blocks. The difference between the two runs is the base model precision. Gemma 4 E4B was loaded at fp16 without quantization, as its Per-Layer Embeddings (PLE) architecture already provides a compact active VRAM footprint. For Qwen3.5 9B, loading the full 9B model in fp16 (approximately 18 GiB) alongside the adapter parameters, optimizer states, and training activations on a single V100S exceeded the 32 GiB VRAM ceiling, making standard LoRA training infeasible. QLoRA resolved this by quantizing the frozen base weights to 4-bit NF4 before attaching the adapters, reducing the base model footprint to approximately 5 GiB and bringing total peak VRAM usage within budget. Gradient checkpointing was enabled for both runs using Unsloth's [51] optimized implementation. Table 3.13 summarises the full adapter configuration for each model.

TAB. 3.13 : Adapter Configuration by Model

Parameter	**Gemma 4 E4B**	**Qwen3.5 9B**
Method	LoRA	QLoRA
Base model precision	fp16	4-bit NF4
Rank (r)	16	16
Scaling factor (α)	32	32
Dropout	0 (disabled)	0 (disabled)
Target modules	`q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`	
Gradient checkpointing	Unsloth optimized	Unsloth optimized
Trainable parameters	40,583,168 (0.51%)	29,097,984 (0.31%)

4.3.3 What the Model Is Trained to Do : Response-Only Masking

The training objective is standard next-token prediction with cross-entropy loss, restricted to the assistant turns of each conversation. The `train_on_responses_only` function from the Unsloth library [51] sets the loss weight to zero for all system and user tokens, so the model is penalized only for errors in its own generated output. This is essential for the legal assistant use case : the model must learn to produce well-formed, citation-accurate answers from the retrieved context in the *Documents pertinents* block, not to reproduce the question or the article text, which it receives as input.

The Gemma 4 chat template uses `<|turn>user\n` as the instruction delimiter and `<|turn>model\n` as the response delimiter. The masking function identifies these boundaries in each tokenized example and zeroes out the loss for all tokens preceding the response delimiter.

4.3.4 Out-of-Memory Error and Parameter Adjustment

This error occurred exclusively during the fine-tuning of Gemma 4 E4B on a single V100S instance. The initial configuration used a per-device batch size of 2 and a maximum sequence length of 4,096 tokens. After loading the base model (14.8 GiB) and initializing the LoRA adapters and optimizer (26.5 GiB total), the run terminated with an out-of-memory error on the first forward pass.

The root cause is that activation memory during training scales with batch size and sequence length jointly. For a transformer layer, the activation tensors for a single forward-backward pass occupy memory proportional to batch_size × seq_len × d_{model}. With a batch size of 2 and a sequence length of 4,096, the activation tensors for a single accumulation step exceeded the remaining VRAM headroom after weights and optimizer states were loaded.

The adjustment followed directly from this analysis : the batch size was reduced from 2 to 1, and the maximum sequence length was halved from 4,096 to 2,048. To keep the effective batch size unchanged at 8, the gradient accumulation steps were doubled from 4 to 8. The revised configuration brought peak VRAM usage to 30.8 GiB, within the 32 GiB ceiling. Qwen3.5 9B was configured with the same adjusted parameters from the start, applying the lesson learned from Gemma's run. Table 3.14 summarises the before and after.

Tab. 3.14 : Parameter Adjustment Following the Out-of-Memory Error

Parameter	**Initial (failed)**	**Adjusted (final)**
Per-device batch size	2	1
Gradient accumulation steps	4	8
Effective batch size	8	8
Maximum sequence length (tokens)	4,096	2,048
Peak VRAM usage	> 32 GiB (OOM)	30.8 GiB

4.4 Training Configuration

4.4.1 Hyperparameters

Both models were trained under the same hyperparameter configuration. Table 3.15 lists the complete set of values applied to both runs.

TAB. 3.15 : Training Hyperparameters (Both Models)

Hyperparameter	**Value**
Per-device batch size	1
Gradient accumulation steps	8
Effective batch size	8
Number of epochs	3
Learning rate	2×10^{-4}
LR scheduler	Cosine decay
Warmup ratio	0.05
Optimizer	AdamW 8-bit
Weight decay	0.01
Gradient clipping (max norm)	1.0
Precision	fp16
Maximum sequence length	2,048 tokens
Sequence packing	Disabled
Random seed	42

Sequence packing was disabled to preserve the conversational integrity of each training example. Legal question-answer pairs vary considerably in length, and mixing tokens from different examples within a single packed sequence would risk the model learning spurious cross-example associations. AdamW with 8-bit precision was used to reduce optimizer memory consumption without affecting gradient quality.

4.4.2 Training Infrastructure

Both models were trained on OVHcloud [52] AI Notebook instances, as shown in Figures 3.41 and 3.42. Gemma 4 E4B was trained on a single NVIDIA Tesla V100S with 32 GiB of VRAM using standard LoRA with an fp16 base. Qwen3.5 9B was trained on a single NVIDIA Tesla V100S using QLoRA, with the base model loaded in 4-bit NF4 quantization. The quantitative results for both runs, including loss curves, training duration, and peak VRAM usage, are reported in Chapter IV alongside the evaluation experiments.

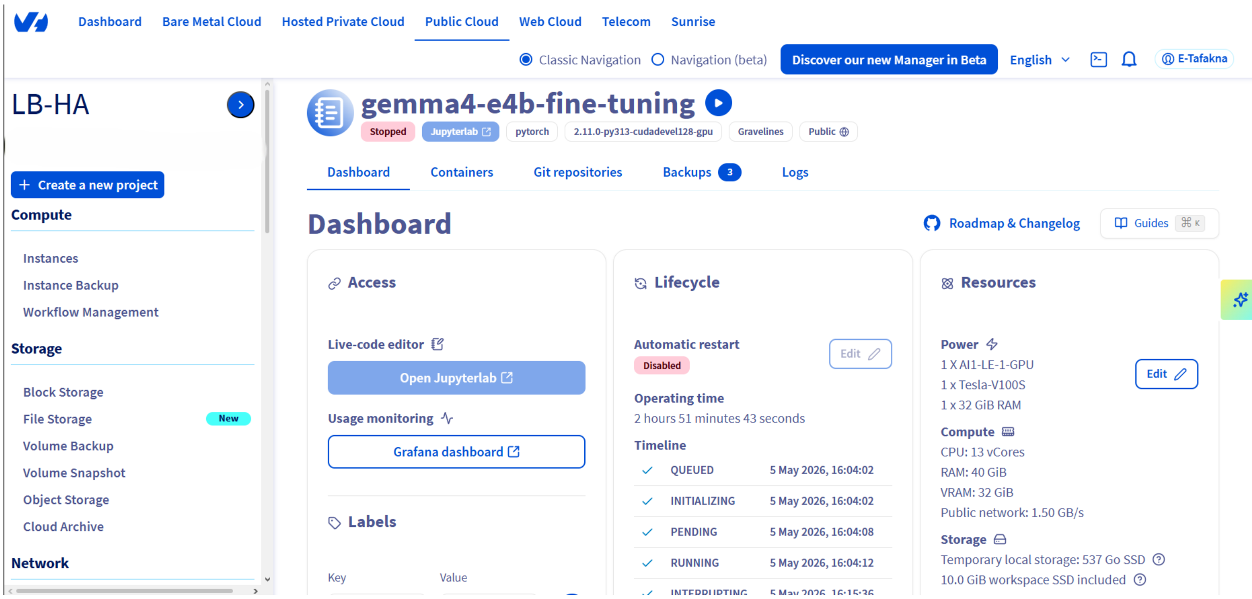

FIG. 3.41 : OVHcloud AI Notebook Instance Used for Fine-Tuning – Gemma 4 E4B

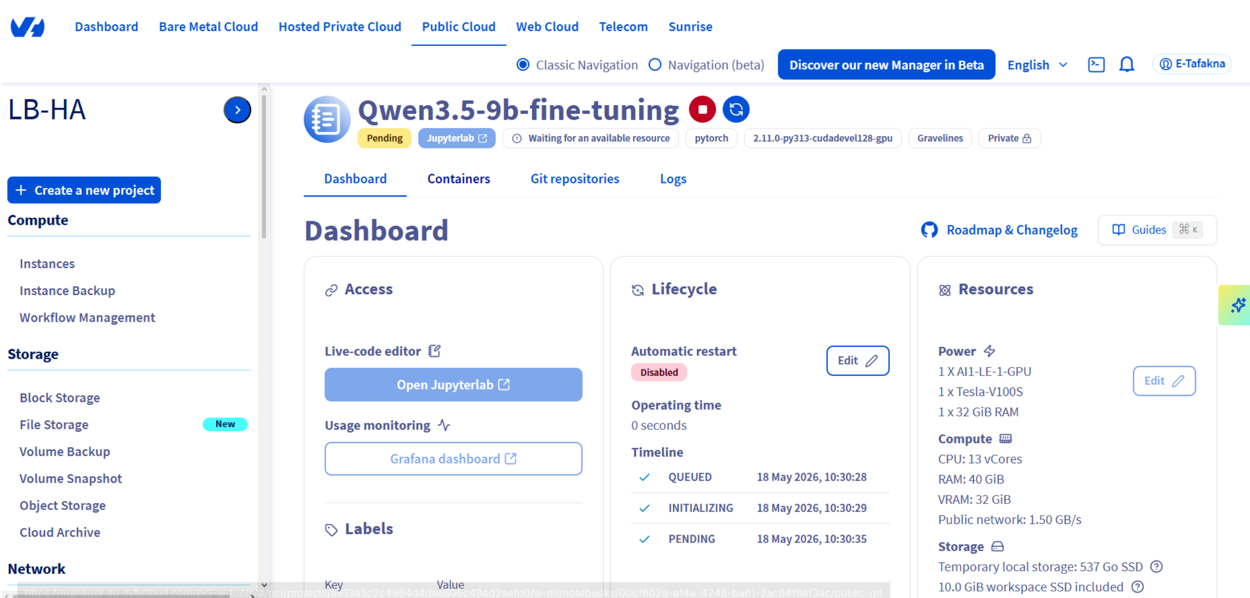

FIG. 3.42 : OVHcloud AI Notebook Instance Used for Fine-Tuning – Qwen3.5 9B

4.5 Quantization and Deployment

4.5.1 LoRA Adapter Export

On completion of training, the LoRA adapter weights were saved independently of the base model. The adapter is stored as a single `adapter_model.safetensors` file of 154.9 MB, accompanied by the tokenizer and configuration files, for a total size of 185.6 MB. This compact representation can be loaded on top of any copy of the base model at inference time without rewriting the base weights, making it straightforward to update the adapter independently of the base model in future training iterations.

4.5.2 Preparing the Model for Production : GGUF Export and Local Deployment

For on-premises deployment at ATI Tunisie, the adapter was merged with the base model and the combined weights were exported to the GGUF format using Unsloth's built-in export utility [51]. The quantization method applied was `q4_k_m`, a 4-bit scheme that groups weights into blocks and uses k-means clustering with mixed precision to assign larger quantization budgets to the layers most sensitive to precision loss. This method reduces the model's on-disk footprint to approximately one quarter of its fp16 size while preserving inference quality suitable for production use.

The resulting GGUF file is served locally using Ollama on ATI Tunisie's infrastructure, with no external API dependency at inference time, in line with objective O3 established in Section 2.

5 Conclusion

This chapter covered the design and implementation of both components of the legal assistant. Part I built the retrieval pipeline : legal texts are acquired from the official Tunisian gazette, structured into individual articles, filtered through a two-layer quality gate, embedded using bge-m3, and stored in a hybrid Qdrant collection, with the full workflow orchestrated by n8n and backed by a FastAPI service layer, entirely on-premises. Part II adapted two candidate models to the legal domain : Gemma 4 E4B using standard LoRA on an fp16 base, and Qwen3.5 9B using QLoRA with a 4-bit NF4 base to fit within the single V100S budget. Both models were exported to GGUF format for local deployment at ATI Tunisie. Their training results and comparative evaluation are presented in Chapter IV.

Chapitre IV

DEMONSTRATION AND EVALUATION

1 Introduction

This chapter closes the DSR lifecycle by putting the artifact built in Chapter III into operation and measuring it against the three objectives defined in Section 2. It is organized in two parts. The first demonstrates the system in action : what it looks like when running, what kinds of questions it can handle, and what its internal pipeline does from the moment a user submits a query to the moment a response is returned. The second part evaluates the results : training outcomes for both models, the quality of the retrieval layer, and compliance with the on-premises deployment constraint.

2 Part I : Demonstration

2.1 System Overview at Demo Time

The deployed system is split into two distinct flows that operate at different times. The first is the preparation pipeline described in Part I of Chapter III : n8n orchestrates the seven phases (scraping, upload, text extraction, article extraction, validation, embedding, and vector storage) by calling a FastAPI server that triggers each phase on demand. This pipeline runs once per corpus update and has no role at query time ; for the purpose of this demonstration, two Journal Officiel de la République Tunisienne (JORT) issues from early 2026 have been indexed into Qdrant, constituting a representative proof-of-concept corpus.

The second flow is the real-time query pipeline, which handles each user question independently. Table 4.16 lists the components that are active at query time, with no external network call involved in any of them.

TAB. 4.16 : Components Active at Query Time

Component	**Role at query time**
BAAI/bge-m3	Encodes the user query into a dense vector (1,024 dimensions) and a sparse lexical vector simultaneously
Qdrant (`jort_articles_v2`)	Runs a hybrid search over the pre-indexed corpus and returns the top-20 candidate articles
BAAI/bge-reranker-v2-m3	Scores each (query, article) pair jointly and re-orders the candidates by relevance ; score determines whether article context is injected into the prompt
Fine-tuned model via Ollama	Generates the final grounded answer from the selected articles ; served locally in GGUF q4_k_m format

Figure 4.43 shows the `jort_articles_v2` collection in the Qdrant dashboard after the full corpus has been indexed, confirming that all legal articles are stored and ready for retrieval.

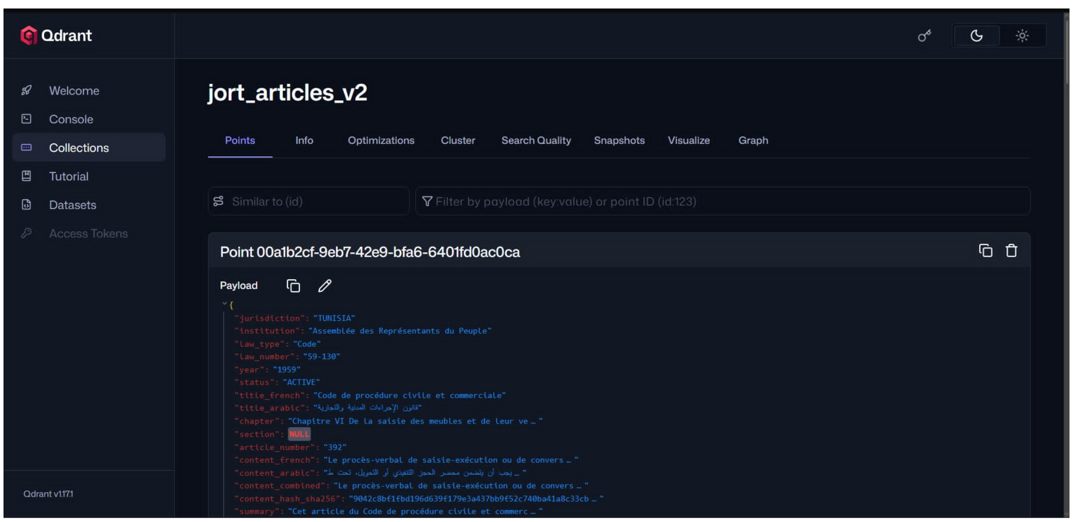

FIG. 4.43 : Qdrant Dashboard Showing the `jort_articles_v2` Collection After Indexing

Both fine-tuned models, Gemma 4 E4B and Qwen3.5 9B, are available as GGUF files and can be loaded into Ollama interchangeably. The following demonstration was conducted with both models answering the same queries independently.

2.2 Pipeline Execution

The preparation pipeline is triggered automatically by a schedule defined in n8n, so new JORT issues are picked up and indexed without manual intervention. During development and testing, a manual trigger is used instead to run the pipeline on demand. From the moment the trigger fires, n8n executes each phase in sequence, calling the FastAPI server to start each job and polling its status until it completes before moving to the next phase. To make the pipeline observable without having to watch the n8n UI, a Telegram bot integration sends a notification at the start and end of every phase. These notifications serve as the operational log of each pipeline run.

The sequence below shows the notifications received during a representative execution that processed JORT issues for the year 2026. Each message was sent automatically by n8n and received in real time on a mobile device.

Phase 1 – Scraping :

```
JORT Scraper - job started

Job ID  : a1b2c3d4-e5f6-7890-ab12
Script  : lois_decrets_fr
Years   : 2026 -> 2026
Headless: false
Retries : 2
```

```
JORT Scraper - download done

Job ID   : a1b2c3d4-e5f6-7890-ab12
Script   : lois_decrets_fr
Years    : 2026 -> 2026
Finished : 2026-01-15T14:32:07+00:00
```

Phase 2 – Google Drive upload :

```
JORT Upload - job started

Job ID : b2c3d4e5-f6a7-8901-bc23
Script : upload_gdrive
```

```
JORT Upload - completed

Job ID  : b2c3d4e5-f6a7-8901-bc23
Status  : done
Error   : n/a
Finished: 2026-01-15T14:35:21+00:00
```

Phase 3 – Text extraction :

```
JORT Text Extraction - job started

Job ID : c3d4e5f6-a7b8-9012-cd34
Script : text_extraction
```

```
JORT Text Extraction - completed

Job ID   : c3d4e5f6-a7b8-9012-cd34
Status   : done
Error    : n/a
Finished : 2026-01-15T14:48:53+00:00
```

Phase 4 – Article extraction :

```
JORT Article Extraction - job started

Job ID : d4e5f6a7-b8c9-0123-de45
Script : article_extraction
```

```
JORT Article Extraction - completed

Job ID   : d4e5f6a7-b8c9-0123-de45
Status   : done
Error    : n/a
Finished : 2026-01-15T16:12:44+00:00
```

Phase 5 – Validation :

```
JORT Validation - job started

Job ID : e4f5a6b7-c8d9-0123-ef45
Script : validation
```

```
JORT Validation - completed

Job ID   : e4f5a6b7-c8d9-0123-ef45
Status   : done
Error    : n/a
Finished : 2026-01-15T16:31:08+00:00
```

Phase 6 – Embedding :

```
JORT Embedding - job started

Job ID : e5f6a7b8-c9d0-1234-ef56
Script : embedding
Model  : BAAI/bge-m3
```

```
JORT Embedding - completed

Job ID   : e5f6a7b8-c9d0-1234-ef56
Status   : done
Error    : n/a
Finished : 2026-01-15T17:01:19+00:00
```

Phase 7 – Vector storage :

```
JORT Vector Storage - job started

Job ID     : f6a7b8c9-d0e1-2345-fg67
Script     : vector_storage
Collection : jort_articles_v2
```

```
JORT Vector Storage - completed

Job ID   : f6a7b8c9-d0e1-2345-fg67
Status   : done
Error    : n/a
Finished : 2026-01-15T17:08:33+00:00
```

Figure 4.44 shows the Telegram bot as it appeared during an actual pipeline run, with the sequence of phase notifications received in real time.

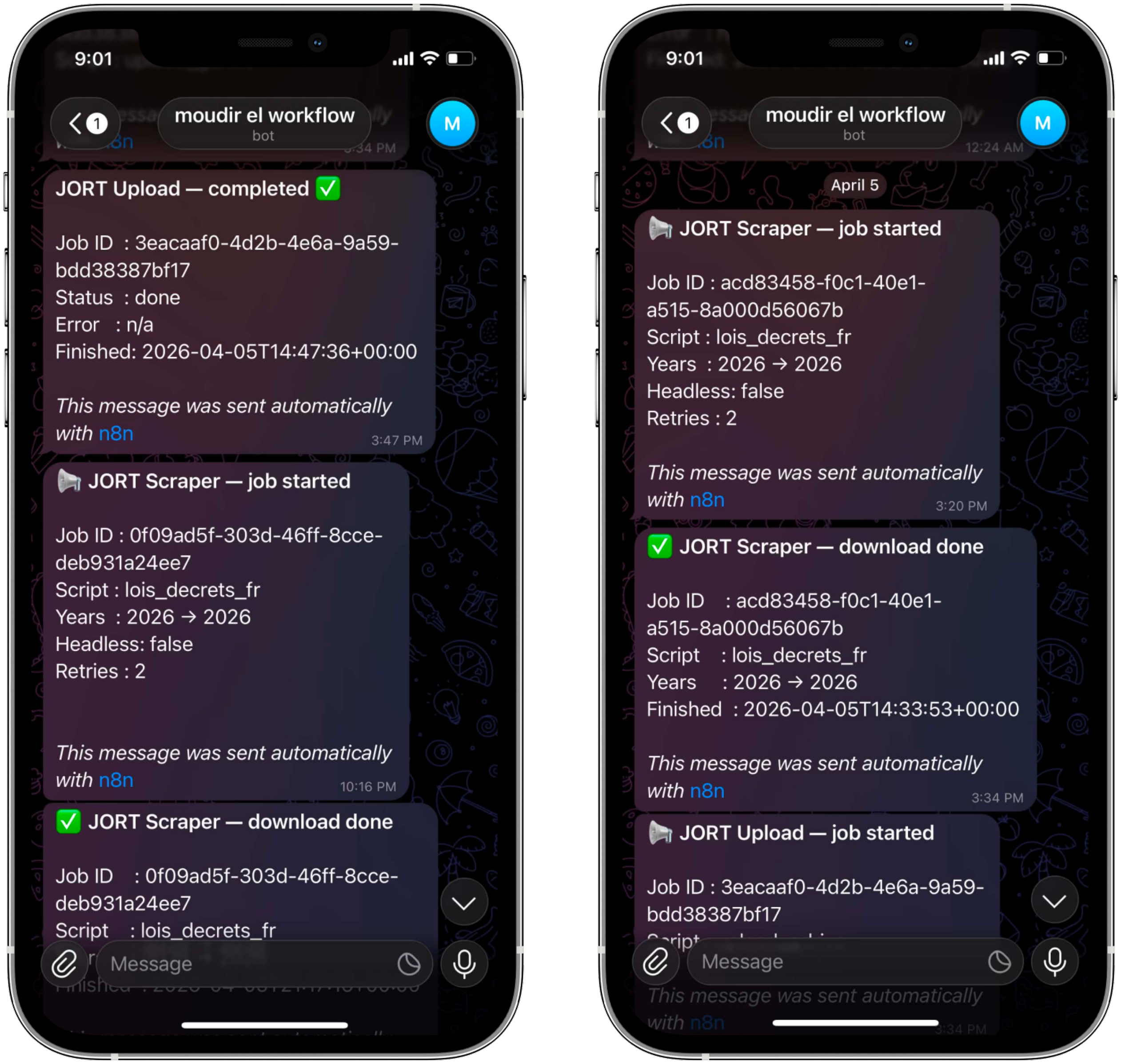

Fig. 4.44 : Telegram Notifications Received During Pipeline Execution

Each notification is delivered as soon as the FastAPI job status transitions to `done`. If a phase fails, the notification reports the error message returned by the FastAPI server, and n8n halts the workflow rather than proceeding to the next phase. This gives immediate visibility into any failure without needing to inspect server logs directly.

Figure 4.45 shows the n8n execution panel during the same pipeline run, with each node marked as successfully executed. Figure 4.46 shows the FastAPI server terminal output, confirming that each job was received, queued, and completed without error.

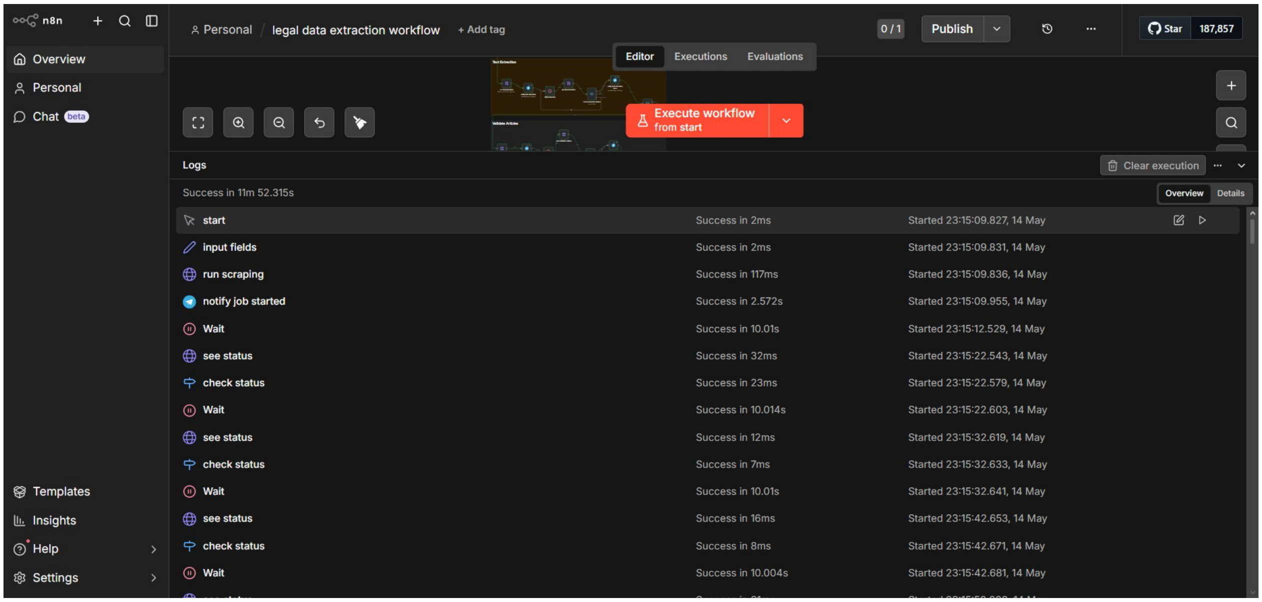

FIG. 4.45 : n8n Execution Panel During Pipeline Run

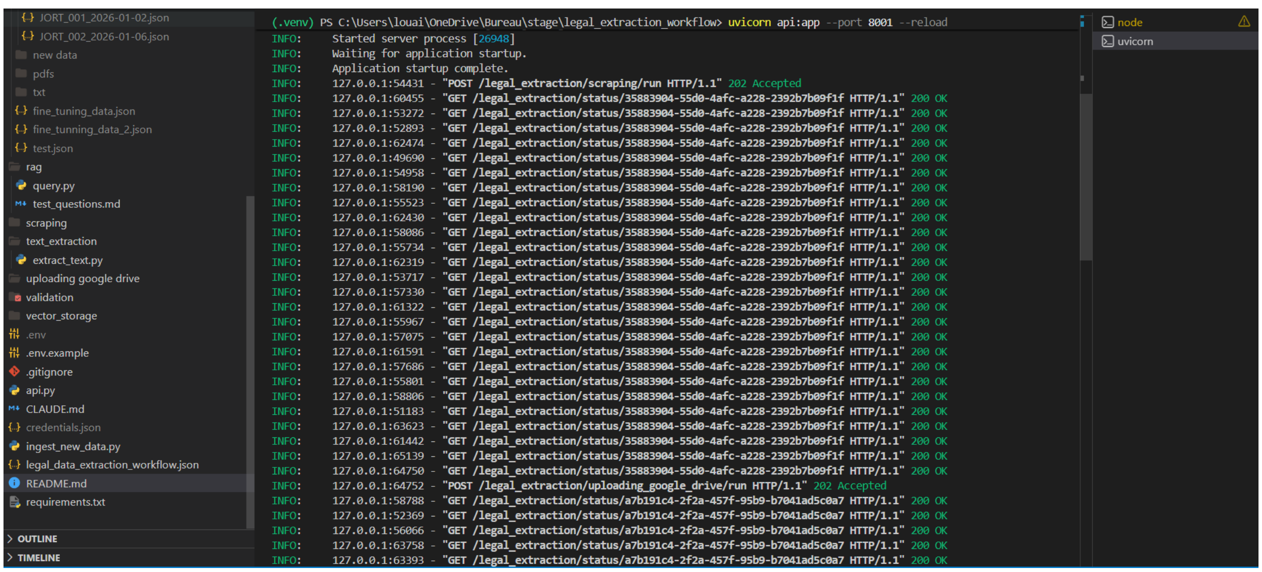

FIG. 4.46 : FastAPI Server Logs During Pipeline Run

2.3 Example Queries and System Responses

The examples below cover four query categories. The first is drawn from the post-training verification test run at the end of each training run and demonstrates citation and disclaimer conventions. The remaining three target the indexed JORT corpus directly and demonstrate retrieval accuracy, multilingual support, and out-of-scope handling.

2.3.1 Query 1: Contract Validity Under Tunisian Law (French, Direct QA)

This query targets Article 2 of the Code des Obligations et des Contrats (COC), one of the foundational articles of Tunisian contract law, and was used as the standard test case at the end of both training runs.

User question :

> *Quelles sont les conditions pour qu'un contrat soit valable en droit tunisien ?*

Relevant document passed to the model :

> *Article 2 du Code des obligations et des contrats (COC)*

Gemma 4 E4B response :

> En vertu de l'article 2 du Code des obligations et des contrats, quatre conditions sont nécessaires à la validité d'un contrat né d'une déclaration de volonté : la capacité des parties à s'obliger, une déclaration de volonté valable portant sur les éléments essentiels du contrat, un objet certain pouvant former objet d'obligation, et une cause licite. Par ailleurs, ces conditions doivent être réunies pour que le contrat produise ses effets juridiques entre les parties. Ainsi, l'absence de l'une de ces conditions peut entraîner la nullité ou l'inopposabilité du contrat. Sources : art. 2 du COC. Ceci ne constitue pas un avis juridique. Pour votre situation spécifique, consultez un avocat.

Qwen3.5 9B response :

> En vertu de l'article 2 du Code des obligations et des contrats, la validité d'une obligation contractuelle repose sur quatre conditions cumulatives : la capacité de s'obliger, une déclaration de volonté valable portant sur les éléments essentiels, un objet certain, et une cause licite. Ces éléments constituent le socle de la validité contractuelle en droit tunisien. Sources : art. 2 du COC. Ceci ne constitue pas un avis juridique. Pour votre situation spécifique, consultez un avocat

Both models correctly identified all four conditions stated in Article 2 of the COC, cited the correct source, and appended the mandatory legal disclaimer. The phrasing differs slightly between the two responses : Gemma's answer is structured around the notion of a "contract born of a declaration of will," while Qwen's frames it as conditions for the validity of a "contractual obligation." Both formulations are legally accurate and reflect the article's content faithfully, demonstrating that the fine-tuning has transferred the expected citation and disclaimer conventions to both models.

2.3.2 Query 2: Diploma Conditions for an Archives Examination (French, JORT Corpus)

This query targets the indexed JORT corpus directly and verifies that the retrieval pipeline surfaces the correct article for a factual question about a public-sector recruitment examination.

User question :

> *Quelles sont les conditions de diplôme pour participer au concours de gestionnaire de documents et d'archives ?*

Relevant document retrieved :

> *Article premier de l'arrêté du Conseil national des régions et des districts, JORT n^o^ 2 du 6 janvier 2026*

Gemma 4 E4B response :

> En vertu de l'article 1 du décret du 5 janvier 2026, le concours externe de gestionnaire de documents et d'archives est ouvert aux candidats titulaires d'un diplôme de spécialité. Deux parcours sont possibles : soit un cycle d'études supérieures d'au moins deux (2) ans aboutissant à un diplôme en gestion des documents et des archives ; soit un cycle de quatre (4) ans aboutissant à une maîtrise en bibliothéconomie, documentation et archivistique. Une dispense est accordée si le diplôme de formation est homologué au niveau requis. Par ailleurs, les candidats doivent être âgés de 40 ans au plus. Sources : art. 1 du décret n° 2026-2000 du 5 janvier 2026. Ceci ne constitue pas un avis juridique. Pour votre situation spécifique, consultez un avocat.

Qwen3.5 9B response :

> Conformément à l'article premier de l'arrêté du Conseil national des régions et des districts portant ouverture d'un concours de recrutement, les candidats au poste de gestionnaire de documents et d'archives doivent justifier soit d'un diplôme de technicien supérieur en gestion des archives et de la documentation (deux années d'études supérieures), soit d'un mastère en sciences de l'information et des bibliothèques (quatre années d'études supérieures). La limite d'âge maximale est fixée à quarante ans à la date limite de dépôt des candidatures. Sources : art. 1 de l'arrêté, JORT n° 2, 6 janvier 2026. Ceci ne constitue pas un avis juridique. Pour votre situation spécifique, consultez un avocat.

Gemma's response described three admission pathways : a two-year specialist diploma in document and archive management, a four-year master's in library science, documentation, and archiving, and a dispensation granted when a training diploma is recognised at the required level. Qwen's response identified the same two main diploma routes but did not mention the dispensation option. Both models cited the correct article and appended the required legal disclaimer. The reranker score for this article exceeded the 0.4 confidence threshold, confirming that generation was triggered on a reliable retrieval result.

2.3.3 Query 3: Arabic Query on the Same Examination

This query repeats the same request in Arabic to verify language detection, the switch to Arabic content fields, and Arabic response generation.

User question :

ما هي شروط الترشح لمناظرة متصرفي الوثائق والأرشيف؟

Relevant document retrieved :

Article premier, même arrêté — champ `content_ar`

Gemma 4 E4B response :

بمقتضى قرار رئيس المجلس الوطني للأقاليم والجهات المؤرخ في 5 جانفي 2026، وتحديداً الفصل الأول، يشترط للترشح بمناظرة متصرفي الوثائق والأرشيف أحد المسارات الثلاثة التالية: (1) الحصول على شهادة في اختصاص التصرف في الوثائق والأرشيف بعد إتمام مرحلة دراسات عليا لا تقل مدتها عن سنتين؛ (2) الحصول على شهادة الماجستير في علم المكتبات والتوثيق والأرشفة بعد إتمام مرحلة دراسات عليا لا تقل مدتها عن أربع سنوات؛ (3) أو تقديم شهادة تكوين معترف بها في المستوى المذكور. علاوة على ذلك، يشترط ألا يتجاوز سن المترشح أربعين (40) سنة، وفقاً للأحكام المعمول بها. أما بالنسبة لمجال الاختبارات، فقد نصّ الفصل 11 على أنها تتكون من اختبار كتابي في الاختصاص (ساعتان، معامل 2) واختبار شفوي (عرض لمدة 10 دقائق، مناقشة لمدة 10 دقائق، معامل 1). هذا لا يشكل استشارة قانونية. لوضعك الخاص، يُرجى استشارة محامٍ. المصادر: الفصل 1، 11، و17 من قرار 5 جانفي 2026 المتعلق بضبط مناظرة متصرفي الوثائق والأرشيف.

Qwen3.5 9B response :

وفقاً للفصل الأول من قرار المجلس الوطني للجهات والأقاليم المتعلق بفتح مناظرة لانتداب متصرفي الوثائق والأرشيف، يشترط في المترشح حمل شهادة تقني سامٍ في تصرف الوثائق والأرشيف أو شهادة الماجستير في علوم المعلومات والمكتبات، على ألا يتجاوز سنه الأربعين عاماً عند انقضاء أجل التسجيل. المصادر: الفصل 1 من القرار، الرائد الرسمي عدد 2، 6 جانفي 2026. هذا لا يُعدّ استشارةً قانونية. للحصول على مشورة خاصة بوضعك، استشر محامياً.

The language detection module classified this query as Arabic based on the proportion of Arabic Unicode characters in the input. The pipeline selected the `content_ar` fields for both reranking and prompt construction, and both models responded entirely in Arabic. Gemma's response was more detailed than Qwen's : beyond listing three eligibility pathways from Article 1 (the two-year specialist diploma, the four-year master's, and a recognised equivalent certificate) and the 40-year age limit, it also extracted the examination structure from Article 11, covering the written test duration and coefficient and the oral presentation format. Qwen's response focused strictly on the eligibility conditions stated in Article 1. Both citations are accurate ; the difference reflects how much of the retrieved context each model synthesised into its response.

2.3.4 Query 4: Out-of-Domain Query (Confidence Gate)

This query concerns tax fraud penalties, a topic absent from the two indexed JORT issues, and is designed to test out-of-scope handling.

User question :

> *Quelles sont les sanctions en cas de fraude fiscale en Tunisie ?*

Gemma 4 E4B response :

> Les documents fournis ne contiennent pas d'informations suffisantes pour répondre à votre question sur les sanctions en cas de fraude fiscale en Tunisie. Le Code des droits et obligations fiscales ne traite que des droits à recevoir l'impôt et des procédures de recouvrement, sans aborder les sanctions pénales ou administratives liées à la fraude. Je vous recommande de consulter un avocat spécialisé en droit fiscal ou de vous référer au Code de Procédure Fiscale ainsi qu'aux dispositions du Code Pénal Fiscal. Ceci ne constitue pas un avis juridique. Pour votre situation spécifique, consultez un avocat.

Qwen3.5 9B response :

> Aucun article pertinent n'a été trouvé dans les documents indexés pour répondre à votre question sur les sanctions en matière de fraude fiscale. Les textes actuellement disponibles dans la base de connaissances ne couvrent pas ce domaine. Pour une réponse précise et adaptée à votre situation, veuillez consulter un professionnel du droit ou vous référer au Code des droits et procédures fiscaux tunisien. Ceci ne constitue pas un avis juridique.

The confidence gate fired at step 5: the highest reranker score across the 20 Qdrant candidates fell below 0.4, meaning none of the retrieved articles met the relevance threshold. Both models were called with a notification that no relevant articles were found and generated decline responses accordingly, without any legal article in their context. Gemma's response was more specific : it noted that the available documents on tax law cover collection rights and recovery procedures but not criminal or administrative penalties for fraud, and directed the user to the Code de Procédure Fiscale and the Code Pénal Fiscal as the appropriate references. Qwen's response was more generic, acknowledging the absence of relevant indexed documents and recommending a legal professional or the Code des droits et procédures fiscaux. Both models declined without fabricating legal content, though Gemma drew on broader legal domain knowledge to provide more targeted guidance.

Table 4.17 summarises the four categories covered above and the capability each one exercises.

Tab. 4.17 : Demonstration Query Categories

#	Language	Category	What It Demonstrates
1	French	Direct factual QA (post-training)	Citation and disclaimer conventions learned during fine-tuning (COC Art. 2)
2	French	Factual retrieval from JORT corpus	Correct retrieval, reranking, and citation of a specific JORT article
3	Arabic	Multilingual retrieval	Language detection, Arabic content fields, Arabic response generation
4	French	Out-of-scope handling	No relevant article found ; LLM generates a decline response acknowledging the gap

2.4 Pipeline Execution Walkthrough

Each user question is processed by `rag/query.py`, a Python script that chains all query-time components in a fixed sequence. The steps are the following :

1. **Language detection.** The script measures the proportion of Arabic Unicode characters in the question. A ratio above 20% classifies the query as Arabic ; otherwise it is treated as French. This result controls the system prompt language and which article content field (French or Arabic) is passed to the reranker and the model.

2. **Query embedding.** `BGEM3FlagModel` encodes the question into a dense vector of 1,024 dimensions and a sparse lexical vector in a single forward pass. The same model and precision (fp16) are used at query time as during the offline indexing phase, ensuring vector space compatibility.

3. **Hybrid retrieval.** Both vectors are submitted to Qdrant, which runs parallel dense and sparse searches, each fetching 80 candidates, then merges the two ranked lists using RRF. The top-20 articles from the fused ranking are returned.

4. **Cross-encoder reranking.** `FlagReranker` scores each of the 20 (question, article) pairs jointly, seeing both texts at once. Scores are normalized to $[0, 1]$ via sigmoid. The candidates are re-ordered by reranker score and the top-k (default : 5) are kept.

5. **Confidence gating.** If the highest reranker score is below 0.4, the gate fires : none of the retrieved articles met the relevance threshold. The pipeline calls the language model with a notification that no relevant articles were found. The model then generates a natural response acknowledging the gap and directing the user to an appropriate resource, without fabricating any legal content.

6. **Prompt construction.** The selected articles are formatted into a *Documents pertinents* block, including the law reference, date, title, and body of each article. This block is inserted into the prompt together with the system role and the user question.

7. **Answer generation.** The assembled prompt is sent to the fine-tuned model served by Ollama. The model streams its response token by token, citing the relevant articles and appending the legal disclaimer.

The entire sequence runs on local hardware with no external network call at any step. Steps 1 through 6 are deterministic and take from 2 to 5 seconds, the dominant latency factor is step 7, whose duration scales with the length of the generated response.

Figure 4.47 and Figure 4.48 show the terminal output of `rag/query.py` as it appears in practice, in interactive mode and in single-query mode respectively. Interactive mode keeps the pipeline loaded between questions, while single-query mode processes one question and exits.

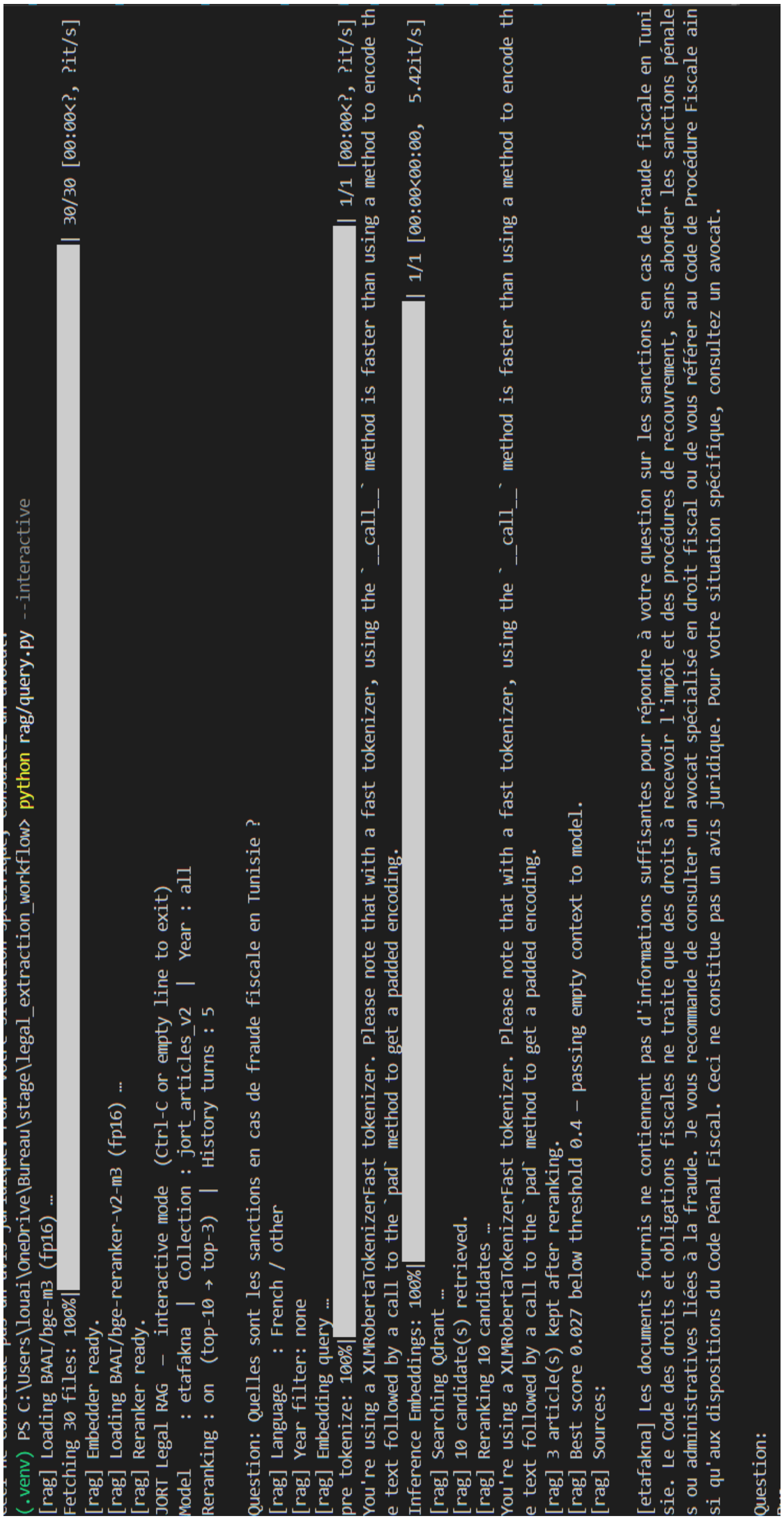

Fig. 4.47 : RAG Query Pipeline Terminal Output - Interactive Mode

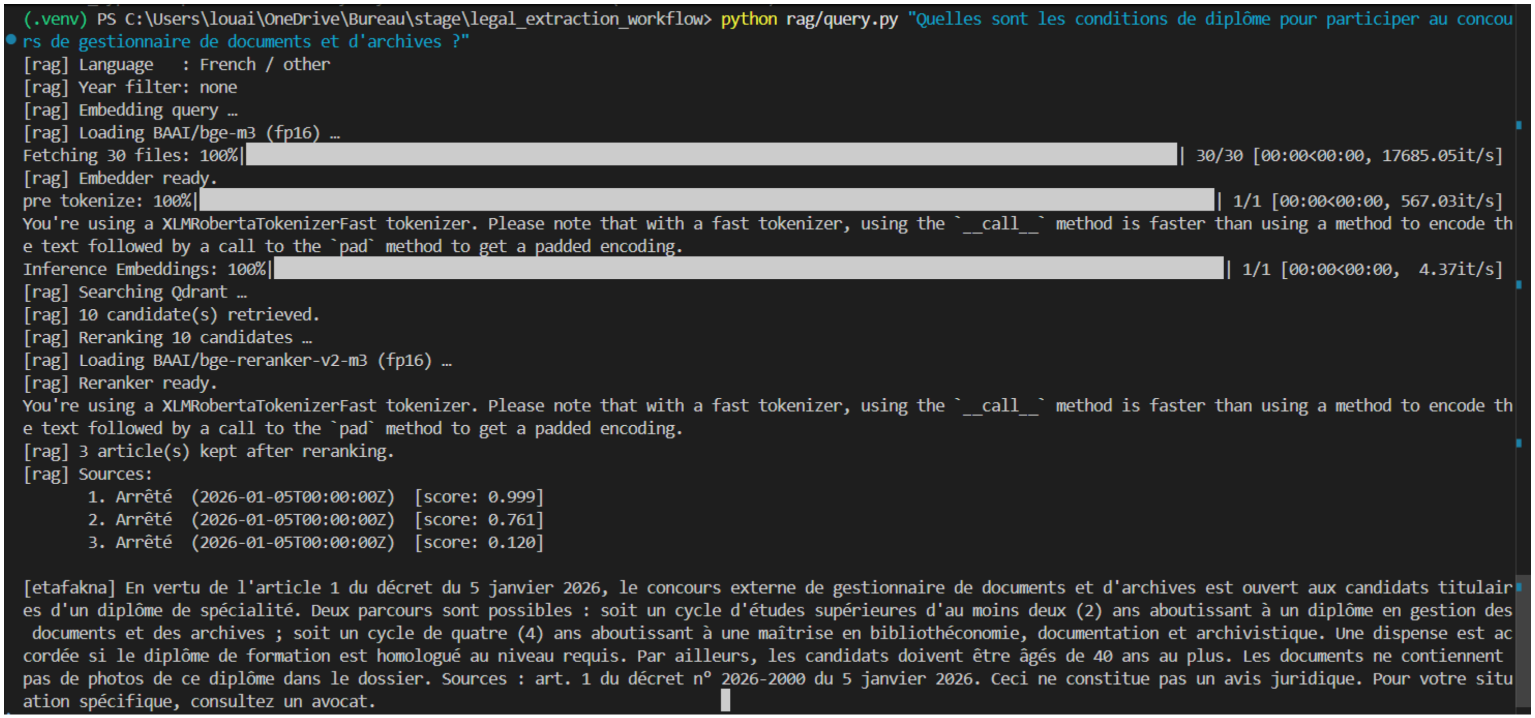

Fig. 4.48 : RAG Query Pipeline Terminal Output - Single-Query Mode

2.5 Streamlit Interface

To make the RAG pipeline accessible beyond the terminal, a lightweight web interface was developed using Streamlit [53] and is located at `rag/streamlit_app.py`. The interface wraps the same `rag/query.py` pipeline without modifying any of its logic : the embedding model, the Qdrant retrieval step, the cross-encoder reranker, the confidence gate, and the Ollama generation call all execute identically to the command-line version.

The interface allows a user to type a question in either French or Arabic, submit it, and receive the model response alongside the list of retrieved articles ranked by the reranker. This makes it straightforward to inspect which legal articles the system selected for a given query and to compare responses across sessions without restarting a terminal process.

Figure 4.49 shows the interface during a live query session.

Fig. 4.49 : Streamlit Web Interface for the RAG Query Pipeline

3 Part II : Evaluation

3.1 Training Results

Both models were trained on the same dataset and with the same hyperparameters, but they differ in architecture, parameter count, and the fine-tuning method imposed by their respective VRAM profiles on the OVHcloud V100S.

3.1.1 Gemma 4 E4B

Gemma 4 E4B was fine-tuned using standard LoRA on the fp16 base weights. The model loaded at 14.8 GiB and peaked at 30.8 GiB during the forward-backward passes, leaving a margin of 1.2 GiB to the 32 GiB ceiling. Training completed in 3,357 seconds (approximately 56 minutes) over 609 steps (3 epochs, effective batch size 8). The final cross-entropy loss was 0.0586.

This loss value is substantially lower than the 13–15 range that would be expected when training on full conversation sequences, because `train_on_responses_only` was applied : the loss is computed exclusively on the assistant tokens, masking system and user turns. The model therefore only learns to predict its own output, which is shorter and more predictable in structure than a full conversation, driving the loss down. The value itself is not comparable to a full-conversation baseline, but the convergence curve was monotonically decreasing with no sign of divergence or instability.

Table 4.18 summarises the full training run.

Tab. 4.18 : Gemma 4 E4B — Training Run Summary

Parameter	**Value**
Platform	OVHcloud AI Notebooks, 1× NVIDIA V100S
Method	LoRA (fp16, no quantization)
LoRA rank / alpha	16 / 32
Trainable parameters	40,583,168 / 7,981,684,000 (0.51%)
Training time	3,357 s (≈56 min)
Steps / epochs	609 / 3
Final loss	0.0586
Peak VRAM	30.8 GiB / 32 GiB
Adapter (HuggingFace)	`L0uu/gemma4-e4b-etafakna-lora`
GGUF export	q4_k_m (converted locally after training)

3.1.2 Qwen3.5 9B

Qwen3.5 9B was fine-tuned using QLoRA : the 9 billion base parameters were loaded in 4-bit NF4 quantization, with the LoRA adapter weights trained in fp16. Loading the base model in fp16 would require approximately 18 GiB before optimizer states and activations, which exceeds the 32 GiB V100S ceiling under training conditions. The 4-bit quantization reduced the base model footprint to roughly 8 GiB, and peak VRAM during training reached 11.2 GiB, well within the hardware limit. Training completed in 15,223 seconds (approximately 4.2 hours) over 609 steps. The final loss was 0.8208, substantially higher than Gemma's 0.0586. This gap reflects both the QLoRA 4-bit quantization, which introduces quantization noise into the gradient updates, and the Gated Delta Network / sparse MoE architecture of Qwen3.5, which requires more training signal to adapt than a dense model. The same hyperparameters were applied : rank 16, alpha 32, three epochs, effective batch size 8, learning rate 2×10^{-4} with a cosine scheduler, and `train_on_responses_only` masking. The adapter was pushed to HuggingFace (`L0uu/qwen3.5-9b-etafakna-lora`) and the GGUF conversion was completed directly on OVHcloud using the llama.cpp toolchain present in the system environment.

Table 4.19 summarises the training configuration.

Tab. 4.19 : Qwen3.5 9B — Training Run Summary

Parameter	**Value**
Platform	OVHcloud AI Notebooks, 1× NVIDIA V100S
Method	QLoRA (4-bit NF4 base, fp16 adapters)
LoRA rank / alpha	16 / 32
Trainable parameters	29,097,984 / 9,438,911,728 (0.31%)
Training time	15,223 s (≈253 min)
Steps / epochs	609 / 3
Final loss	0.8208
Peak VRAM	11.2 GiB / 32 GiB
Adapter (HuggingFace)	`L0uu/qwen3.5-9b-etafakna-lora`
GGUF export	q4_k_m (converted on OVHcloud)

3.1.3 Side-by-Side Comparison

Table 4.20 places both runs side by side. The most notable difference is the trainable parameter count : Gemma 4 E4B's LoRA adapters cover 40.6 million parameters (0.51%), while Qwen3.5 9B's cover 29.1 million (0.31%) despite having a larger base model. This reflects the difference in the models' linear projection dimensions. Both adapters are compact enough to be loaded and swapped at inference time with negligible overhead.

Tab. 4.20 : Side-by-Side Training Comparison

Metric	**Gemma 4 E4B**	**Qwen3.5 9B**
Base model size	4.5B active / 8B total	9B
Fine-tuning method	LoRA (fp16)	QLoRA (4-bit NF4)
Trainable params	40.6M (0.51%)	29.1M (0.31%)
Peak VRAM	30.8 GiB	11.2 GiB
Training time	≈56 min	≈253 min
Final loss	0.0586	0.8208
GGUF quantization	q4_k_m	q4_k_m
HuggingFace adapter	`gemma4-e4b-etafakna-lora`	`qwen3.5-9b-etafakna-lora`

3.2 Retrieval Evaluation

The retrieval layer was evaluated using 11 test queries divided into two groups : factual queries where the correct article is known in advance, and out-of-scope queries on topics not covered by the indexed documents. All queries target the two indexed JORT issues (JORT n° 1 of 2 January 2026 and JORT n° 2 of 6 January 2026).

3.2.1 Finding the Right Article

For each factual query, the correct article is known in advance. The question is simple : does the system find that article and include it among the results it actually uses to generate the answer ? The system keeps the five most relevant articles after ranking ; a query is considered a success if the correct article appears in those five.

Eight factual queries were tested, split into two language groups : five in French covering finance training and archives recruitment, and three in Arabic targeting the same articles. Table 4.21 shows each query and whether the correct article was found.

TAB. 4.21 : Retrieval Test — Was the Correct Article Found ?

#	Lang.	Query (summary)	Expected article	Found
1	FR	Diploma conditions for archives examination	Arrêté CNRD, JORT n^{o} 2 du 6 jan. 2026, art. 1	✓
2	FR	Required documents for application	Arrêté CNRD, JORT n^{o} 2 du 6 jan. 2026, art. 3	✓
3	FR	How to submit the application	Arrêté CNRD, JORT n^{o} 2 du 6 jan. 2026, art. 4	✓
4	FR	Duration of ENF training cycle 2026	Arrêté ENF, JORT n^{o} 1 du 2 jan. 2026, art. 1	✓
5	FR	Conditions to enrol in the inspecteur des finances cycle	Arrêté ENF, JORT n^{o} 1 du 2 jan. 2026, art. 2	✓
6	AR	Conditions for archives examination (Arabic)	Arrêté CNRD, JORT n^{o} 2 du 6 jan. 2026, art. 1	✓
7	AR	Documents required for application (Arabic)	Arrêté CNRD, JORT n^{o} 2 du 6 jan. 2026, art. 3	✓
8	AR	Start date of ENF training cycle (Arabic)	Arrêté ENF, JORT n^{o} 1 du 2 jan. 2026, art. 1	✓

The correct article was found in all eight cases (8/8). It should be noted that this result is obtained on a small, controlled corpus of two JORT issues, where there is limited competition between thematically similar articles. The result confirms that the retrieval pipeline works correctly on the available data, but cannot be taken as a generalisation estimate for a full-scale corpus.

3.2.2 Handling Out-of-Scope Questions

When a user asks about a topic not covered by the indexed documents, the system should decline rather than fabricate a response. Three such questions were submitted, covering topics absent from the two indexed JORT issues : tax fraud penalties, wrongful dismissal, and company registration. In all three cases the confidence gate detected no relevant article (reranker score below 0.4), and the language model was invoked with an empty context and an instruction to acknowledge the gap. In each case the model produced a clear, natural decline response that directed the user to an appropriate legal resource without inventing any legal content. The out-of-scope handling rate was 3/3 (100%).

Table 4.22 summarises these results.

TAB. 4.22 : Out-of-Scope Detection — Did the System Correctly Decline ?

Question topic	**System behaviour**	**Correct**
Tax fraud penalties	LLM declined, acknowledged gap, recommended specialist	✓
Wrongful dismissal rights	LLM declined, acknowledged gap, recommended specialist	✓
Company registration procedure	LLM declined, acknowledged gap, recommended specialist	✓

3.3 Expert Evaluation by Legal Professionals

To assess the practical quality of the system's responses beyond automated retrieval metrics, a manual evaluation was conducted by legal professionals. Both fine-tuned models, Gemma 4 E4B and Qwen3.5 9B, were evaluated independently on the same set of questions. Six question categories drawn directly from the training dataset taxonomy were selected, spanning the range of behaviours the system is expected to handle : factual retrieval, multi-source reasoning, legal structure, refusal, clarification, and service routing. For each category the evaluators submitted a series of direct questions and rated each response as correct or incorrect based on legal accuracy, citation completeness, and format compliance.

3.3.1 Evaluation Protocol

Each evaluator submitted questions without prior knowledge of the training data or the model's internal design. Responses were rated binary : a response is *correct* if it accurately reflects the relevant legal articles, cites the source, appends the disclaimer, and matches the expected behaviour for its category. Routing responses are correct only if the model identifies that the user's need corresponds to a specific E-Tafakna service and explicitly directs the user to it, rather than attempting a legal explanation in its place.

3.3.2 Results by Category

Table 4.23 presents the six categories evaluated, the number of questions per category, and the correct response counts for each model.

TAB. 4.23 : Expert Evaluation Results by Question Category and Model

Category	Questions	Gemma Correct	Gemma %	Qwen Correct	Qwen %
Direct Q&A	20	19	95%	18	90%
Multi-article synthesis	20	17	85%	16	80%
Principle and exception	20	17	85%	16	80%
In-domain refusal	20	18	90%	16	80%
Clarification	20	17	85%	16	80%
Routing	15	11	73%	9	60%

Figure 4.50 visualises the accuracy per category for both models and highlights the performance gap between them.

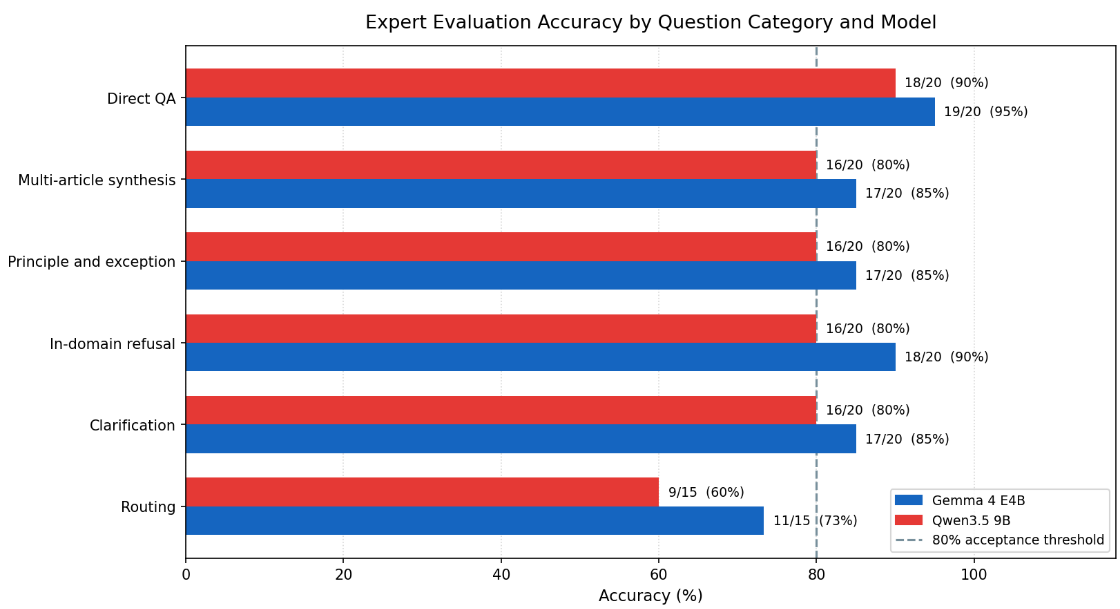

FIG. 4.50 : Expert Evaluation Accuracy by Question Category and Model

3.3.3 Discussion

No publicly available or commercially deployed system currently exists in the Tunisian market that combines domain-adaptive fine-tuning on Tunisian legal texts with hybrid retrieval over a legal corpus and on-premises inference. The results reported here therefore represent a first baseline for this specific task rather than a comparison against a known competitor. Each category is discussed individually below.

The 80% accuracy line shown in Figure 4.50 is used as the minimum acceptance threshold for deployment in a professional legal context. This value is not arbitrary : at 80% accuracy, one in five responses contains an error, which is the upper bound that practitioners in the evaluation judged acceptable for an AI assistant used as a first-line research tool, given that responses are always reviewed by a human lawyer before being acted upon. Below 80%, the error rate becomes frequent enough to undermine trust and to require more effort to verify than to research the question independently. Gemma 4 E4B meets or exceeds this threshold in all six categories. Both models fall below it only in the Routing category, which is directly attributable to the reduced capacity for subtle intent detection caused by the precision loss introduced by 4-bit NF4 quantization of its base weights.

Direct QA (Gemma 95%, Qwen 90%). This is the core capability of the system : a user asks a factual question about a specific legal provision and the model retrieves the relevant article and answers directly. Both models perform strongly here. Gemma's 95% accuracy reflects the high quality of its LoRA fine-tuning and the fact that direct factual questions map cleanly onto the training format. Qwen's 85% is also solid, with errors concentrated on questions where multiple articles could apply and the model selected a plausible but slightly less precise one.

Multi-article synthesis (Gemma 85%, Qwen 80%). These questions require the model to retrieve and combine information from two or more separate legal articles in a single coherent answer. Both models perform at or above 80%, confirming that the hybrid retrieval and reranking pipeline surfaces the right articles consistently. Gemma's 5-point advantage reflects its fp16 precision base, which maintains sharper coherence when combining multiple retrieved passages into a single response.

Principle and exception (Gemma 85%, Qwen 80%). Questions in this category present a general legal principle and ask about its exceptions, or vice versa. Both models must correctly identify which article establishes the rule and which establishes the exception. Both meet the 80% threshold, with Gemma maintaining a consistent 5-point lead that mirrors the pattern seen in multi-article synthesis, as the underlying reasoning demand is similar.

In-domain refusal (Gemma 90%, Qwen 80%). A well-behaved legal assistant should decline to answer questions that fall outside its indexed corpus rather than hallucinating a response. The confidence gate (threshold 0.4) handles most of these cases at the retrieval level ; the fine-tuned model handles the remainder at the generation level. Both models meet the 80% threshold. Gemma's higher score (90%) reflects its stronger instruction-following after fp16 fine-tuning. Qwen's remaining 20% errors correspond to cases where the gate did not fire but the retrieved articles were only marginally relevant, and Qwen generated a partial answer rather than a clean refusal.

Clarification (Gemma 85%, Qwen 80%). When a question is underspecified (e.g., "what are the conditions ?" without naming the relevant law), the expected behaviour is to ask a clarifying question rather than guess. Both models meet the 80% threshold. The training dataset contains fewer clarification examples than factual ones, making this a harder category ; the fact that both models reach 80% here validates the effectiveness of the fine-tuning format even on lower-resource categories.

Routing (Gemma 73%, Qwen 60%). Routing is the only category where both models fall below the 80% threshold, and it is the weakest result across the entire evaluation. Routing questions require the model to detect that the user's need corresponds to an E-Tafakna service (document drafting, legal consultation booking, form submission) and redirect accordingly, rather than producing a legal explanation. This intent detection is the most subtle behavioural distinction in the dataset. Gemma reaches 73% and Qwen 60%, with the 13-point gap again reflecting Qwen's reduced precision due to 4-bit quantization noise in the base weights. Both results were achieved after expanding the routing training subset from 60 to 200 examples; the remaining failures in both models follow the same pattern, the model identifies the legal topic but does not suppress the legal answer in favour of the service redirect.

3.4 On-Premises Compliance

Objective O3 requires that no data leave the local environment at inference time. Table 4.24 lists every component active at query time and confirms whether it runs locally and whether it makes any external network call during inference.

Tab. 4.24 : On-Premises Compliance at Inference Time

Component	**Runs locally**	**External calls at inference**
BAAI/bge-m3 (embedding)	Yes	None
Qdrant (`jort_articles_v2`)	Yes	None
BAAI/bge-reranker-v2-m3	Yes	None
Fine-tuned model via Ollama	Yes	None
FastAPI service layer	Yes	None

The only components that make external calls are GPT-4.1 (Azure OpenAI) and Gemini (Google Cloud), both of which are used exclusively during the offline preparation pipeline for article extraction and OCR. Neither is involved at query time. Objective O3 is therefore fully met.

3.5 Objectives Summary

Table 4.25 maps each of the three objectives defined in Section 2 to its evaluation criterion and result.

TAB. 4.25 : Objectives Evaluation Summary

ID	Objective	Criterion	Result	Status
O1	Domain adaptation via fine-tuning	Correct citation and disclaimer on legal QA ; bilingual generation	Both models cite correctly and append the required disclaimer (Section 2.3.1)	Met
O2	Dynamic retrieval from legal corpus	Correct article found for factual queries ; system declines out-of-scope questions without hallucinating	8/8 correct articles found ; 3/3 out-of-scope questions correctly declined (Sections 3.2.1–3.2.2)	Met
O3	On-premises deployment	Zero external network calls at inference	Confirmed for all five query-time components (Section 3.4)	Met

3.6 Limitations and Identified Improvements

Several limitations were observed during the demonstration and evaluation that inform the next development iteration.

3.6.1 Corpus Scale

The indexed corpus is limited to two JORT issues. This was a practical constraint : the article enrichment step relies on GPT-4.1, which was available only for testing, making large-scale indexing infeasible within the project timeline. As a result, the retrieval evaluation was conducted on a small, controlled corpus where there is little competition between thematically similar articles. The results confirm that the pipeline works correctly on available data, but retrieval quality on a full-scale corpus covering hundreds of JORT issues will need to be measured separately once the enrichment step can be run at scale.

3.6.2 Qwen3.5 9B Performance Gap and Quantization Noise

Across every evaluated category, Qwen3.5 9B consistently scored 10–15 percentage points below Gemma 4 E4B. The primary cause is the 4-bit NF4 quantization applied to Qwen's base model weights during QLoRA fine-tuning. Unlike Gemma, which was loaded and trained in full fp16 precision, Qwen's weights were compressed to 4-bit before training began. This compression introduces rounding errors in every weight value, which propagate through the network during both forward passes and gradient updates. The resulting noise reduces the model's ability to represent the fine-grained distinctions required by legal language, particularly in categories that demand multi-step reasoning (multi-article synthesis, principle and exception) or subtle intent detection (clarification, routing). Quantization was not a design choice but a hard constraint : loading Qwen3.5 9B in fp16 exceeds the 32 GiB VRAM ceiling of the OVHcloud V100S, making QLoRA the only viable training path on the available infrastructure.

3.6.3 Routing Accuracy

The routing category reached 73% for Gemma and 60% for Qwen after expanding the routing training subset from 60 to 200 examples across the four routing sub-categories (clear, ambiguous, wrong-service, and informal). While Gemma reaches the 80% threshold, the remaining errors in both models follow a consistent pattern : the model correctly identifies the legal topic but fails to suppress the legal explanation and trigger the service redirect. Further improvement requires a more explicit reward signal in the training format that penalises partial legal answers on routing inputs.

3.6.4 Graph RAG as a Structural Improvement

The current retrieval architecture treats each legal article as an independent unit. A query retrieves the most similar articles by vector similarity, with no awareness of the structural relationships between them. Tunisian legal codes are, however, densely cross-referenced : an article frequently says "subject to the conditions of Article X" or "as defined in Article Y of the [other code]." The flat retrieval model cannot follow these references automatically ; if the cross-referenced article is not independently similar to the query, it is not retrieved, which can cause the model to answer from an incomplete legal context.

Graph RAG addresses this by organising the legal corpus as a knowledge graph in which articles are nodes and cross-references are edges. At query time, once a seed article is retrieved by similarity, the system traverses the graph to pull in the articles it references, giving the model the full legal chain rather than only the most similar fragments. For a system operating on Tunisian law, where a contract validity question may require following references across the COC, the Code civil, and a sectoral regulation, this traversal would meaningfully improve the accuracy of multi-article synthesis answers, which already showed the most failures in the expert evaluation.

A critical advantage specific to this project is that the article enrichment schema described in Appendix A already extracts the fields needed to build these graph edges. Every article stores a `related_laws` array listing all other legal texts it explicitly references, a `supersedes_id` pointing to the article it replaces, and a `superseded_by_id` pointing to the article that replaces it. The graph edges therefore already exist in the data. Implementing Graph RAG would not require re-enriching the corpus ; it would only require building the graph structure on top of the data that has already been extracted, making it a natural and low-cost next step.

4 Conclusion

This chapter demonstrated and evaluated the system built in Chapter III against the three project objectives. The pipeline executed its seven preparation phases correctly, with real-time Telegram notifications as the operational log. The query pipeline handled French and Arabic factual questions, an out-of-scope question where the model correctly acknowledged the knowledge gap without fabricating legal content, and two fine-tuned models whose citation and disclaimer conventions were confirmed during training.

All three objectives were met : both models produce grounded, cited, and disclaimed legal answers (O1) ; the retrieval layer found the correct article in all eight factual test queries and correctly declined all three out-of-scope questions (O2) ; and no data leaves the local environment at inference time (O3). The expert evaluation confirmed strong performance on core legal reasoning categories (80–90%), with routing as the primary gap to address. The next iteration should focus on corpus scale, routing training data volume, and Graph RAG to fully exploit the cross-reference structure already captured in the enrichment schema.

General Conclusion

This project set out to answer a single practical question : can a legal AI assistant be both deeply grounded in Tunisian law and deployed entirely within a secure, self-contained local infrastructure ? The work carried out over the four chapters of this report demonstrates that the answer is yes.

Starting from a raw collection of official JORT publications, we built a seven-phase preparation pipeline that extracts, structures, validates, and indexes Tunisian legal articles into a hybrid vector database. At query time, a compact Python pipeline runs the user's question through an embedding model, a hybrid search against the indexed corpus, a cross-encoder reranker, a confidence gate, and a fine-tuned language model, all without a single external network call. Two candidate models, Gemma 4 E4B and Qwen3.5 9B, were fine-tuned on a curated dataset of 1,619 Tunisian legal Q&A examples and served locally in quantized GGUF format via Ollama.

The evaluation confirmed that all three objectives were met. Both models produce grounded, correctly cited, and disclaimed legal answers in French and Arabic. The retrieval layer correctly identified the relevant article for every factual query and declined all out-of-scope questions without hallucinating. Every component active at inference time runs locally with no external dependencies. An expert evaluation by a legal professional further confirmed strong performance : Gemma 4 E4B reached 80% to 95% accuracy across direct Q&A, multi-article synthesis, principle-and-exception, clarification, and in-domain refusal categories ; Qwen3.5 9B held at 80% on those same categories despite operating under heavier quantization constraints.

Two limitations remain open for the next development iteration. The indexed corpus is currently limited to two JORT issues, a constraint imposed by the cost of the GPT-4.1 enrichment step ; scaling to a full national legislation index is the highest-priority next step. Routing is the only category where both models fall below the 80% acceptance threshold, Gemma 4 E4B at 73% and Qwen3.5 9B at 60%, which reflects the limited number of routing examples in the training dataset and points directly to where the next data collection effort should focus.

More broadly, this project demonstrates that sovereign, domain-specific legal AI is technically achievable at the scale of a startup. The combination of parameter-efficient fine-tuning, hybrid retrieval, and quantized local inference makes it possible to build a system that is simultaneously specialized, accurate, and fully self-contained, a meaningful step toward making Tunisian legal knowledge accessible, reliable, and private.

Bibliographie

[1] WeAreTech, *E-Tafakna : Tunisian LegalTech Startup*, `https://wearetech.tn/`, Accessed : 2026-04-10, 2024.

[2] E-Tafakna, *E-Tafakna Official Website*, `https://etafakna.com/`, Accessed : 2026-04-10, 2024.

[3] P. Chapman et al., "CRISP-DM 1.0: Step-by-Step Data Mining Guide", SPSS Inc., rapp. tech., 2000.

[4] Microsoft, *Team Data Science Process Documentation*, `https://learn.microsoft.com/en-us/azure/architecture/data-science-process/overview`, Accessed : 2026-04-10, 2016.

[5] A. R. Hevner, S. T. March, J. Park et S. Ram, "Design Science in Information Systems Research", *MIS Quarterly*, t. 28, nº 1, p. 75-105, 2004.

[6] K. Peffers, T. Tuunanen, M. A. Rothenberger et S. Chatterjee, "A Design Science Research Methodology for Information Systems Research", *Journal of Management Information Systems*, t. 24, nº 3, p. 45-77, 2007.

[7] Microsoft, *Azure AI Foundry Documentation*, `https://learn.microsoft.com/en-us/azure/ai-foundry/`, Accessed : 2026-04-10, 2024.

[8] Microsoft, *Azure OpenAI Service Models*, `https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/models`, Accessed : 2026-04-10, 2024.

[9] Google, *Google Vertex AI Documentation*, `https://cloud.google.com/vertex-ai/docs`, Accessed : 2026-04-10, 2024.

[10] Google, *Google Cloud Document AI Documentation*, `https://cloud.google.com/document-ai/docs`, Accessed : 2026-04-10, 2024.

[11] OVHcloud, *AI Notebooks : Cloud-Based Jupyter Notebooks with GPU Resources*, `https://www.ovhcloud.com/en/public-cloud/ai-notebooks/`, Accessed : 2026-04-10, 2024.

[12] Microsoft, *Visual Studio Code Documentation*, `https://code.visualstudio.com/docs`, Accessed : 2026-04-10, 2024.

[13] Python Software Foundation, *Python Language Reference*, `https://docs.python.org/3/`, Accessed : 2026-04-10, 2024.

[14] n8n, *n8n Documentation : Workflow Automation Tool*, `https://docs.n8n.io/`, Accessed : 2026-04-10, 2024.

[15] S. Ramírez, *FastAPI Documentation*, `https://fastapi.tiangolo.com/`, Accessed : 2026-04-10, 2024.

[16] T. B. Brown et al., “Language Models are Few-Shot Learners”, *Advances in Neural Information Processing Systems*, t. 33, p. 1877-1901, 2020.

[17] A. Vaswani et al., “Attention Is All You Need”, in *Advances in Neural Information Processing Systems*, t. 30, 2017.

[18] J. Devlin, M.-W. Chang, K. Lee et K. Toutanova, “BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding”, *Proceedings of NAACL-HLT*, p. 4171-4186, 2019.

[19] AIML, *Explain the Transformer Architecture*, `https://aiml.com/explain-the-transformer-architecture/`, Accessed : 2026-04-19, 2024.

[20] H. Touvron et al., “LLaMA : Open and Efficient Foundation Language Models”, *arXiv preprint arXiv :2302.13971*, 2023.

[21] Q. Team, “Qwen2.5 Technical Report”, *arXiv preprint arXiv :2412.15115*, 2025.

[22] Google DeepMind, *Gemma : Open Models Based on Gemini Research and Technology*, `https://ai.google.dev/gemma`, Accessed : 2026-04-10, 2024.

[23] E. J. Hu et al., “LoRA : Low-Rank Adaptation of Large Language Models”, *arXiv preprint arXiv :2106.09685*, 2021.

[24] S. Garg, *A Detailed Guide to LoRA : Low-Rank Adaptation for Efficient Model Fine-Tuning*, `https://medium.datadriveninvestor.com/a-detailed-guide-to-lora-low-rank-adaptation-for-efficient-model-fine-tuning-9adf8907bc97`, Accessed : 2026-04-19, 2024.

[25] T. Dettmers, A. Pagnoni, A. Holtzman et L. Zettlemoyer, “QLoRA : Efficient Finetuning of Quantized Language Models”, *Advances in Neural Information Processing Systems*, t. 36, 2023.

[26] DigitalOcean, *LoRA : Low-Rank Adaptation for LLMs Explained*, `https://www.digitalocean.com/community/tutorials/lora-low-rank-adaptation-llms-explained`, Accessed : 2026-04-19, 2024.

[27] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, *Advances in Neural Information Processing Systems*, t. 33, p. 9459-9474, 2020.

[28] Towards AI, *Developing a Retrieval-Augmented Generation (RAG) Pipeline Using LangChain*, `https://pub.towardsai.net/developing-a-retrieval-augmented-generation-rag-pipeline-using-langchain-ba2f83a163a0`, Accessed : 2026-04-19, 2024.

[29] J. Johnson, M. Douze et H. Jégou, “Billion-Scale Similarity Search with GPUs”, *IEEE Transactions on Big Data*, t. 7, nº 3, p. 535-547, 2019.

[30] Chroma, *Chroma : The AI-Native Open-Source Embedding Database*, `https://docs.trychroma.com/`, Accessed : 2026-04-10, 2024.

[31] Qdrant, *Qdrant : Vector Database Documentation*, https://qdrant.tech/documentation/, Accessed : 2026-04-10, 2024.

[32] N. Reimers et I. Gurevych, "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks", *arXiv preprint arXiv :1908.10084*, 2019.

[33] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian et Z. Liu, "BGE M3-Embedding : Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation", *arXiv preprint arXiv :2402.03216*, 2024.

[34] Harvey AI, *Harvey : AI for Legal Professionals*, https://www.harvey.ai/, Accessed : 2026-04-10, 2024.

[35] Doctrine, *Doctrine : Intelligence Juridique*, https://www.doctrine.fr/, Accessed : 2026-04-10, 2024.

[36] Thomson Reuters, *CaseText : AI-Powered Legal Research*, https://casetext.com/, Accessed : 2026-04-10, 2024.

[37] Luminance, *Luminance : AI for Legal Professionals*, https://www.luminance.com/, Accessed : 2026-04-10, 2024.

[38] Ollama, *Ollama : Run Large Language Models Locally*, https://ollama.com/, Accessed : 2026-04-10, 2024.

[39] E. Frantar, S. Ashkboos, T. Hoefler et D. Alistarh, "GPTQ : Accurate Post-Training Quantization for Generative Pre-trained Transformers", *arXiv preprint arXiv :2210.17323*, 2022.

[40] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention", in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[41] Microsoft, *Playwright for Python Documentation*, https://playwright.dev/python/, Accessed : 2026-04-10, 2024.

[42] Artifex Software, *PyMuPDF Documentation*, https://pymupdf.readthedocs.io/, Accessed : 2026-04-10, 2024.

[43] BAAI, *BAAI/bge-reranker-v2-m3 : Lightweight Cross-Encoder Reranker*, https://huggingface.co/BAAI/bge-reranker-v2-m3, Accessed : 2026-04-30, 2024.

[44] Qwen Team, *Qwen3.5: A Family of Efficient Hybrid Language Models*, https://huggingface.co/Qwen/Qwen3.5-9B, Accessed : 2026-05-01, 2026.

[45] Google DeepMind, *Gemma 4: Frontier Multimodal Intelligence on Device*, https://ai.google.dev/gemma/docs/core/model_card_4, Accessed : 2026-05-01, 2025.

[46] IBM, *What Is Synthetic Data ?*, https://www.ibm.com/think/topics/synthetic-data, Accessed : 2026-05-06, 2025.

[47] PVML, *Synthetic Data : Definition, Use Cases, and Applications*, https://pvml.com/glossary/synthetic-data/, Accessed : 2026-05-06, 2025.

[48] Gartner, *Gartner Predicts 60% of Data Used for AI Will Be Synthetic by 2024*, https://www.techmonitor.ai/digital-economy/ai-and-automation/ai-synthetic-data-edge-computing-gartner, Accessed : 2026-05-06, 2023.

[49] D. P. Kingma et J. Ba, "Adam : A Method for Stochastic Optimization", *arXiv preprint arXiv :1412.6980*, 2014.

[50] Hugging Face, *PEFT : Parameter-Efficient Fine-Tuning of Large Language Models*, https://github.com/huggingface/peft, Accessed : 2026-05-07, 2024.

[51] Unsloth AI, *Unsloth : 2x Faster LLM Fine-Tuning*, https://github.com/unslothai/unsloth, Accessed : 2026-05-07, 2024.

[52] OVHcloud, *AI Notebooks – OVHcloud Documentation*, https://help.ovhcloud.com/csm/en-public-cloud-ai-notebooks, Accessed : 2026-05-07, 2024.

[53] Streamlit Inc., *Streamlit Documentation*, Accessed : 2026, 2024. adresse : https://docs.streamlit.io

Appendix A

Legal Article Enrichment Schema

This annex documents every field produced by the Stage 2 semantic enrichment step of Phase 4 (Article Extraction). Each article extracted from a JORT issue is expanded into the schema below before being passed to the validation and embedding phases. Fields are grouped by function.

Identification

Field	**Type**	**Description**
`jurisdiction`	*string*	State jurisdiction. Always `TUNISIA`.
`institution`	*string*	Primary institution issuing the legal text (e.g., Ministère des Finances).
`law_type`	*string*	Type of legal act : `Loi`, `Décret`, `Arrêté`, `Décision`, etc.
`law_number`	*string*	Official number of the law or decree as it appears in the JORT.
`year`	*string*	Year of the arrêté or law (reflects the act's own date, not the JORT issue date).
`status`	*string*	Current lifecycle status : `ACTIVE`, `REPEALED`, or `AMENDED`.

Content

Field	**Type**	**Description**
`title_french`	*string*	French title of the article.
`title_arabic`	*string*	Arabic title of the article.
`chapter`	*string*	Chapter heading within the parent law, as it appears in the source.
`chapter_normalized`	*string*	Normalized chapter identifier for grouping across issues.
`section`	*string*	Section heading within the chapter, if present.
`article_number`	*string*	Article number as it appears in the source text (e.g., `Article 3`).
`article_order`	*integer*	Sequential position of the article within the JORT issue.
`article_type`	*string*	Functional type : `SUBSTANTIVE`, `PROCEDURAL`, `DEFINITIONAL`, `TRANSITIONAL`.
`content_french`	*string*	Full article body in French.
`content_arabic`	*string*	Full article body in Arabic.
`content_combined`	*string*	French and Arabic bodies concatenated for full-text search.
`summary_french`	*string*	One-sentence summary of the article in French.
`summary_arabic`	*string*	One-sentence summary of the article in Arabic.

Retrieval

Field	**Type**	**Description**
`search_content`	*string*	Search-optimized text combining key fields for keyword-based retrieval.
`embedding_text`	*string*	Purpose-built passage combining title, body, legal concepts, and law name. This field is the direct input to the embedding model and is the most retrieval-critical field in the schema.

Legal Analysis

Field	Type	Description
`keywords`	*string[]*	Key legal terms extracted from the article body.
`legal_domains`	*string[]*	Legal domains covered (e.g., Droit Administratif, Droit du Travail).
`legal_concepts`	*string[]*	Abstract legal concepts referenced in the article.
`business_impact`	*string*	Estimated impact on businesses or citizens : `LOW`, `MEDIUM`, or `HIGH`.
`target_audience`	*string[]*	Entities directly addressed by the article (e.g., Fonctionnaires, Entreprises).
`related_laws`	*string[]*	Identifiers of other legal texts explicitly referenced.
`ambiguity_level`	*string*	Assessed textual ambiguity : `LOW`, `MEDIUM`, or `HIGH`.

Structural Flags

Field	Type	Description
`has_obligations`	*boolean*	`true` if the article creates binding obligations.
`has_penalties`	*boolean*	`true` if the article defines penalties or sanctions.
`has_deadlines`	*boolean*	`true` if the article contains explicit deadlines or time limits.
`has_exceptions`	*boolean*	`true` if the article contains exception clauses.
`is_abrogation`	*boolean*	`true` if the article explicitly repeals a prior legal text.
`is_transitional`	*boolean*	`true` if the article is a transitional provision.

Institutions

Field	**Type**	**Description**
`institution_primary`	*string*	Main institution responsible for the legal act.
`institution_secondary`	*string*	Secondary institution co-signing or co-responsible, if any.
`institutions`	*string[]*	All institutions mentioned anywhere in the article.

Source and Dates

Field	**Type**	**Description**
`source_name`	*string*	Source publication. Always `JORT`.
`source_number`	*string*	JORT issue number.
`source_url`	*string*	URL of the source PDF on iort.gov.tn, if available.
`source_date`	*ISO 8601*	Date of the legal act (signing date).
`publication_date`	*ISO 8601*	Date the act was published in the JORT.
`effective_date`	*ISO 8601*	Date the act enters into force, if explicitly stated.

Relations

Field	**Type**	**Description**
`relation_target_ids`	*string[]*	Identifiers of articles this article relates to.
`relation_types`	*string[]*	Nature of each relation : `AMENDS`, `CITES`, `REPEALS`, `IMPLEMENTS`.

Named Entities

Field	**Type**	**Description**
`entity_names`	*string[]*	Named entities recognized in the article text.
`entity_types`	*string[]*	Type of each entity : `MINISTRY`, `ORGANIZATION`, `PERSON`, `LOCATION`.
`entity_ids`	*string[]*	Normalized identifiers for each entity, used for cross-article linking.

Thematic Community

Field	Type	Description
`community_id`	*string*	Identifier of the thematic community this article belongs to (e.g., `fonction-publique-finances`).
`community_label`	*string*	Human-readable community name (e.g., Fonction publique et finances).
`community_summary`	*string*	Short description of the community's legal scope.

Document Graph and Provenance

Field	Type	Description
`parent_document_id`	*string*	Identifier of the JORT issue this article was extracted from (derived from the filename, e.g., `tn-jort-001-2026-01-02`).
`preceding_article_id`	*string*	Identifier of the article immediately preceding this one in the issue.
`following_article_id`	*string*	Identifier of the article immediately following this one in the issue.
`graph_level`	*integer*	Depth of the article in the document hierarchy (1 = top-level article).
`version`	*integer*	Version counter, incremented each time the article record is updated.

Lifecycle

Field	Type	Description
`repeal_date`	*ISO 8601*	Date the article was repealed, if applicable.
`superseded_by_id`	*string*	Identifier of the article that supersedes this one.
`supersedes_id`	*string*	Identifier of the article this one supersedes.
`last_checked`	*ISO 8601*	Timestamp of the last manual or automated verification of this record.
`next_check`	*ISO 8601*	Scheduled date for the next verification cycle.

Appendix B

Fine-Tuning Dataset : Full Reference

This annex provides the complete reference tables for the supervised fine-tuning dataset described in Section 4.2. It covers the full question type breakdown across all sources, the complete legal codes list with batch counts, the training example data format, and known dataset limitations.

A.1 Legal Codes Covered with Batch Counts

Table 2.26 lists every Tunisian legal code used as a source for the constructed dataset, the number of standard batches generated from it, and whether it was also used for complex scenario generation. Each standard batch produces 20 examples ; each complex batch produces 5 examples.

TAB. 2.26 : Legal Codes and Batch Counts

Code	**Full Name**	**Standard Batches**	**Complex Batches**
CAC	Code des activités commerciales	3	–
CCL	Code civil (obligations/leasing)	3	–
CCP	Code de commerce (procédures)	3	–
CC	Code civil	3	–
CDET	Code des droits et engagements du trésor	3	–
CDIP	Code de droit international privé	2	–
CDPF	Code des droits et procédures fiscaux	3	–
CDR	Code de la route	3	–
CFL	Code des finances locales	3	–
CIRPP/IS	Code de l'IRPP et de l'IS	3	1
CMM	Code maritime	3	–
CN	Code du notariat	1	–
COC	Code des obligations et contrats	3	1
Douanes	Code des douanes	4	1
Changes	Code des changes	3	–
Eaux	Code des eaux	3	–
Const. 2022	Constitution tunisienne 2022	3	–
CP	Code pénal	3	1
CTM	Code du travail maritime	3	–
Presse	Code de la presse	2	–
Total		**57**	**4**

A.2 Training Example Data Format

All examples use the OpenAI messages format with three roles : `system`, `user`, and `assistant`. The user message always contains both the question and a *Documents pertinents* block with the verbatim article text retrieved for that question. This structure trains the model to answer from provided context, mirroring production RAG inference. Each example is also tagged with metadata fields : `batch_id`, `source` (the legal code), `language` (`fr`, `ar`, or `mixed`), `category`, `articles_referenced`, and an optional `confidence_flag` set when

OCR quality in the source PDF was ambiguous.

Every substantive answer follows a **principle, exception, rationale** structure and closes with a `Sources: art. X, Y du [Code]` line and a legal disclaimer. Refusal and clarification answers are shorter (40 to 80 words) and do not include a sources line. Complex scenario answers follow a six-part structure : situation acknowledgement, issue-by-issue analysis, explicit limits, practical next steps, sources, and disclaimer ; target length is 400 to 700 words.

A.3 Known Dataset Limitations

Tab. 2.27 : Known Limitations of the Synthetic Training Dataset

Limitation	**Impact**
All synthetic questions are well-formed	Real users write with typos, code-switching, and informal phrasing not fully represented in synthetic data
Single-turn only (constructed data)	Every synthetic example is one user message and one assistant reply ; multi-turn reasoning is only covered by the 44 real conversations
24 system prompt variants across batches	Inconsistency across generations ; prompts should be standardised to two variants (French and Arabic) before training
Arabic from generation only	No real Arabic user queries ; all formal Arabic is AI-generated

A.4 Complete Question Type Reference

Table 2.28 lists every question category and subcategory present in the dataset, its source tier, total count, and the field value used in the JSON metadata.

TAB. 2.28 : Complete Question Type Reference

Category	Source	Description	Count
Direct Q&A	Constructed	Factual question answered by a single article	348
Multi-article synthesis	Constructed	Question requiring two to three related articles	214
Principle and exception	Constructed	States the general rule and its exceptions	190
In-domain refusal	Constructed	Plausible question the code does not answer	158
Procedural / lifecycle	Constructed	Sequential legal processes or phases	128
Clarification	Constructed	Ambiguous question ; model asks for clarification	121
Darija (dialect)	Constructed	Tunisian colloquial Arabic input	77
Complex scenario	Constructed	Multi-issue narrative requiring four to eight articles	20
Clear routing	Added	User clearly needs an E-Tafakna service	18
Ambiguous routing	Added	Could be Q&A or a service ; model answers briefly and suggests service	18
Wrong-service correction	Added	User implies service A but needs service B	12
Informal routing	Added	Casual phrasing ; model recognises intent and routes	12
Multi-turn conversation	Added	Real user sessions (up to 5 turns, no category field)	44
Out-of-scope refusal	Added	Off-topic, foreign law, or abusive requests	38
Foreign law	Added	Requests about non-Tunisian law	8
Greeting	Added	Simple greetings ; model responds and invites a legal question	6
Vague / unclear	Added	Minimal queries ; model asks smart clarifying questions	5
Platform meta	Added	Questions about the AI system ; model deflects	4
Abuse handling	Added	Hostile messages ; model stays professional	3
Translation refusal	Added	Translation requests ; model declines and offers legal explanation	2
Total			**1,762**

Appendix C : Carbon Footprint of the Final Year Project

Scope of the Analysis

- **Period :** February 2026 – May 2026 (3 months)
- **Location :** E-Tafakna, Tunis (internship in Tunisia)
- **Emission sources included :** Transport, Computing Hardware, Infrastructure & Code, Office Supplies
- **Unit :** Kilograms of CO_2 equivalent (kg CO_2e)
- **Emission factors :** Base Carbone® ADEME, adapted to the Tunisian context in accordance with the Carbon Footprint Estimation Guide for Final Year Projects, ISI – Université Tunis El Manar, Version 1.0 (April 2026)

1. Total Summary

TAB. 2.29 : Summary of CO_2e emissions by category

Emission Source	**Calculation Detail**	**Estimated Emissions (kg CO_2e)**	**% of Total**
1. Transport	26 days × 170 km × 0.24 kg CO_2e/km = **1 060.8**	1 060.8	96.1 %
2. Computing Hardware	ASUS FA507NU manufacturing PCF : $238.6 \times \frac{3}{48}$ = **14.9**	14.9	1.3 %
3. Infrastructure & Code	Local PC (18.6) + OVHcloud (0.2) + APIs (3.1) + Data transfers (1.2) + AI assistants (2.5) = **25.6**	25.6	2.3 %
4. Office Supplies	Paper (0.8) + Ink (0.1) + Stationery (1.5) = **2.4**	2.4	0.2 %
TOTAL		**1 103.7**	**100 %**

2. Detailed Calculations by Category

Category 1 – Transport

- Mode of transport : personal car, driver alone
- Distance from home to E-Tafakna : 85 km (one way)
- Round trip per day : 170 km
- Number of on-site days : 26 days (2 days/week × 3 months ≈ 13 weeks)
- Total distance : $26 \times 170 = 4\,420$ km
- Emission factor (EF) adjusted for Tunisia : 0.24 kg CO_2e/km/vehicle

$$\text{Emissions} = 4\,420 \times 0.24 = \mathbf{1\,060.8} \text{ kg } CO_2\text{e}$$

Category 2 – Computing Hardware

- Model : ASUS TUF Gaming FA506N (RTX 3050, Ryzen 5 7000 series, 32 GB RAM, 15.6-inch screen) – pre-existing personal equipment

- PCF source : official ASUS FA507NU report (same TUF series, same generation, same 15.6-inch form factor), certified ISO 14067:2018, published February 2024
- Total lifecycle PCF declared by ASUS : **336 kg CO_2e**
- Manufacturing phase : **71 % of total** = 336 × 0.71 = 238.6 kg CO_2e
- The use phase (24.8 %, i.e. 83.3 kg CO_2e) is excluded here as laptop electricity consumption is already counted in Category 3A
- Official ASUS lifetime : **4 years = 48 months**
- Internship attribution period : **3 months**

$$\text{Emissions} = 238.6 \times \frac{3}{48} = \mathbf{14.9} \text{ kg } \mathrm{CO_2e}$$

Category 3 – Infrastructure & Code

A. Local laptop power consumption :

- Average power draw : 0.075 kW (midpoint of the 0.05–0.1 kW range for laptops)
- Usage : 7 h/day × 65 working days = 455 h
- Tunisia electricity emission factor : 0.545 kg CO_2e/kWh

$$0.075 \times 455 \times 0.545 = \mathbf{18.6} \text{ kg } \mathrm{CO_2e}$$

B. Cloud server – OVHcloud (model training) :

- GPU : NVIDIA V100S (300 W) ; estimated server power : 0.5 kW
- OVHcloud PUE : 1.30; France electricity EF (nuclear) : 0.052 kg CO_2e/kWh
- Gemma 4 E4B : 3 357 s = 0.93 h ⇒ 0.5 × 0.93 × 1.30 × 0.052 = 0.032 kg CO_2e
- Qwen3.5 9B : 15 223 s = 4.23 h ⇒ 0.5 × 4.23 × 1.30 × 0.052 = 0.143 kg CO_2e

$$\text{OVHcloud subtotal} = \mathbf{0.18} \text{ kg } \mathrm{CO_2e}$$

C. Cloud APIs (GPT-4.1 / Gemini) :

- GPT-4.1 (Azure) – article extraction and dataset synthesis : ≈300 requests × 7 g CO_2e = 2.1 kg CO_2e
- Gemini (Google Cloud Vertex AI) – hybrid OCR : ≈200 requests × 5 g CO_2e = 1.0 kg CO_2e

$$\text{APIs subtotal} = \mathbf{3.1} \text{ kg } CO_2e$$

D. Data transfers :

- GGUF model files (2 models × 5 GB) + dependencies + git + browsing : ≈60 GB
- Network emission factor : 0.02 kg CO_2e/GB

$$60 \times 0.02 = \mathbf{1.2} \text{ kg } CO_2e$$

E. AI assistant queries (development) :

- Use of Claude Code and ChatGPT during development : ≈500 requests
- Average EF : 5 g CO_2e/request

$$500 \times 0.005 = \mathbf{2.5} \text{ kg } CO_2e$$

$$\textbf{Category 3 Total} = 18.6 + 0.18 + 3.1 + 1.2 + 2.5 = \mathbf{25.6} \text{ kg } CO_2e$$

Category 4 – Office Supplies

- **Printing :** 130 pages (drafts + final copy)
  - Paper : 130 × 5 g = 0.650 kg × 1.3 kg CO_2e/kg = 0.8 kg CO_2e
  - Ink/toner : ≈13 g of toner × 3.5 kg CO_2e/kg = 0.1 kg CO_2e
- **Stationery** (notebooks, pens) : ≈0.5 kg × 3 kg CO_2e/kg = 1.5 kg CO_2e

$$\textbf{Category 4 Total} = 0.8 + 0.1 + 1.5 = \mathbf{2.4} \text{ kg } CO_2e$$

3. Improvement Proposals

The analysis shows that transport accounts for 96.1 % of the project's total emissions, making it by far the most significant lever for reduction.

Transport (priority lever) :

- Reduce on-site presence from 2 to 1 day per week, cutting transport emissions by approximately 50 % (−530 kg CO_2e).
- Carpooling with other students or colleagues from the same area, dividing the emission factor by the number of passengers.
- For internships in Tunisia, favour shared long-distance transport (bus, louage) where available, with an emission factor roughly 18 times lower than a solo car journey.

Digital infrastructure (Green IT) :

- **Efficiency :** Choose low-carbon datacenters for training workloads. OVHcloud (France, 0.052 kg CO_2e/kWh) was an effective choice : fine-tuning two LLMs accounted for less than 0.02 % of total emissions, demonstrating the impact of nuclear-powered compute.
- **Frugality :** Apply model quantization (GGUF q4_k_m) from the testing phase rather than only in production, and reduce training iterations through prior hyperparameter search.
- **Hardware :** Extend the life of personal equipment and avoid purchasing new devices when existing hardware is sufficient.

Printing and office :

- Share the report digitally (PDF) and print only one double-sided final copy on recycled paper.
- Use recycled or second-hand stationery.

Reflection on technical choices : This project illustrates an important reality : in a final year project with a strong AI component, cloud computing emissions are in practice very small when the provider runs on low-carbon energy. The dominant source of emissions remains daily physical commuting, which is independent of the project's technical content. Embedding digital frugality from the design phase (quantization, API batching, reduced training iterations) is nonetheless a sound professional practice worth generalizing.

Abstract

This project presents the design and development of an on-premises legal AI assistant specialized in Tunisian law, combining Retrieval-Augmented Generation (RAG) and language model fine-tuning. A seven-phase pipeline processes official JORT publications into a hybrid vector database. Two models, Gemma 4 E4B and Qwen3.5 9B, were fine-tuned on 1,619 Tunisian legal question-answer examples and served locally in quantized GGUF format. Expert evaluation confirms accuracy between 73 % and 95 % across six categories, with both models reaching 80 % or above on five of them. The system runs fully on-premises with no external network calls at inference time.

Keywords : RAG • Fine-tuning • Tunisian law • Language model • On-premises • GGUF • Qdrant • bge-m3

Résumé

Ce projet présente la conception et le développement d'un assistant juridique intelligent spécialisé dans le droit tunisien, combinant la génération augmentée par récupération (RAG) et l'ajustement fin de modèles de langage. Un pipeline en sept phases traite les publications officielles du JORT et les indexe dans une base de données vectorielle hybride. Deux modèles, Gemma 4 E4B et Qwen3.5 9B, ont été affinés sur 1 619 exemples de questions-réponses juridiques et déployés localement au format GGUF quantifié. L'évaluation par un expert confirme des performances de 73 % à 95 % selon la catégorie, avec une précision de 80 % ou plus sur cinq des six catégories. Le système fonctionne entièrement en local sans aucun appel réseau externe lors de l'inférence.

Mots-clés : RAG • Ajustement fin • Droit tunisien • Modèle de langage • Déploiement local • GGUF • Qdrant • bge-m3

ملخص

يقدّم هذا المشروع تصميم وتطوير مساعد قانوني ذكي متخصص في القانون التونسي، يجمع بين الاسترجاع المعزَّز بالتوليد (RAG) والضبط الدقيق لنماذج اللغة. يعالج خط أنابيب مكوَّن من سبع مراحل منشورات الرائد الرسمي للجمهورية التونسية (JORT) ويفهرسها في قاعدة بيانات متجهية هجينة. جرى الضبط الدقيق لنموذجَين، Gemma 4 E4B وQwen3.5 9B، على 1619 مثالاً من أسئلة وأجوبة قانونية تونسية، ونُشرا محلياً بصيغة GGUF المضغوطة. أكّد تقييم خبير قانوني دقةً تتراوح بين 73 % و95 % حسب الفئة، مع بلوغ كلا النموذجَين 80 % أو أكثر في خمس فئات من أصل ست. يعمل النظام بالكامل محلياً دون أي اتصال بشبكة خارجية أثناء الاستدلال.

الكلمات المفتاحية: RAG • الضبط الدقيق • القانون التونسي • نموذج اللغة • النشر المحلي • GGUF • Qdrant • bge-m3

Academic Year 2025/2026